Zero to Language

Building the Lateralus Compiler from Scratch

Lateralus Language

bad-antics · April 2026 · Lateralus Language Research

ABSTRACT Lateralus Language -- lateralus.dev

0. Introduction to Zero to Language

Compiler construction sits at the intersection of formal language theory and practical systems engineering. The theoretical foundations -- regular languages for lexing, context-free grammars for parsing, type theory for semantic analysis -- give compiler writers the tools to reason about correctness. The engineering challenges -- performance, error recovery, incremental compilation, cross-platform code generation -- require the tools of systems programming. Lateralus's compiler is implemented in Lateralus itself (bootstrapped from an initial Python implementation), which means the compiler is both a producer and consumer of all the language features it compiles.

The lexer transforms a stream of characters into a stream of tokens. Each token has a type (IDENT, INT_LIT, STRING, PLUS, PIPEFWD, etc.) and a value (the matched text). Lateralus's lexer handles several non-trivial features: string interpolation (parsing 'Hello, ${name}!' into a sequence of string literal, expression, and string literal tokens), multi-line strings (tracking line numbers for accurate error reporting), and the pipeline operator |> (which must be distinguished from the bitwise-or operator | followed by >). The lexer is implemented as a handwritten DFA for maximum performance, achieving about 50MB/s throughput on typical source files.

Pratt parsing, also called top-down operator precedence parsing, is the technique used in Lateralus's expression parser. Each token type is assigned a binding power -- a number representing its precedence. The parser is a loop: parse a primary expression (literal, identifier, parenthesized expression), then repeatedly consume infix operators whose binding power exceeds the current right-binding context. This naturally handles operator precedence and associativity without explicit grammar rules for each precedence level. The Pratt parser for Lateralus handles 25 infix operators, 8 prefix operators, and 5 postfix operators in about 300 lines of code.

Implementation Details

Hindley-Milner type inference, formalized as Algorithm W, is the theoretical foundation for Lateralus's type checker. The algorithm processes expressions in a single pass, generating type constraints (equations between type variables) and then solving them using unification. Unification finds the most general substitution that satisfies all constraints simultaneously. The result is that every expression in a Lateralus program receives its most general type without any programmer-provided annotations. Annotations, when provided, are checked against the inferred type and produce an error if they are more restrictive or less restrictive than the inferred type.

Single Static Assignment (SSA) form is the intermediate representation used in Lateralus's optimizer. In SSA, every variable is assigned exactly once, and uses that occur after a control flow merge point are mediated by phi functions that select the correct value based on which predecessor block was executed. SSA makes data flow explicit: from any use of a variable, you can trace a single chain of definitions to find where the value came from. This property enables efficient implementations of dominance-based optimizations: constant propagation (replace uses of a constant-valued variable with the constant), dead code elimination (remove assignments whose values are never used), and value numbering (identify equivalent expressions).

```
// Lexer token types
enum Token {
    Ident(str), IntLit(int), FloatLit(float), StrLit(str),
    Plus, Minus, Star, Slash, PipeFwd,  // |>
    BitorOp,                            // |
```

```
    Assign, Eq, Neq, Lt, Gt, Le, Ge,
    LParen, RParen, LBrace, RBrace,
    Fn, Let, If, Else, Return, Match, Struct, Enum,
    Eof,
}

fn next_token(src: &str, pos: &mut int) -> Token {
    skip_whitespace(src, pos)
    match src[*pos] {
        '|' => { *pos += 1
                 if src[*pos] == '>' { *pos += 1; return Token.PipeFwd }
                 return Token.BitorOp },
        // ...
    }
}
```

Pass Input Output Time% Incremental

Lexer source text tokens 5% no

Parser tokens CST 8% file

Name res CST resolved AST 4% module

Type check resolved AST typed AST 28% interface

IR gen typed AST SSA IR 5% module

Optimize SSA IR opt IR 15% module

Codegen opt IR machine code 30% module

Link objects binary 5% changed

1. Compiler Architecture Overview

The lexer transforms a stream of characters into a stream of tokens. Each token has a type (IDENT, INT_LIT, STRING, PLUS, PIPEFWD, etc.) and a value (the matched text). Lateralus's lexer handles several non-trivial features: string interpolation (parsing 'Hello, ${name}!' into a sequence of string literal, expression, and string literal tokens), multi-line strings (tracking line numbers for accurate error reporting), and the pipeline operator |> (which must be distinguished from the bitwise-or operator | followed by >). The lexer is implemented as a handwritten DFA for maximum performance, achieving about 50MB/s throughput on typical source files.

Pratt parsing, also called top-down operator precedence parsing, is the technique used in Lateralus's expression parser. Each token type is assigned a binding power -- a number representing its precedence. The parser is a loop: parse a primary expression (literal, identifier, parenthesized expression), then repeatedly consume infix operators whose binding power exceeds the current right-binding context. This naturally handles operator precedence and associativity without explicit grammar rules for each precedence level. The Pratt parser for Lateralus handles 25 infix operators, 8 prefix operators, and 5 postfix operators in about 300 lines of code.

Hindley-Milner type inference, formalized as Algorithm W, is the theoretical foundation for Lateralus's type checker. The algorithm processes expressions in a single pass, generating type constraints (equations between type variables) and then solving them using unification. Unification finds the most general substitution that satisfies all constraints simultaneously. The result is that every expression in a Lateralus program receives its most general type without any programmer-provided annotations. Annotations, when provided, are checked against the inferred type and produce an error if they are

more restrictive or less restrictive than the inferred type.

Implementation Details

Single Static Assignment (SSA) form is the intermediate representation used in Lateralus's optimizer. In SSA, every variable is assigned exactly once, and uses that occur after a control flow merge point are mediated by phi functions that select the correct value based on which predecessor block was executed. SSA makes data flow explicit: from any use of a variable, you can trace a single chain of definitions to find where the value came from. This property enables efficient implementations of dominance-based optimizations: constant propagation (replace uses of a constant-valued variable with the constant), dead code elimination (remove assignments whose values are never used), and value numbering (identify equivalent expressions).

Register allocation is the process of mapping IR variables to physical machine registers. Modern CPUs have 16-32 general-purpose registers; programs may use thousands of variables. When there are more live variables at a point than available registers, some variables must be 'spilled' to the stack. The goal of register allocation is to minimize spills. Lateralus uses the linear scan register allocator for its speed (O(n) in the number of instructions) and the graph-coloring allocator for its quality (minimizing spills for hot paths). The two allocators are applied in sequence: the graph-coloring allocator handles the function body, and linear scan handles critical loops.

```
// Pratt parser: precedence climbing
const PREC: map = {
    "+": 10, "-": 10, "*": 20, "/": 20,
    "|>": 2, "==": 5, "!=": 5, "<": 5, ">": 5,
    ".": 30,  // field access
}

fn parse_expr(p: &Parser, min_prec: int) -> Expr {

    let mut left = parse_primary(p)
    while p.peek().prec() >= min_prec {
        let op = p.consume()
        let right = parse_expr(p, PREC[op.text] + 1)
        left = Expr.BinOp(op.kind, left, right)
    }
    return left
}
```

Pass Input Output Time% Incremental

Lexer source text tokens 5% no

Parser tokens CST 8% file

Name res CST resolved AST 4% module

Type check resolved AST typed AST 28% interface

IR gen typed AST SSA IR 5% module

Optimize SSA IR opt IR 15% module

Codegen opt IR machine code 30% module

Link objects binary 5% changed

2. Lexer: Character Classes and State Machine

Pratt parsing, also called top-down operator precedence parsing, is the technique used in Lateralus's expression parser. Each token type is assigned a binding power -- a number representing its precedence. The parser is a loop: parse a primary expression (literal, identifier, parenthesized expression), then repeatedly consume infix operators whose binding power exceeds the current right-binding context. This naturally handles operator precedence and associativity without explicit grammar rules for each precedence level. The Pratt parser for Lateralus handles 25 infix operators, 8 prefix operators, and 5 postfix operators in about 300 lines of code.

Hindley-Milner type inference, formalized as Algorithm W, is the theoretical foundation for Lateralus's type checker. The algorithm processes expressions in a single pass, generating type constraints (equations between type variables) and then solving them using unification. Unification finds the most general substitution that satisfies all constraints simultaneously. The result is that every expression in a Lateralus program receives its most general type without any programmer-provided annotations. Annotations, when provided, are checked against the inferred type and produce an error if they are more restrictive or less restrictive than the inferred type.

Single Static Assignment (SSA) form is the intermediate representation used in Lateralus's optimizer. In SSA, every variable is assigned exactly once, and uses that occur after a control flow merge point are mediated by phi functions that select the correct value based on which predecessor block was executed. SSA makes data flow explicit: from any use of a variable, you can trace a single chain of definitions to find where the value came from. This property enables efficient implementations of dominance-based optimizations: constant propagation (replace uses of a constant-valued variable with the constant), dead code elimination (remove assignments whose values are never used), and value numbering (identify equivalent expressions).

Implementation Details

Register allocation is the process of mapping IR variables to physical machine registers. Modern CPUs have 16-32 general-purpose registers; programs may use thousands of variables. When there are more live variables at a point than available registers, some variables must be 'spilled' to the stack. The goal of register allocation is to minimize spills. Lateralus uses the linear scan register allocator for its speed (O(n) in the number of instructions) and the graph-coloring allocator for its quality (minimizing spills for hot paths). The two allocators are applied in sequence: the graph-coloring allocator handles the function body, and linear scan handles critical loops.

The Lateralus Bytecode (LBC) format is designed for the Lateralus virtual machine. Instructions are 32-bit words: the top 8 bits identify the opcode, and the remaining 24 bits carry operands (register numbers, constant pool indices, jump offsets). The fixed-width instruction format allows O(1) random access to any instruction, which is important for the JIT compiler's code patching mechanism. The constant pool stores all non-integer literals (floating-point numbers, strings, type descriptors) referenced by instructions. A separate debug section stores source file names, line numbers, and local

variable names; this section can be stripped for production builds.

```
// Hindley-Milner unification
fn unify(t1: Type, t2: Type, subst: &mut Subst) -> Result[(), TypeError] {
    let t1 = apply(subst, t1)
    let t2 = apply(subst, t2)
    match (t1, t2) {
        (Type.Var(a), Type.Var(b)) if a == b => Ok(()),
        (Type.Var(a), t) | (t, Type.Var(a)) => {
            if occurs(a, t) { return Err(TypeError.InfiniteType(a, t)) }
            subst.insert(a, t); Ok(())
```

```
        },
        (Type.App(f1, a1), Type.App(f2, a2)) => {
            unify(f1, f2, subst)?;
            unify(a1, a2, subst)
        },
        _ => Err(TypeError.Mismatch(t1, t2)),
    }
}
```

Pass Input Output Time% Incremental

Lexer source text tokens 5% no

Parser tokens CST 8% file

Name res CST resolved AST 4% module

Type check resolved AST typed AST 28% interface

IR gen typed AST SSA IR 5% module

Optimize SSA IR opt IR 15% module

Codegen opt IR machine code 30% module

Link objects binary 5% changed

3. Lexer: String Interpolation Handling

Hindley-Milner type inference, formalized as Algorithm W, is the theoretical foundation for Lateralus's type checker. The algorithm processes expressions in a single pass, generating type constraints (equations between type variables) and then solving them using unification. Unification finds the most general substitution that satisfies all constraints simultaneously. The result is that every expression in a Lateralus program receives its most general type without any programmer-provided annotations. Annotations, when provided, are checked against the inferred type and produce an error if they are more restrictive or less restrictive than the inferred type.

Single Static Assignment (SSA) form is the intermediate representation used in Lateralus's optimizer. In SSA, every variable is assigned exactly once, and uses that occur after a control flow merge point are mediated by phi functions that select the correct value based on which predecessor block was executed. SSA makes data flow explicit: from any use of a variable, you can trace a single chain of definitions to find where the value came from. This property enables efficient implementations of dominance-based optimizations: constant propagation (replace uses of a constant-valued variable with the constant), dead code elimination (remove assignments whose values are never used), and value numbering (identify equivalent expressions).

Register allocation is the process of mapping IR variables to physical machine registers. Modern CPUs have 16-32 general-purpose registers; programs may use thousands of variables. When there are more live variables at a point than available registers, some variables must be 'spilled' to the stack. The goal of register allocation is to minimize spills. Lateralus uses the linear scan register allocator for its speed (O(n) in the number of instructions) and the graph-coloring allocator for its quality (minimizing spills for hot paths). The two allocators are applied in sequence: the graph-coloring allocator handles the function body, and linear scan handles critical loops.

Implementation Details

The Lateralus Bytecode (LBC) format is designed for the Lateralus virtual machine. Instructions are 32-bit words: the top 8 bits identify the opcode, and the remaining 24 bits carry operands (register numbers, constant pool indices, jump offsets). The fixed-width instruction format allows O(1) random access to any instruction, which is important for the JIT

compiler's code patching mechanism. The constant pool stores all non-integer literals (floating-point numbers, strings, type descriptors) referenced by instructions. A separate debug section stores source file names, line numbers, and local variable names; this section can be stripped for production builds.

Bootstrapping a compiler means writing the compiler in the language it compiles. The process has three stages: Stage 1 is the initial implementation in a different language (Python for Lateralus); Stage 2 compiles the Lateralus compiler source with Stage 1 to produce a binary; Stage 3 re-compiles the Lateralus compiler source with Stage 2 and verifies that the Stage 2 and Stage 3 binaries are identical. This three-stage bootstrap, known as the Trusting Trust test, verifies that the compiler correctly compiles itself. Any discrepancy between Stage 2 and Stage 3 indicates a bug in the compiler's self-compilation path.

```
// SSA IR: a simple example
// Source: fn f(x: int) -> int { if x > 0 { x * 2 } else { -x } }
// SSA IR:
//   block entry(x_0: int):
//     t_0 = cmp_gt x_0, 0
//     branch t_0, then_block, else_block
//   block then_block():
//     t_1 = mul x_0, 2
//     jump merge_block(t_1)
//   block else_block():
//     t_2 = neg x_0
//     jump merge_block(t_2)
//   block merge_block(t_3: int):   // phi node
//     return t_3
```

Pass Input Output Time% Incremental

Lexer source text tokens 5% no

Parser tokens CST 8% file

Name res CST resolved AST 4% module

Type check resolved AST typed AST 28% interface

IR gen typed AST SSA IR 5% module

Optimize SSA IR opt IR 15% module

Codegen opt IR machine code 30% module

Link objects binary 5% changed

4. Lexer: Operator Disambiguation (|> vs |)

Single Static Assignment (SSA) form is the intermediate representation used in Lateralus's optimizer. In SSA, every variable is assigned exactly once, and uses that occur after a control flow merge point are mediated by phi functions that select the correct value based on which predecessor block was executed. SSA makes data flow explicit: from any use of a variable, you can trace a single chain of definitions to find where the value came from. This property enables efficient implementations of dominance-based optimizations: constant propagation (replace uses of a constant-valued variable with the constant), dead code elimination (remove assignments whose values are never used), and value numbering (identify equivalent expressions).

Register allocation is the process of mapping IR variables to physical machine registers. Modern CPUs have 16-32 general-purpose registers; programs may use thousands of variables. When there are more live variables at a point than available registers, some variables must be 'spilled' to the stack. The goal of register allocation is to minimize spills. Lateralus uses the linear scan register allocator for its speed (O(n) in the number of instructions) and the graph-coloring allocator for its quality (minimizing spills for hot paths). The two allocators are applied in sequence: the graph-coloring allocator handles the function body, and linear scan handles critical loops.

The Lateralus Bytecode (LBC) format is designed for the Lateralus virtual machine. Instructions are 32-bit words: the top 8 bits identify the opcode, and the remaining 24 bits carry operands (register numbers, constant pool indices, jump offsets). The fixed-width instruction format allows O(1) random access to any instruction, which is important for the JIT compiler's code patching mechanism. The constant pool stores all non-integer literals (floating-point numbers, strings, type descriptors) referenced by instructions. A separate debug section stores source file names, line numbers, and local

variable names; this section can be stripped for production builds.

Implementation Details

Bootstrapping a compiler means writing the compiler in the language it compiles. The process has three stages: Stage 1 is the initial implementation in a different language (Python for Lateralus); Stage 2 compiles the Lateralus compiler source with Stage 1 to produce a binary; Stage 3 re-compiles the Lateralus compiler source with Stage 2 and verifies that the Stage 2 and Stage 3 binaries are identical. This three-stage bootstrap, known as the Trusting Trust test, verifies that the compiler correctly compiles itself. Any discrepancy between Stage 2 and Stage 3 indicates a bug in the compiler's self-compilation path.

Incremental compilation avoids recompiling the entire program when a single file changes. Lateralus's module system makes incremental compilation tractable: each module has a well-defined interface (the set of exported types and functions), and changes to a module's implementation that do not change its interface do not require recompilation of dependent modules. The compiler maintains a dependency graph and a cache of compiled module interfaces. When a source file changes, the compiler recompiles that module, checks whether its interface changed, and if so, propagates recompilation to dependent modules. For large codebases, incremental compilation reduces rebuild time from minutes to seconds.

```
// LBC bytecode instruction encoding
// Bits: [31:24]=opcode [23:16]=dst [15:8]=src1 [7:0]=src2
const OP_ADD:  u8 = 0x01   // dst = src1 + src2
const OP_SUB:  u8 = 0x02   // dst = src1 - src2
const OP_MUL:  u8 = 0x03   // dst = src1 * src2
const OP_CALL: u8 = 0x10   // call func at const_pool[src1], argc=src2
const OP_RET:  u8 = 0x11   // return src1
const OP_JMP:  u8 = 0x20   // unconditional jump to offset
const OP_JZ:   u8 = 0x21   // jump if src1 == 0
```

```
fn encode(op: u8, dst: u8, s1: u8, s2: u8) -> u32 {
    return (op as u32 << 24) | (dst as u32 << 16) | (s1 as u32 << 8) | s2 as u32
}
```

Pass Input Output Time% Incremental

Lexer source text tokens 5% no

Parser tokens CST 8% file

Name res CST resolved AST 4% module

Type check resolved AST typed AST 28% interface

IR gen typed AST SSA IR 5% module

Optimize SSA IR opt IR 15% module

Codegen opt IR machine code 30% module

Link objects binary 5% changed

5. Lexer: Performance Optimization

Register allocation is the process of mapping IR variables to physical machine registers. Modern CPUs have 16-32 general-purpose registers; programs may use thousands of variables. When there are more live variables at a point than available registers, some variables must be 'spilled' to the stack. The goal of register allocation is to minimize spills. Lateralus uses the linear scan register allocator for its speed (O(n) in the number of instructions) and the graph-coloring allocator for its quality (minimizing spills for hot paths). The two allocators are applied in sequence: the graph-coloring allocator handles the function body, and linear scan handles critical loops.

The Lateralus Bytecode (LBC) format is designed for the Lateralus virtual machine. Instructions are 32-bit words: the top 8 bits identify the opcode, and the remaining 24 bits carry operands (register numbers, constant pool indices, jump offsets). The fixed-width instruction format allows O(1) random access to any instruction, which is important for the JIT compiler's code patching mechanism. The constant pool stores all non-integer literals (floating-point numbers, strings, type descriptors) referenced by instructions. A separate debug section stores source file names, line numbers, and local variable names; this section can be stripped for production builds.

Bootstrapping a compiler means writing the compiler in the language it compiles. The process has three stages: Stage 1 is the initial implementation in a different language (Python for Lateralus); Stage 2 compiles the Lateralus compiler source with Stage 1 to produce a binary; Stage 3 re-compiles the Lateralus compiler source with Stage 2 and verifies that the Stage 2 and Stage 3 binaries are identical. This three-stage bootstrap, known as the Trusting Trust test, verifies that the compiler correctly compiles itself. Any discrepancy between Stage 2 and Stage 3 indicates a bug in the compiler's self-compilation path.

Implementation Details

Incremental compilation avoids recompiling the entire program when a single file changes. Lateralus's module system makes incremental compilation tractable: each module has a well-defined interface (the set of exported types and functions), and changes to a module's implementation that do not change its interface do not require recompilation of dependent modules. The compiler maintains a dependency graph and a cache of compiled module interfaces. When a source file changes, the compiler recompiles that module, checks whether its interface changed, and if so, propagates recompilation to dependent modules. For large codebases, incremental compilation reduces rebuild time from minutes to seconds.

Code generation translates the high-level SSA IR into machine code for a specific target architecture. Lateralus supports four backends: Python (transpilation to Python source, for portability), C (transpilation to C99, for maximum compatibility), RISC-V assembly (for the LateralusOS target and embedded systems), and LBC bytecode (for the Lateralus VM). Each backend performs instruction selection (choosing machine instructions for IR operations), register allocation (mapping IR values to machine registers or stack slots), and instruction scheduling (reordering instructions to avoid pipeline stalls). The C backend is the most commonly used for production deployments, as it leverages mature C compilers like GCC and Clang for the final machine code generation.

```
// Lexer token types
enum Token {
    Ident(str), IntLit(int), FloatLit(float), StrLit(str),
    Plus, Minus, Star, Slash, PipeFwd,  // |>
    BitorOp,                            // |
    Assign, Eq, Neq, Lt, Gt, Le, Ge,
    LParen, RParen, LBrace, RBrace,
    Fn, Let, If, Else, Return, Match, Struct, Enum,
    Eof,
}
```

```
fn next_token(src: &str, pos: &mut int) -> Token {
    skip_whitespace(src, pos)
    match src[*pos] {
        '|' => { *pos += 1
                 if src[*pos] == '>' { *pos += 1; return Token.PipeFwd }
                 return Token.BitorOp },
        // ...
    }
}
```

Pass Input Output Time% Incremental

Lexer source text tokens 5% no

Parser tokens CST 8% file

Name res CST resolved AST 4% module

Type check resolved AST typed AST 28% interface

IR gen typed AST SSA IR 5% module

Optimize SSA IR opt IR 15% module

Codegen opt IR machine code 30% module

Link objects binary 5% changed

6. Parser: Grammar Design

The Lateralus Bytecode (LBC) format is designed for the Lateralus virtual machine. Instructions are 32-bit words: the top

8 bits identify the opcode, and the remaining 24 bits carry operands (register numbers, constant pool indices, jump offsets). The fixed-width instruction format allows O(1) random access to any instruction, which is important for the JIT compiler's code patching mechanism. The constant pool stores all non-integer literals (floating-point numbers, strings, type descriptors) referenced by instructions. A separate debug section stores source file names, line numbers, and local variable names; this section can be stripped for production builds.

Bootstrapping a compiler means writing the compiler in the language it compiles. The process has three stages: Stage 1 is the initial implementation in a different language (Python for Lateralus); Stage 2 compiles the Lateralus compiler source with Stage 1 to produce a binary; Stage 3 re-compiles the Lateralus compiler source with Stage 2 and verifies that the Stage 2 and Stage 3 binaries are identical. This three-stage bootstrap, known as the Trusting Trust test, verifies that the compiler correctly compiles itself. Any discrepancy between Stage 2 and Stage 3 indicates a bug in the compiler's self-compilation path.

Incremental compilation avoids recompiling the entire program when a single file changes. Lateralus's module system makes incremental compilation tractable: each module has a well-defined interface (the set of exported types and functions), and changes to a module's implementation that do not change its interface do not require recompilation of dependent modules. The compiler maintains a dependency graph and a cache of compiled module interfaces. When a source file changes, the compiler recompiles that module, checks whether its interface changed, and if so, propagates recompilation to dependent modules. For large codebases, incremental compilation reduces rebuild time from minutes to seconds.

Implementation Details

Code generation translates the high-level SSA IR into machine code for a specific target architecture. Lateralus supports four backends: Python (transpilation to Python source, for portability), C (transpilation to C99, for maximum compatibility), RISC-V assembly (for the LateralusOS target and embedded systems), and LBC bytecode (for the Lateralus VM). Each backend performs instruction selection (choosing machine instructions for IR operations), register allocation (mapping IR values to machine registers or stack slots), and instruction scheduling (reordering instructions to avoid pipeline stalls). The C backend is the most commonly used for production deployments, as it leverages mature C compilers like GCC and Clang for the final machine code generation.

Compiler construction sits at the intersection of formal language theory and practical systems engineering. The theoretical foundations -- regular languages for lexing, context-free grammars for parsing, type theory for semantic analysis -- give compiler writers the tools to reason about correctness. The engineering challenges -- performance, error recovery, incremental compilation, cross-platform code generation -- require the tools of systems programming. Lateralus's compiler is implemented in Lateralus itself (bootstrapped from an initial Python implementation), which means the compiler is both a producer and consumer of all the language features it compiles.

```
// Pratt parser: precedence climbing
const PREC: map = {
    "+": 10, "-": 10, "*": 20, "/": 20,
    "|>": 2, "==": 5, "!=": 5, "<": 5, ">": 5,
    ".": 30,  // field access
}

fn parse_expr(p: &Parser, min_prec: int) -> Expr {
    let mut left = parse_primary(p)
    while p.peek().prec() >= min_prec {
        let op = p.consume()
        let right = parse_expr(p, PREC[op.text] + 1)
        left = Expr.BinOp(op.kind, left, right)
    }
    return left
}
```

Pass Input Output Time% Incremental

Lexer source text tokens 5% no

Parser tokens CST 8% file

Name res CST resolved AST 4% module

Type check resolved AST typed AST 28% interface

IR gen typed AST SSA IR 5% module

Optimize SSA IR opt IR 15% module

Codegen opt IR machine code 30% module

Link objects binary 5% changed

7. Parser: Recursive Descent

Bootstrapping a compiler means writing the compiler in the language it compiles. The process has three stages: Stage 1 is the initial implementation in a different language (Python for Lateralus); Stage 2 compiles the Lateralus compiler source with Stage 1 to produce a binary; Stage 3 re-compiles the Lateralus compiler source with Stage 2 and verifies that the Stage 2 and Stage 3 binaries are identical. This three-stage bootstrap, known as the Trusting Trust test, verifies that the compiler correctly compiles itself. Any discrepancy between Stage 2 and Stage 3 indicates a bug in the compiler's self-compilation path.

Incremental compilation avoids recompiling the entire program when a single file changes. Lateralus's module system makes incremental compilation tractable: each module has a well-defined interface (the set of exported types and functions), and changes to a module's implementation that do not change its interface do not require recompilation of dependent modules. The compiler maintains a dependency graph and a cache of compiled module interfaces. When a source file changes, the compiler recompiles that module, checks whether its interface changed, and if so, propagates recompilation to dependent modules. For large codebases, incremental compilation reduces rebuild time from minutes to seconds.

Code generation translates the high-level SSA IR into machine code for a specific target architecture. Lateralus supports four backends: Python (transpilation to Python source, for portability), C (transpilation to C99, for maximum compatibility), RISC-V assembly (for the LateralusOS target and embedded systems), and LBC bytecode (for the Lateralus VM). Each backend performs instruction selection (choosing machine instructions for IR operations), register allocation (mapping IR values to machine registers or stack slots), and instruction scheduling (reordering instructions to avoid pipeline stalls). The C backend is the most commonly used for production deployments, as it leverages mature C compilers like GCC and Clang for the final machine code generation.

Implementation Details

Compiler construction sits at the intersection of formal language theory and practical systems engineering. The theoretical foundations -- regular languages for lexing, context-free grammars for parsing, type theory for semantic analysis -- give compiler writers the tools to reason about correctness. The engineering challenges -- performance, error recovery, incremental compilation, cross-platform code generation -- require the tools of systems programming. Lateralus's compiler is implemented in Lateralus itself (bootstrapped from an initial Python implementation), which means the compiler is both a producer and consumer of all the language features it compiles.

The lexer transforms a stream of characters into a stream of tokens. Each token has a type (IDENT, INT_LIT, STRING, PLUS, PIPEFWD, etc.) and a value (the matched text). Lateralus's lexer handles several non-trivial features: string interpolation (parsing 'Hello, ${name}!' into a sequence of string literal, expression, and string literal tokens), multi-line strings (tracking line numbers for accurate error reporting), and the pipeline operator |> (which must be distinguished from the bitwise-or operator | followed by >). The lexer is implemented as a handwritten DFA for maximum performance, achieving

about 50MB/s throughput on typical source files.

```
// Hindley-Milner unification
fn unify(t1: Type, t2: Type, subst: &mut Subst) -> Result[(), TypeError] {
    let t1 = apply(subst, t1)
    let t2 = apply(subst, t2)
    match (t1, t2) {
        (Type.Var(a), Type.Var(b)) if a == b => Ok(()),
        (Type.Var(a), t) | (t, Type.Var(a)) => {
            if occurs(a, t) { return Err(TypeError.InfiniteType(a, t)) }
            subst.insert(a, t); Ok(())
        },
        (Type.App(f1, a1), Type.App(f2, a2)) => {
            unify(f1, f2, subst)?;

            unify(a1, a2, subst)
        },
        _ => Err(TypeError.Mismatch(t1, t2)),
    }
}
```

Pass Input Output Time% Incremental

Lexer source text tokens 5% no

Parser tokens CST 8% file

Name res CST resolved AST 4% module

Type check resolved AST typed AST 28% interface

IR gen typed AST SSA IR 5% module

Optimize SSA IR opt IR 15% module

Codegen opt IR machine code 30% module

Link objects binary 5% changed

8. Parser: Pratt Expression Parsing

Incremental compilation avoids recompiling the entire program when a single file changes. Lateralus's module system makes incremental compilation tractable: each module has a well-defined interface (the set of exported types and functions), and changes to a module's implementation that do not change its interface do not require recompilation of dependent modules. The compiler maintains a dependency graph and a cache of compiled module interfaces. When a source file changes, the compiler recompiles that module, checks whether its interface changed, and if so, propagates recompilation to dependent modules. For large codebases, incremental compilation reduces rebuild time from minutes to seconds.

Code generation translates the high-level SSA IR into machine code for a specific target architecture. Lateralus supports four backends: Python (transpilation to Python source, for portability), C (transpilation to C99, for maximum compatibility), RISC-V assembly (for the LateralusOS target and embedded systems), and LBC bytecode (for the Lateralus VM). Each backend performs instruction selection (choosing machine instructions for IR operations), register allocation (mapping IR values to machine registers or stack slots), and instruction scheduling (reordering instructions to avoid pipeline stalls). The C backend is the most commonly used for production deployments, as it leverages mature C compilers like GCC and Clang for the final machine code generation.

Compiler construction sits at the intersection of formal language theory and practical systems engineering. The theoretical foundations -- regular languages for lexing, context-free grammars for parsing, type theory for semantic analysis -- give compiler writers the tools to reason about correctness. The engineering challenges -- performance, error recovery, incremental compilation, cross-platform code generation -- require the tools of systems programming. Lateralus's compiler is implemented in Lateralus itself (bootstrapped from an initial Python implementation), which means the compiler is both a producer and consumer of all the language features it compiles.

Implementation Details

The lexer transforms a stream of characters into a stream of tokens. Each token has a type (IDENT, INT_LIT, STRING, PLUS, PIPEFWD, etc.) and a value (the matched text). Lateralus's lexer handles several non-trivial features: string interpolation (parsing 'Hello, ${name}!' into a sequence of string literal, expression, and string literal tokens), multi-line strings (tracking line numbers for accurate error reporting), and the pipeline operator |> (which must be distinguished from the bitwise-or operator | followed by >). The lexer is implemented as a handwritten DFA for maximum performance, achieving about 50MB/s throughput on typical source files.

Pratt parsing, also called top-down operator precedence parsing, is the technique used in Lateralus's expression parser. Each token type is assigned a binding power -- a number representing its precedence. The parser is a loop: parse a primary expression (literal, identifier, parenthesized expression), then repeatedly consume infix operators whose binding power exceeds the current right-binding context. This naturally handles operator precedence and associativity without explicit grammar rules for each precedence level. The Pratt parser for Lateralus handles 25 infix operators, 8 prefix operators, and 5 postfix operators in about 300 lines of code.

```
// SSA IR: a simple example

// Source: fn f(x: int) -> int { if x > 0 { x * 2 } else { -x } }
// SSA IR:
//   block entry(x_0: int):
//     t_0 = cmp_gt x_0, 0
//     branch t_0, then_block, else_block
//   block then_block():
//     t_1 = mul x_0, 2
//     jump merge_block(t_1)
//   block else_block():
//     t_2 = neg x_0
//     jump merge_block(t_2)
//   block merge_block(t_3: int):   // phi node
//     return t_3
```

Pass Input Output Time% Incremental

Lexer source text tokens 5% no

Parser tokens CST 8% file

Name res CST resolved AST 4% module

Type check resolved AST typed AST 28% interface

IR gen typed AST SSA IR 5% module

Optimize SSA IR opt IR 15% module

Codegen opt IR machine code 30% module

Link objects binary 5% changed

9. Parser: Error Recovery

Code generation translates the high-level SSA IR into machine code for a specific target architecture. Lateralus supports four backends: Python (transpilation to Python source, for portability), C (transpilation to C99, for maximum compatibility), RISC-V assembly (for the LateralusOS target and embedded systems), and LBC bytecode (for the Lateralus VM). Each backend performs instruction selection (choosing machine instructions for IR operations), register allocation (mapping IR values to machine registers or stack slots), and instruction scheduling (reordering instructions to avoid pipeline stalls). The C backend is the most commonly used for production deployments, as it leverages mature C compilers like GCC and Clang for the final machine code generation.

Compiler construction sits at the intersection of formal language theory and practical systems engineering. The theoretical foundations -- regular languages for lexing, context-free grammars for parsing, type theory for semantic analysis -- give compiler writers the tools to reason about correctness. The engineering challenges -- performance, error recovery, incremental compilation, cross-platform code generation -- require the tools of systems programming. Lateralus's compiler is implemented in Lateralus itself (bootstrapped from an initial Python implementation), which means the compiler is both a producer and consumer of all the language features it compiles.

The lexer transforms a stream of characters into a stream of tokens. Each token has a type (IDENT, INT_LIT, STRING, PLUS, PIPEFWD, etc.) and a value (the matched text). Lateralus's lexer handles several non-trivial features: string interpolation (parsing 'Hello, ${name}!' into a sequence of string literal, expression, and string literal tokens), multi-line strings (tracking line numbers for accurate error reporting), and the pipeline operator |> (which must be distinguished from the bitwise-or operator | followed by >). The lexer is implemented as a handwritten DFA for maximum performance, achieving about 50MB/s throughput on typical source files.

Implementation Details

Pratt parsing, also called top-down operator precedence parsing, is the technique used in Lateralus's expression parser. Each token type is assigned a binding power -- a number representing its precedence. The parser is a loop: parse a primary expression (literal, identifier, parenthesized expression), then repeatedly consume infix operators whose binding power exceeds the current right-binding context. This naturally handles operator precedence and associativity without explicit grammar rules for each precedence level. The Pratt parser for Lateralus handles 25 infix operators, 8 prefix operators, and 5 postfix operators in about 300 lines of code.

Hindley-Milner type inference, formalized as Algorithm W, is the theoretical foundation for Lateralus's type checker. The

algorithm processes expressions in a single pass, generating type constraints (equations between type variables) and then solving them using unification. Unification finds the most general substitution that satisfies all constraints simultaneously. The result is that every expression in a Lateralus program receives its most general type without any programmer-provided annotations. Annotations, when provided, are checked against the inferred type and produce an error if they are more restrictive or less restrictive than the inferred type.

```
// LBC bytecode instruction encoding
// Bits: [31:24]=opcode [23:16]=dst [15:8]=src1 [7:0]=src2
const OP_ADD:  u8 = 0x01   // dst = src1 + src2
const OP_SUB:  u8 = 0x02   // dst = src1 - src2
const OP_MUL:  u8 = 0x03   // dst = src1 * src2
const OP_CALL: u8 = 0x10   // call func at const_pool[src1], argc=src2
const OP_RET:  u8 = 0x11   // return src1
const OP_JMP:  u8 = 0x20   // unconditional jump to offset
```

```
const OP_JZ:   u8 = 0x21   // jump if src1 == 0

fn encode(op: u8, dst: u8, s1: u8, s2: u8) -> u32 {
    return (op as u32 << 24) | (dst as u32 << 16) | (s1 as u32 << 8) | s2 as u32
}
```

Pass Input Output Time% Incremental

Lexer source text tokens 5% no

Parser tokens CST 8% file

Name res CST resolved AST 4% module

Type check resolved AST typed AST 28% interface

IR gen typed AST SSA IR 5% module

Optimize SSA IR opt IR 15% module

Codegen opt IR machine code 30% module

Link objects binary 5% changed

10. AST Design: Nodes and Visitors

Compiler construction sits at the intersection of formal language theory and practical systems engineering. The theoretical foundations -- regular languages for lexing, context-free grammars for parsing, type theory for semantic analysis -- give compiler writers the tools to reason about correctness. The engineering challenges -- performance, error recovery, incremental compilation, cross-platform code generation -- require the tools of systems programming. Lateralus's compiler is implemented in Lateralus itself (bootstrapped from an initial Python implementation), which means the compiler is both a producer and consumer of all the language features it compiles.

The lexer transforms a stream of characters into a stream of tokens. Each token has a type (IDENT, INT_LIT, STRING, PLUS, PIPEFWD, etc.) and a value (the matched text). Lateralus's lexer handles several non-trivial features: string interpolation (parsing 'Hello, ${name}!' into a sequence of string literal, expression, and string literal tokens), multi-line strings (tracking line numbers for accurate error reporting), and the pipeline operator |> (which must be distinguished from the bitwise-or operator | followed by >). The lexer is implemented as a handwritten DFA for maximum performance, achieving about 50MB/s throughput on typical source files.

Pratt parsing, also called top-down operator precedence parsing, is the technique used in Lateralus's expression parser. Each token type is assigned a binding power -- a number representing its precedence. The parser is a loop: parse a primary expression (literal, identifier, parenthesized expression), then repeatedly consume infix operators whose binding power exceeds the current right-binding context. This naturally handles operator precedence and associativity without explicit grammar rules for each precedence level. The Pratt parser for Lateralus handles 25 infix operators, 8 prefix operators, and 5 postfix operators in about 300 lines of code.

Implementation Details

Hindley-Milner type inference, formalized as Algorithm W, is the theoretical foundation for Lateralus's type checker. The algorithm processes expressions in a single pass, generating type constraints (equations between type variables) and then solving them using unification. Unification finds the most general substitution that satisfies all constraints

simultaneously. The result is that every expression in a Lateralus program receives its most general type without any programmer-provided annotations. Annotations, when provided, are checked against the inferred type and produce an error if they are more restrictive or less restrictive than the inferred type.

Single Static Assignment (SSA) form is the intermediate representation used in Lateralus's optimizer. In SSA, every variable is assigned exactly once, and uses that occur after a control flow merge point are mediated by phi functions that select the correct value based on which predecessor block was executed. SSA makes data flow explicit: from any use of a variable, you can trace a single chain of definitions to find where the value came from. This property enables efficient implementations of dominance-based optimizations: constant propagation (replace uses of a constant-valued variable with the constant), dead code elimination (remove assignments whose values are never used), and value numbering (identify equivalent expressions).

```
// Lexer token types
enum Token {
    Ident(str), IntLit(int), FloatLit(float), StrLit(str),
    Plus, Minus, Star, Slash, PipeFwd,  // |>
    BitorOp,                            // |
    Assign, Eq, Neq, Lt, Gt, Le, Ge,
    LParen, RParen, LBrace, RBrace,
    Fn, Let, If, Else, Return, Match, Struct, Enum,
    Eof,
}

fn next_token(src: &str, pos: &mut int) -> Token {
    skip_whitespace(src, pos)
    match src[*pos] {
        '|' => { *pos += 1
                 if src[*pos] == '>' { *pos += 1; return Token.PipeFwd }
                 return Token.BitorOp },
        // ...
    }
}
```

Pass Input Output Time% Incremental

Lexer source text tokens 5% no

Parser tokens CST 8% file

Name res CST resolved AST 4% module

Type check resolved AST typed AST 28% interface

IR gen typed AST SSA IR 5% module

Optimize SSA IR opt IR 15% module

Codegen opt IR machine code 30% module

Link objects binary 5% changed

11. Name Resolution: Scopes and Shadowing

The lexer transforms a stream of characters into a stream of tokens. Each token has a type (IDENT, INT_LIT, STRING, PLUS, PIPEFWD, etc.) and a value (the matched text). Lateralus's lexer handles several non-trivial features: string interpolation (parsing 'Hello, ${name}!' into a sequence of string literal, expression, and string literal tokens), multi-line strings (tracking line numbers for accurate error reporting), and the pipeline operator |> (which must be distinguished from the bitwise-or operator | followed by >). The lexer is implemented as a handwritten DFA for maximum performance, achieving about 50MB/s throughput on typical source files.

Pratt parsing, also called top-down operator precedence parsing, is the technique used in Lateralus's expression parser. Each token type is assigned a binding power -- a number representing its precedence. The parser is a loop: parse a primary expression (literal, identifier, parenthesized expression), then repeatedly consume infix operators whose binding power exceeds the current right-binding context. This naturally handles operator precedence and associativity without explicit grammar rules for each precedence level. The Pratt parser for Lateralus handles 25 infix operators, 8 prefix operators, and 5 postfix operators in about 300 lines of code.

Hindley-Milner type inference, formalized as Algorithm W, is the theoretical foundation for Lateralus's type checker. The algorithm processes expressions in a single pass, generating type constraints (equations between type variables) and then solving them using unification. Unification finds the most general substitution that satisfies all constraints simultaneously. The result is that every expression in a Lateralus program receives its most general type without any programmer-provided annotations. Annotations, when provided, are checked against the inferred type and produce an error if they are more restrictive or less restrictive than the inferred type.

Implementation Details

Single Static Assignment (SSA) form is the intermediate representation used in Lateralus's optimizer. In SSA, every variable is assigned exactly once, and uses that occur after a control flow merge point are mediated by phi functions that select the correct value based on which predecessor block was executed. SSA makes data flow explicit: from any use of a variable, you can trace a single chain of definitions to find where the value came from. This property enables efficient implementations of dominance-based optimizations: constant propagation (replace uses of a constant-valued variable with the constant), dead code elimination (remove assignments whose values are never used), and value numbering (identify equivalent expressions).

Register allocation is the process of mapping IR variables to physical machine registers. Modern CPUs have 16-32 general-purpose registers; programs may use thousands of variables. When there are more live variables at a point than available registers, some variables must be 'spilled' to the stack. The goal of register allocation is to minimize spills. Lateralus uses the linear scan register allocator for its speed (O(n) in the number of instructions) and the graph-coloring allocator for its quality (minimizing spills for hot paths). The two allocators are applied in sequence: the graph-coloring allocator handles the function body, and linear scan handles critical loops.

```
// Pratt parser: precedence climbing
const PREC: map = {
    "+": 10, "-": 10, "*": 20, "/": 20,
    "|>": 2, "==": 5, "!=": 5, "<": 5, ">": 5,
    ".": 30,  // field access
}

fn parse_expr(p: &Parser, min_prec: int) -> Expr {
    let mut left = parse_primary(p)
    while p.peek().prec() >= min_prec {
```

```
        let op = p.consume()
        let right = parse_expr(p, PREC[op.text] + 1)
        left = Expr.BinOp(op.kind, left, right)
    }
    return left
}
```

Pass Input Output Time% Incremental

Lexer source text tokens 5% no

Parser tokens CST 8% file

Name res CST resolved AST 4% module

Type check resolved AST typed AST 28% interface

IR gen typed AST SSA IR 5% module

Optimize SSA IR opt IR 15% module

Codegen opt IR machine code 30% module

Link objects binary 5% changed

12. Name Resolution: Import Resolution

Pratt parsing, also called top-down operator precedence parsing, is the technique used in Lateralus's expression parser. Each token type is assigned a binding power -- a number representing its precedence. The parser is a loop: parse a primary expression (literal, identifier, parenthesized expression), then repeatedly consume infix operators whose binding power exceeds the current right-binding context. This naturally handles operator precedence and associativity without explicit grammar rules for each precedence level. The Pratt parser for Lateralus handles 25 infix operators, 8 prefix

operators, and 5 postfix operators in about 300 lines of code.

Hindley-Milner type inference, formalized as Algorithm W, is the theoretical foundation for Lateralus's type checker. The algorithm processes expressions in a single pass, generating type constraints (equations between type variables) and then solving them using unification. Unification finds the most general substitution that satisfies all constraints simultaneously. The result is that every expression in a Lateralus program receives its most general type without any programmer-provided annotations. Annotations, when provided, are checked against the inferred type and produce an error if they are more restrictive or less restrictive than the inferred type.

Single Static Assignment (SSA) form is the intermediate representation used in Lateralus's optimizer. In SSA, every variable is assigned exactly once, and uses that occur after a control flow merge point are mediated by phi functions that select the correct value based on which predecessor block was executed. SSA makes data flow explicit: from any use of a variable, you can trace a single chain of definitions to find where the value came from. This property enables efficient implementations of dominance-based optimizations: constant propagation (replace uses of a constant-valued variable with the constant), dead code elimination (remove assignments whose values are never used), and value numbering (identify equivalent expressions).

Implementation Details

Register allocation is the process of mapping IR variables to physical machine registers. Modern CPUs have 16-32 general-purpose registers; programs may use thousands of variables. When there are more

live variables at a point than available registers, some variables must be 'spilled' to the stack. The goal of register allocation is to minimize spills. Lateralus uses the linear scan register allocator for its speed (O(n) in the number of instructions) and the graph-coloring allocator for its quality (minimizing spills for hot paths). The two allocators are applied in sequence: the graph-coloring allocator handles the function body, and linear scan handles critical loops.

The Lateralus Bytecode (LBC) format is designed for the Lateralus virtual machine. Instructions are 32-bit words: the top 8 bits identify the opcode, and the remaining 24 bits carry operands (register numbers, constant pool indices, jump offsets). The fixed-width instruction format allows O(1) random access to any instruction, which is important for the JIT compiler's code patching mechanism. The constant pool stores all non-integer literals (floating-point numbers, strings, type descriptors) referenced by instructions. A separate debug section stores source file names, line numbers, and local variable names; this section can be stripped for production builds.

```
// Hindley-Milner unification
fn unify(t1: Type, t2: Type, subst: &mut Subst) -> Result[(), TypeError] {
    let t1 = apply(subst, t1)
    let t2 = apply(subst, t2)
    match (t1, t2) {
        (Type.Var(a), Type.Var(b)) if a == b => Ok(()),
        (Type.Var(a), t) | (t, Type.Var(a)) => {
            if occurs(a, t) { return Err(TypeError.InfiniteType(a, t)) }
            subst.insert(a, t); Ok(())
        },
        (Type.App(f1, a1), Type.App(f2, a2)) => {
            unify(f1, f2, subst)?;
            unify(a1, a2, subst)
        },
        _ => Err(TypeError.Mismatch(t1, t2)),
    }
}
```

Pass Input Output Time% Incremental

Lexer source text tokens 5% no

Parser tokens CST 8% file

Name res CST resolved AST 4% module

Type check resolved AST typed AST 28% interface

IR gen typed AST SSA IR 5% module

Optimize SSA IR opt IR 15% module

Codegen opt IR machine code 30% module

Link objects binary 5% changed

13. Name Resolution: Forward References

Hindley-Milner type inference, formalized as Algorithm W, is the theoretical foundation for Lateralus's type checker. The algorithm processes expressions in a single pass, generating type constraints (equations between type variables) and then solving them using unification. Unification finds the most general substitution that satisfies all constraints simultaneously. The result is that every expression in a Lateralus program receives its most general type without any programmer-provided annotations. Annotations, when provided, are checked against the inferred type and produce an error if they are more restrictive or less restrictive than the inferred type.

Single Static Assignment (SSA) form is the intermediate representation used in Lateralus's optimizer. In SSA, every variable is assigned exactly once, and uses that occur after a control flow merge point are mediated by phi functions that select the correct value based on which predecessor block was executed. SSA makes data flow explicit: from any use of a variable, you can trace a single chain of definitions to find where the value came from. This property enables efficient implementations of dominance-based optimizations: constant propagation (replace uses of a constant-valued variable with the constant), dead code elimination (remove assignments whose values are never used), and value numbering (identify equivalent expressions).

Register allocation is the process of mapping IR variables to physical machine registers. Modern CPUs have 16-32 general-purpose registers; programs may use thousands of variables. When there are more live variables at a point than available registers, some variables must be 'spilled' to the stack. The goal of register allocation is to minimize spills. Lateralus uses the linear scan register allocator for its speed (O(n) in the number of instructions) and the graph-coloring allocator for its quality (minimizing spills for hot paths). The two allocators are applied in sequence: the graph-coloring allocator handles the function body, and linear scan handles critical loops.

Implementation Details

The Lateralus Bytecode (LBC) format is designed for the Lateralus virtual machine. Instructions are 32-bit words: the top 8 bits identify the opcode, and the remaining 24 bits carry operands (register numbers, constant pool indices, jump offsets). The fixed-width instruction format allows O(1) random access to any instruction, which is important for the JIT compiler's code patching mechanism. The constant pool stores all non-integer literals (floating-point numbers, strings, type descriptors) referenced by instructions. A separate debug section stores source file names, line numbers, and local variable names; this section can be stripped for production builds.

Bootstrapping a compiler means writing the compiler in the language it compiles. The process has three stages: Stage 1 is the initial implementation in a different language (Python for Lateralus); Stage 2 compiles the Lateralus compiler source with Stage 1 to produce a binary; Stage 3 re-compiles the Lateralus compiler source with Stage 2 and verifies that the Stage 2 and Stage 3 binaries are identical. This three-stage bootstrap, known as the Trusting Trust test, verifies that the compiler correctly compiles itself. Any discrepancy between Stage 2 and Stage 3 indicates a bug in the compiler's self-compilation path.

```
// SSA IR: a simple example
// Source: fn f(x: int) -> int { if x > 0 { x * 2 } else { -x } }
// SSA IR:
//   block entry(x_0: int):
//     t_0 = cmp_gt x_0, 0
//     branch t_0, then_block, else_block
//   block then_block():
//     t_1 = mul x_0, 2
//     jump merge_block(t_1)
//   block else_block():
```

```
//     t_2 = neg x_0
//     jump merge_block(t_2)
//   block merge_block(t_3: int):   // phi node
//     return t_3
```

Pass Input Output Time% Incremental

Lexer source text tokens 5% no

Parser tokens CST 8% file

Name res CST resolved AST 4% module

Type check resolved AST typed AST 28% interface

IR gen typed AST SSA IR 5% module

Optimize SSA IR opt IR 15% module

Codegen opt IR machine code 30% module

Link objects binary 5% changed

14. Type System: Kinds and Sorts

Single Static Assignment (SSA) form is the intermediate representation used in Lateralus's optimizer. In SSA, every variable is assigned exactly once, and uses that occur after a control flow merge point are mediated by phi functions that select the correct value based on which predecessor block was executed. SSA makes data flow explicit: from any use of a variable, you can trace a single chain of definitions to find where the value came from. This property enables efficient implementations of dominance-based optimizations: constant propagation (replace uses of a constant-valued variable with the constant), dead code elimination (remove assignments whose values are never used), and value numbering (identify equivalent expressions).

Register allocation is the process of mapping IR variables to physical machine registers. Modern CPUs have 16-32 general-purpose registers; programs may use thousands of variables. When there are more live variables at a point than available registers, some variables must be 'spilled' to the stack. The goal of register allocation is to minimize spills. Lateralus uses the linear scan register allocator for its speed (O(n) in the number of instructions) and the graph-coloring allocator for its quality (minimizing spills for hot paths). The two allocators are applied in sequence: the graph-coloring allocator handles the function body, and linear scan handles critical loops.

The Lateralus Bytecode (LBC) format is designed for the Lateralus virtual machine. Instructions are 32-bit words: the top 8 bits identify the opcode, and the remaining 24 bits carry operands (register numbers, constant pool indices, jump offsets). The fixed-width instruction format allows O(1) random access to any instruction, which is important for the JIT compiler's code patching mechanism. The constant pool stores all non-integer literals (floating-point numbers, strings, type descriptors) referenced by instructions. A separate debug section stores source file names, line numbers, and local variable names; this section can be stripped for production builds.

Implementation Details

Bootstrapping a compiler means writing the compiler in the language it compiles. The process has three stages: Stage 1 is the initial implementation in a different language (Python for Lateralus); Stage 2 compiles the Lateralus compiler source with Stage 1 to produce a binary; Stage 3 re-compiles the Lateralus compiler source with Stage 2 and verifies that the Stage 2 and Stage 3 binaries are identical. This three-stage bootstrap, known as the Trusting Trust test, verifies that the compiler correctly

compiles itself. Any discrepancy between Stage 2 and Stage 3 indicates a bug in the compiler's self-compilation path.

Incremental compilation avoids recompiling the entire program when a single file changes. Lateralus's module system makes incremental compilation tractable: each module has a well-defined interface (the set of exported types and functions), and changes to a module's implementation that do not change its interface do not require recompilation of dependent modules. The compiler maintains a dependency graph and a cache of compiled module interfaces. When a source file changes, the compiler recompiles that module, checks whether its interface changed, and if so, propagates recompilation to dependent modules. For large codebases, incremental compilation reduces rebuild time from minutes to seconds.

```
// LBC bytecode instruction encoding
// Bits: [31:24]=opcode [23:16]=dst [15:8]=src1 [7:0]=src2
const OP_ADD:  u8 = 0x01   // dst = src1 + src2
const OP_SUB:  u8 = 0x02   // dst = src1 - src2
const OP_MUL:  u8 = 0x03   // dst = src1 * src2
const OP_CALL: u8 = 0x10   // call func at const_pool[src1], argc=src2
const OP_RET:  u8 = 0x11   // return src1
const OP_JMP:  u8 = 0x20   // unconditional jump to offset
const OP_JZ:   u8 = 0x21   // jump if src1 == 0

fn encode(op: u8, dst: u8, s1: u8, s2: u8) -> u32 {
    return (op as u32 << 24) | (dst as u32 << 16) | (s1 as u32 << 8) | s2 as u32
}
```

Pass Input Output Time% Incremental

Lexer source text tokens 5% no

Parser tokens CST 8% file

Name res CST resolved AST 4% module

Type check resolved AST typed AST 28% interface

IR gen typed AST SSA IR 5% module

Optimize SSA IR opt IR 15% module

Codegen opt IR machine code 30% module

Link objects binary 5% changed

15. Type Inference: Algorithm W

Register allocation is the process of mapping IR variables to physical machine registers. Modern CPUs have 16-32 general-purpose registers; programs may use thousands of variables. When there are more live variables at a point than available registers, some variables must be 'spilled' to the stack. The goal of register allocation is to minimize spills. Lateralus uses the linear scan register allocator for its speed (O(n) in the number of instructions) and the graph-coloring allocator for its quality (minimizing spills for hot paths). The two allocators are applied in sequence: the graph-coloring allocator handles the function body, and linear scan handles critical loops.

The Lateralus Bytecode (LBC) format is designed for the Lateralus virtual machine. Instructions are 32-bit words: the top 8 bits identify the opcode, and the remaining 24 bits carry operands (register numbers, constant pool indices, jump offsets). The fixed-width instruction format allows O(1) random access to any instruction, which is important for the JIT compiler's code patching mechanism. The constant pool stores all non-integer literals (floating-point numbers, strings, type descriptors) referenced

by instructions. A separate debug section stores source file names, line numbers, and local variable names; this section can be stripped for production builds.

Bootstrapping a compiler means writing the compiler in the language it compiles. The process has three stages: Stage 1 is the initial implementation in a different language (Python for Lateralus); Stage 2 compiles the Lateralus compiler source with Stage 1 to produce a binary; Stage 3 re-compiles the Lateralus compiler source with Stage 2 and verifies that the Stage 2 and Stage 3 binaries are identical. This three-stage bootstrap, known as the Trusting Trust test, verifies that the compiler correctly compiles itself. Any discrepancy between Stage 2 and Stage 3 indicates a bug in the compiler's self-compilation path.

Implementation Details

Incremental compilation avoids recompiling the entire program when a single file changes. Lateralus's module system makes incremental compilation tractable: each module has a well-defined interface (the set of exported types and functions), and changes to a module's implementation that do not change its interface do not require recompilation of dependent modules. The compiler maintains a dependency graph and a cache of compiled module interfaces. When a source file changes, the compiler recompiles that module, checks whether its interface changed, and if so, propagates recompilation to dependent modules. For large codebases, incremental compilation reduces rebuild time from minutes to seconds.

Code generation translates the high-level SSA IR into machine code for a specific target architecture. Lateralus supports four backends: Python (transpilation to Python source, for portability), C (transpilation to C99, for maximum compatibility), RISC-V assembly (for the LateralusOS target and embedded systems), and LBC bytecode (for the Lateralus VM). Each backend performs instruction selection (choosing machine instructions for IR operations), register allocation (mapping IR values to machine registers or stack slots), and instruction scheduling (reordering instructions to avoid pipeline stalls). The C backend is the most commonly used for production deployments, as it leverages mature C compilers like GCC and Clang for the final machine code generation.

```
// Lexer token types
enum Token {
    Ident(str), IntLit(int), FloatLit(float), StrLit(str),

    Plus, Minus, Star, Slash, PipeFwd,  // |>
    BitorOp,                            // |
    Assign, Eq, Neq, Lt, Gt, Le, Ge,
    LParen, RParen, LBrace, RBrace,
    Fn, Let, If, Else, Return, Match, Struct, Enum,
    Eof,
}

fn next_token(src: &str, pos: &mut int) -> Token {
    skip_whitespace(src, pos)
    match src[*pos] {
        '|' => { *pos += 1
                 if src[*pos] == '>' { *pos += 1; return Token.PipeFwd }
                 return Token.BitorOp },
        // ...
    }
}
```

Pass Input Output Time% Incremental

Lexer source text tokens 5% no

Parser tokens CST 8% file

Name res CST resolved AST 4% module

Type check resolved AST typed AST 28% interface

IR gen typed AST SSA IR 5% module

Optimize SSA IR opt IR 15% module

Codegen opt IR machine code 30% module

Link objects binary 5% changed

16. Type Inference: Unification

The Lateralus Bytecode (LBC) format is designed for the Lateralus virtual machine. Instructions are 32-bit words: the top 8 bits identify the opcode, and the remaining 24 bits carry operands (register numbers, constant pool indices, jump offsets). The fixed-width instruction format allows O(1) random access to any instruction, which is important for the JIT compiler's code patching mechanism. The constant pool stores all non-integer literals (floating-point numbers, strings, type descriptors) referenced by instructions. A separate debug section stores source file names, line numbers, and local variable names; this section can be stripped for production builds.

Bootstrapping a compiler means writing the compiler in the language it compiles. The process has three stages: Stage 1 is the initial implementation in a different language (Python for Lateralus); Stage 2 compiles the Lateralus compiler source with Stage 1 to produce a binary; Stage 3 re-compiles the Lateralus compiler source with Stage 2 and verifies that the Stage 2 and Stage 3 binaries are identical. This three-stage bootstrap, known as the Trusting Trust test, verifies that the compiler correctly compiles itself. Any discrepancy between Stage 2 and Stage 3 indicates a bug in the compiler's self-compilation path.

Incremental compilation avoids recompiling the entire program when a single file changes. Lateralus's module system makes incremental compilation tractable: each module has a well-defined interface (the set of exported types and functions), and changes to a module's implementation that do not change its interface do not require recompilation of dependent modules. The compiler maintains a dependency graph and a cache of compiled module interfaces. When a source file changes, the compiler recompiles that module, checks whether its interface changed, and if so, propagates recompilation to dependent modules. For large codebases, incremental compilation reduces rebuild time from minutes to seconds.

Implementation Details

Code generation translates the high-level SSA IR into machine code for a specific target architecture. Lateralus supports four backends: Python (transpilation to Python source, for portability), C (transpilation to C99, for maximum compatibility), RISC-V assembly (for the LateralusOS target and embedded systems), and LBC bytecode (for the Lateralus VM). Each backend performs instruction selection (choosing machine instructions for IR operations), register

allocation (mapping IR values to machine registers or stack slots), and instruction scheduling (reordering instructions to avoid pipeline stalls). The C backend is the most commonly used for production deployments, as it leverages mature C compilers like GCC and Clang for the final machine code generation.

Compiler construction sits at the intersection of formal language theory and practical systems engineering. The theoretical foundations -- regular languages for lexing, context-free grammars for parsing, type theory for semantic analysis -- give compiler writers the tools to reason about correctness. The engineering challenges -- performance, error recovery, incremental compilation, cross-platform

code generation -- require the tools of systems programming. Lateralus's compiler is implemented in Lateralus itself (bootstrapped from an initial Python implementation), which means the compiler is both a producer and consumer of all the language features it compiles.

```
// Pratt parser: precedence climbing
const PREC: map = {
    "+": 10, "-": 10, "*": 20, "/": 20,
    "|>": 2, "==": 5, "!=": 5, "<": 5, ">": 5,
    ".": 30,  // field access
}

fn parse_expr(p: &Parser, min_prec: int) -> Expr {
    let mut left = parse_primary(p)
    while p.peek().prec() >= min_prec {
        let op = p.consume()
        let right = parse_expr(p, PREC[op.text] + 1)
        left = Expr.BinOp(op.kind, left, right)
    }
    return left
}
```

Pass Input Output Time% Incremental

Lexer source text tokens 5% no

Parser tokens CST 8% file

Name res CST resolved AST 4% module

Type check resolved AST typed AST 28% interface

IR gen typed AST SSA IR 5% module

Optimize SSA IR opt IR 15% module

Codegen opt IR machine code 30% module

Link objects binary 5% changed

17. Type Inference: Occurs Check

Bootstrapping a compiler means writing the compiler in the language it compiles. The process has three stages: Stage 1 is the initial implementation in a different language (Python for Lateralus); Stage 2 compiles the Lateralus compiler source with Stage 1 to produce a binary; Stage 3 re-compiles the Lateralus compiler source with Stage 2 and verifies that the Stage 2 and Stage 3 binaries are identical. This three-stage bootstrap, known as the Trusting Trust test, verifies that the compiler correctly compiles itself. Any discrepancy between Stage 2 and Stage 3 indicates a bug in the compiler's self-compilation path.

Incremental compilation avoids recompiling the entire program when a single file changes. Lateralus's module system makes incremental compilation tractable: each module has a well-defined interface (the set of exported types and functions), and changes to a module's implementation that do not change its interface do not require recompilation of dependent modules. The compiler maintains a dependency graph and a cache of compiled module interfaces. When a source file changes, the compiler recompiles that module, checks whether its interface changed, and if so, propagates recompilation to dependent modules. For large codebases, incremental compilation reduces rebuild time from minutes to seconds.

Code generation translates the high-level SSA IR into machine code for a specific target architecture. Lateralus supports four backends: Python (transpilation to Python source, for portability), C (transpilation to C99, for maximum compatibility), RISC-V assembly (for the LateralusOS target and embedded systems), and LBC bytecode (for the Lateralus VM). Each backend performs instruction selection (choosing machine instructions for IR operations), register

allocation (mapping IR values to machine registers or stack slots), and instruction scheduling (reordering instructions to avoid pipeline stalls). The C backend is the most commonly used for production deployments, as it leverages mature C compilers like GCC and Clang for the final machine code generation.

Implementation Details

Compiler construction sits at the intersection of formal language theory and practical systems engineering. The theoretical foundations -- regular languages for lexing, context-free grammars for parsing, type theory for semantic analysis -- give compiler writers the tools to reason about correctness. The engineering challenges -- performance, error recovery, incremental compilation, cross-platform code generation -- require the tools of systems programming. Lateralus's compiler is implemented in Lateralus itself (bootstrapped from an initial Python implementation), which means the compiler is both a producer and consumer of all the language features it compiles.

The lexer transforms a stream of characters into a stream of tokens. Each token has a type (IDENT, INT_LIT, STRING, PLUS, PIPEFWD, etc.) and a value (the matched text). Lateralus's lexer handles several non-trivial features: string interpolation (parsing 'Hello, ${name}!' into a sequence of string literal, expression, and string literal tokens), multi-line strings (tracking line numbers for accurate error reporting), and the pipeline operator |> (which must be distinguished from the bitwise-or operator | followed by >). The lexer is implemented as a handwritten DFA for maximum performance, achieving about 50MB/s throughput on typical source files.

```
// Hindley-Milner unification
fn unify(t1: Type, t2: Type, subst: &mut Subst) -> Result[(), TypeError] {
    let t1 = apply(subst, t1)
    let t2 = apply(subst, t2)
    match (t1, t2) {
        (Type.Var(a), Type.Var(b)) if a == b => Ok(()),
        (Type.Var(a), t) | (t, Type.Var(a)) => {
            if occurs(a, t) { return Err(TypeError.InfiniteType(a, t)) }
            subst.insert(a, t); Ok(())
        },
        (Type.App(f1, a1), Type.App(f2, a2)) => {
            unify(f1, f2, subst)?;
            unify(a1, a2, subst)
        },
        _ => Err(TypeError.Mismatch(t1, t2)),
    }
}
```

Pass Input Output Time% Incremental

Lexer source text tokens 5% no

Parser tokens CST 8% file

Name res CST resolved AST 4% module

Type check resolved AST typed AST 28% interface

IR gen typed AST SSA IR 5% module

Optimize SSA IR opt IR 15% module

Codegen opt IR machine code 30% module

Link objects binary 5% changed

18. Type Inference: Let Polymorphism

Incremental compilation avoids recompiling the entire program when a single file changes. Lateralus's module system makes incremental compilation tractable: each module has a well-defined interface (the set of exported types and functions), and changes to a module's implementation that do not change its interface do not require recompilation of dependent modules. The compiler maintains a dependency graph and a cache of compiled module interfaces. When a source file changes, the compiler recompiles that module, checks whether its interface changed, and if so, propagates recompilation to dependent modules. For large codebases, incremental compilation reduces rebuild time from minutes to seconds.

Code generation translates the high-level SSA IR into machine code for a specific target architecture. Lateralus supports

four backends: Python (transpilation to Python source, for portability), C (transpilation to C99, for maximum compatibility), RISC-V assembly (for the LateralusOS target and embedded systems), and LBC bytecode (for the Lateralus VM). Each backend performs instruction selection (choosing machine instructions for IR operations), register allocation (mapping IR values to machine registers or stack slots), and instruction scheduling (reordering instructions to avoid pipeline stalls). The C backend is the most commonly used for production deployments, as it leverages mature C compilers like GCC and Clang for the final machine code generation.

Compiler construction sits at the intersection of formal language theory and practical systems engineering. The theoretical foundations -- regular languages for lexing, context-free grammars for parsing, type theory for semantic analysis -- give compiler writers the tools to reason about correctness. The engineering challenges -- performance, error recovery, incremental compilation, cross-platform code generation -- require the tools of systems programming. Lateralus's compiler is implemented in Lateralus itself (bootstrapped from an initial Python implementation), which means the compiler is both a producer and consumer of all the language features it compiles.

Implementation Details

The lexer transforms a stream of characters into a stream of tokens. Each token has a type (IDENT, INT_LIT, STRING, PLUS, PIPEFWD, etc.) and a value (the matched text). Lateralus's lexer handles several non-trivial features: string interpolation (parsing 'Hello, ${name}!' into a sequence of string literal, expression, and string literal tokens), multi-line strings (tracking line numbers for accurate error reporting), and the pipeline operator |> (which must be distinguished from the bitwise-or operator | followed by >). The lexer is implemented as a handwritten DFA for maximum performance, achieving about 50MB/s throughput on typical source files.

Pratt parsing, also called top-down operator precedence parsing, is the technique used in Lateralus's expression parser. Each token type is assigned a binding power -- a number representing its precedence. The parser is a loop: parse a primary expression (literal, identifier, parenthesized expression), then repeatedly consume infix operators whose binding power exceeds the current right-binding context. This naturally handles operator precedence and associativity without explicit grammar rules for each precedence level. The Pratt parser for Lateralus handles 25 infix operators, 8 prefix operators, and 5 postfix operators in about 300 lines of code.

```
// SSA IR: a simple example
// Source: fn f(x: int) -> int { if x > 0 { x * 2 } else { -x } }
```

```
// SSA IR:
//   block entry(x_0: int):
//     t_0 = cmp_gt x_0, 0
//     branch t_0, then_block, else_block
//   block then_block():
//     t_1 = mul x_0, 2
//     jump merge_block(t_1)
//   block else_block():
//     t_2 = neg x_0
//     jump merge_block(t_2)
//   block merge_block(t_3: int):   // phi node
//     return t_3
```

Pass Input Output Time% Incremental

Lexer source text tokens 5% no

Parser tokens CST 8% file

Name res CST resolved AST 4% module

Type check resolved AST typed AST 28% interface

IR gen typed AST SSA IR 5% module

Optimize SSA IR opt IR 15% module

Codegen opt IR machine code 30% module

Link objects binary 5% changed

19. Type Inference: Bidirectional Checking

Code generation translates the high-level SSA IR into machine code for a specific target architecture. Lateralus supports

four backends: Python (transpilation to Python source, for portability), C (transpilation to C99, for maximum compatibility), RISC-V assembly (for the LateralusOS target and embedded systems), and LBC bytecode (for the Lateralus VM). Each backend performs instruction selection (choosing machine instructions for IR operations), register allocation (mapping IR values to machine registers or stack slots), and instruction scheduling (reordering instructions to avoid pipeline stalls). The C backend is the most commonly used for production deployments, as it leverages mature C compilers like GCC and Clang for the final machine code generation.

Compiler construction sits at the intersection of formal language theory and practical systems engineering. The theoretical foundations -- regular languages for lexing, context-free grammars for parsing, type theory for semantic analysis -- give compiler writers the tools to reason about correctness. The engineering challenges -- performance, error recovery, incremental compilation, cross-platform code generation -- require the tools of systems programming. Lateralus's compiler is implemented in Lateralus itself (bootstrapped from an initial Python implementation), which means the compiler is both a producer and consumer of all the language features it compiles.

The lexer transforms a stream of characters into a stream of tokens. Each token has a type (IDENT, INT_LIT, STRING, PLUS, PIPEFWD, etc.) and a value (the matched text). Lateralus's lexer handles several non-trivial features: string interpolation (parsing 'Hello, ${name}!' into a sequence of string literal, expression, and string literal tokens), multi-line strings (tracking line numbers for accurate error reporting), and the pipeline operator |> (which must be distinguished from the bitwise-or operator | followed by >). The lexer is implemented as a handwritten DFA for maximum performance, achieving

about 50MB/s throughput on typical source files.

Implementation Details

Pratt parsing, also called top-down operator precedence parsing, is the technique used in Lateralus's expression parser. Each token type is assigned a binding power -- a number representing its precedence. The parser is a loop: parse a primary expression (literal, identifier, parenthesized expression), then repeatedly consume infix operators whose binding power exceeds the current right-binding context. This naturally handles operator precedence and associativity without explicit grammar rules for each precedence level. The Pratt parser for Lateralus handles 25 infix operators, 8 prefix operators, and 5 postfix operators in about 300 lines of code.

Hindley-Milner type inference, formalized as Algorithm W, is the theoretical foundation for Lateralus's type checker. The algorithm processes expressions in a single pass, generating type constraints (equations between type variables) and then solving them using unification. Unification finds the most general substitution that satisfies all constraints simultaneously. The result is that every expression in a Lateralus program receives its most general type without any programmer-provided annotations. Annotations, when provided, are checked against the inferred type and produce an error if they are more restrictive or less restrictive than the inferred type.

```
// LBC bytecode instruction encoding
// Bits: [31:24]=opcode [23:16]=dst [15:8]=src1 [7:0]=src2
const OP_ADD:  u8 = 0x01   // dst = src1 + src2
const OP_SUB:  u8 = 0x02   // dst = src1 - src2
const OP_MUL:  u8 = 0x03   // dst = src1 * src2
const OP_CALL: u8 = 0x10   // call func at const_pool[src1], argc=src2
const OP_RET:  u8 = 0x11   // return src1
const OP_JMP:  u8 = 0x20   // unconditional jump to offset
const OP_JZ:   u8 = 0x21   // jump if src1 == 0

fn encode(op: u8, dst: u8, s1: u8, s2: u8) -> u32 {
    return (op as u32 << 24) | (dst as u32 << 16) | (s1 as u32 << 8) | s2 as u32
}
```

Pass Input Output Time% Incremental

Lexer source text tokens 5% no

Parser tokens CST 8% file

Name res CST resolved AST 4% module

Type check resolved AST typed AST 28% interface

IR gen typed AST SSA IR 5% module

Optimize SSA IR opt IR 15% module

Codegen opt IR machine code 30% module

Link objects binary 5% changed

20. Type Checking: Trait Bounds

Compiler construction sits at the intersection of formal language theory and practical systems engineering. The theoretical foundations -- regular languages for lexing, context-free grammars for parsing, type theory for semantic analysis -- give compiler writers the tools to reason about correctness. The engineering challenges -- performance, error recovery, incremental compilation, cross-platform code generation -- require the tools of systems programming. Lateralus's compiler is implemented in Lateralus itself (bootstrapped from an initial Python implementation), which means the compiler is both a producer and consumer of all the language features it compiles.

The lexer transforms a stream of characters into a stream of tokens. Each token has a type (IDENT, INT_LIT, STRING, PLUS, PIPEFWD, etc.) and a value (the matched text). Lateralus's lexer handles several non-trivial features: string interpolation (parsing 'Hello, ${name}!' into a sequence of string literal, expression, and string literal tokens), multi-line strings (tracking line numbers for accurate error reporting), and the pipeline operator |> (which must be distinguished from the bitwise-or operator | followed by >). The lexer is implemented as a handwritten DFA for maximum performance, achieving about 50MB/s throughput on typical source files.

Pratt parsing, also called top-down operator precedence parsing, is the technique used in Lateralus's expression parser. Each token type is assigned a binding power -- a number representing its precedence. The parser is a loop: parse a primary expression (literal, identifier, parenthesized expression), then repeatedly consume infix operators whose binding power exceeds the current right-binding context. This naturally handles operator precedence and associativity without explicit grammar rules for each precedence level. The Pratt parser for Lateralus handles 25 infix operators, 8 prefix operators, and 5 postfix operators in about 300 lines of code.

Implementation Details

Hindley-Milner type inference, formalized as Algorithm W, is the theoretical foundation for Lateralus's type checker. The algorithm processes expressions in a single pass, generating type constraints (equations between type variables) and then solving them using unification. Unification finds the most general substitution that satisfies all constraints simultaneously. The result is that every expression in a Lateralus program receives its most general type without any programmer-provided annotations. Annotations, when provided, are checked against the inferred type and produce an error if they are more restrictive or less restrictive than the inferred type.

Single Static Assignment (SSA) form is the intermediate representation used in Lateralus's optimizer. In SSA, every variable is assigned exactly once, and uses that occur after a control flow merge point are mediated by phi functions that select the correct value based on which predecessor block was executed. SSA makes data flow explicit: from any use of a variable, you can trace a single chain of definitions to find where the value came from. This property enables efficient implementations of dominance-based optimizations: constant propagation (replace uses of a constant-valued variable with the constant), dead code elimination (remove assignments whose values are never used), and value numbering (identify equivalent expressions).

```
// Lexer token types
enum Token {
    Ident(str), IntLit(int), FloatLit(float), StrLit(str),
    Plus, Minus, Star, Slash, PipeFwd,  // |>
    BitorOp,                            // |
    Assign, Eq, Neq, Lt, Gt, Le, Ge,
    LParen, RParen, LBrace, RBrace,
    Fn, Let, If, Else, Return, Match, Struct, Enum,
    Eof,
}
```

```
fn next_token(src: &str, pos: &mut int) -> Token {
    skip_whitespace(src, pos)
    match src[*pos] {
        '|' => { *pos += 1
                 if src[*pos] == '>' { *pos += 1; return Token.PipeFwd }
                 return Token.BitorOp },
        // ...

    }
}
```

Pass Input Output Time% Incremental

Lexer source text tokens 5% no

Parser tokens CST 8% file

Name res CST resolved AST 4% module

Type check resolved AST typed AST 28% interface

IR gen typed AST SSA IR 5% module

Optimize SSA IR opt IR 15% module

Codegen opt IR machine code 30% module

Link objects binary 5% changed

21. Type Checking: Pattern Matching Exhaustiveness

The lexer transforms a stream of characters into a stream of tokens. Each token has a type (IDENT, INT_LIT, STRING, PLUS, PIPEFWD, etc.) and a value (the matched text). Lateralus's lexer handles several non-trivial features: string interpolation (parsing 'Hello, ${name}!' into a sequence of string literal, expression, and string literal tokens), multi-line strings (tracking line numbers for accurate error reporting), and the pipeline operator |> (which must be distinguished from the bitwise-or operator | followed by >). The lexer is implemented as a handwritten DFA for maximum performance, achieving about 50MB/s throughput on typical source files.

Pratt parsing, also called top-down operator precedence parsing, is the technique used in Lateralus's expression parser. Each token type is assigned a binding power -- a number representing its precedence. The parser is a loop: parse a primary expression (literal, identifier, parenthesized expression), then repeatedly consume infix operators whose binding power exceeds the current right-binding context. This naturally handles operator precedence and associativity without explicit grammar rules for each precedence level. The Pratt parser for Lateralus handles 25 infix operators, 8 prefix operators, and 5 postfix operators in about 300 lines of code.

Hindley-Milner type inference, formalized as Algorithm W, is the theoretical foundation for Lateralus's type checker. The algorithm processes expressions in a single pass, generating type constraints (equations between type variables) and then solving them using unification. Unification finds the most general substitution that satisfies all constraints simultaneously. The result is that every expression in a Lateralus program receives its most general type without any programmer-provided annotations. Annotations, when provided, are checked against the inferred type and produce an error if they are more restrictive or less restrictive than the inferred type.

Implementation Details

Single Static Assignment (SSA) form is the intermediate representation used in Lateralus's optimizer. In SSA, every variable is assigned exactly once, and uses that occur after a control flow merge point are mediated by phi functions that select the correct value based on which predecessor block was executed. SSA makes data flow explicit: from any use of a variable, you can trace a single chain of definitions to find where the value came from. This property enables efficient implementations of dominance-based optimizations: constant propagation (replace uses of a constant-valued variable with the constant), dead code elimination (remove assignments whose values are never used), and value numbering (identify equivalent expressions).

Register allocation is the process of mapping IR variables to physical machine registers. Modern CPUs have 16-32 general-purpose registers; programs may use thousands of variables. When there are more live variables at a point than available registers, some variables must be 'spilled' to the stack. The goal of register allocation is to minimize spills. Lateralus uses the linear scan register allocator for its speed (O(n) in the number of instructions) and the graph-coloring allocator for its quality (minimizing spills for hot paths). The two allocators are applied in sequence: the graph-coloring allocator handles the function body, and linear scan handles critical loops.

```
// Pratt parser: precedence climbing
const PREC: map = {
    "+": 10, "-": 10, "*": 20, "/": 20,
    "|>": 2, "==": 5, "!=": 5, "<": 5, ">": 5,
    ".": 30,  // field access

}

fn parse_expr(p: &Parser, min_prec: int) -> Expr {
    let mut left = parse_primary(p)
    while p.peek().prec() >= min_prec {
        let op = p.consume()
        let right = parse_expr(p, PREC[op.text] + 1)
        left = Expr.BinOp(op.kind, left, right)
    }
    return left
}
```

Pass Input Output Time% Incremental

Lexer source text tokens 5% no

Parser tokens CST 8% file

Name res CST resolved AST 4% module

Type check resolved AST typed AST 28% interface

IR gen typed AST SSA IR 5% module

Optimize SSA IR opt IR 15% module

Codegen opt IR machine code 30% module

Link objects binary 5% changed

22. Type Errors: Blame Assignment

Pratt parsing, also called top-down operator precedence parsing, is the technique used in Lateralus's expression parser. Each token type is assigned a binding power -- a number representing its precedence. The parser is a loop: parse a primary expression (literal, identifier, parenthesized expression), then repeatedly consume infix operators whose binding power exceeds the current right-binding context. This naturally handles operator precedence and associativity without explicit grammar rules for each precedence level. The Pratt parser for Lateralus handles 25 infix operators, 8 prefix operators, and 5 postfix operators in about 300 lines of code.

Hindley-Milner type inference, formalized as Algorithm W, is the theoretical foundation for Lateralus's type checker. The algorithm processes expressions in a single pass, generating type constraints (equations between type variables) and then solving them using unification. Unification finds the most general substitution that satisfies all constraints simultaneously. The result is that every expression in a Lateralus program receives its most general type without any programmer-provided annotations. Annotations, when provided, are checked against the inferred type and produce an error if they are more restrictive or less restrictive than the inferred type.

Single Static Assignment (SSA) form is the intermediate representation used in Lateralus's optimizer. In SSA, every variable is assigned exactly once, and uses that occur after a control flow merge point are mediated by phi functions that select the correct value based on which predecessor block was executed. SSA makes data flow explicit: from any use of a variable, you can trace a single chain of definitions to find where the value came from. This property enables efficient implementations of dominance-based optimizations: constant propagation (replace uses of a constant-valued variable with the constant), dead code elimination (remove assignments whose values are never used), and value numbering (identify equivalent expressions).

Implementation Details

Register allocation is the process of mapping IR variables to physical machine registers. Modern CPUs have 16-32 general-purpose registers; programs may use thousands of variables. When there are more live variables at a point than available registers, some variables must be 'spilled' to the stack. The goal of register allocation is to minimize spills. Lateralus uses the linear scan register allocator for its speed (O(n) in the number of instructions) and the graph-coloring allocator for its quality (minimizing spills for hot paths). The two allocators are applied in sequence: the graph-coloring allocator handles the function body, and linear scan handles critical loops.

The Lateralus Bytecode (LBC) format is designed for the Lateralus virtual machine. Instructions are 32-bit words: the top 8 bits identify the opcode, and the remaining 24 bits carry operands (register numbers, constant pool indices, jump offsets). The fixed-width instruction format allows O(1) random access to any instruction, which is important for the JIT

compiler's code patching mechanism. The constant pool stores all non-integer literals (floating-point numbers, strings, type descriptors) referenced by instructions. A separate debug section stores source file names, line numbers, and local variable names; this section can be stripped for production builds.

```
// Hindley-Milner unification
fn unify(t1: Type, t2: Type, subst: &mut Subst) -> Result[(), TypeError] {
    let t1 = apply(subst, t1)
    let t2 = apply(subst, t2)
    match (t1, t2) {
        (Type.Var(a), Type.Var(b)) if a == b => Ok(()),
        (Type.Var(a), t) | (t, Type.Var(a)) => {
            if occurs(a, t) { return Err(TypeError.InfiniteType(a, t)) }
            subst.insert(a, t); Ok(())
```

```
        },
        (Type.App(f1, a1), Type.App(f2, a2)) => {
            unify(f1, f2, subst)?;
            unify(a1, a2, subst)
        },
        _ => Err(TypeError.Mismatch(t1, t2)),
    }
}
```

Pass Input Output Time% Incremental

Lexer source text tokens 5% no

Parser tokens CST 8% file

Name res CST resolved AST 4% module

Type check resolved AST typed AST 28% interface

IR gen typed AST SSA IR 5% module

Optimize SSA IR opt IR 15% module

Codegen opt IR machine code 30% module

Link objects binary 5% changed

23. Type Errors: Suggestion Generation

Hindley-Milner type inference, formalized as Algorithm W, is the theoretical foundation for Lateralus's type checker. The algorithm processes expressions in a single pass, generating type constraints (equations between type variables) and then solving them using unification. Unification finds the most general substitution that satisfies all constraints simultaneously. The result is that every expression in a Lateralus program receives its most general type without any programmer-provided annotations. Annotations, when provided, are checked against the inferred type and produce an error if they are more restrictive or less restrictive than the inferred type.

Single Static Assignment (SSA) form is the intermediate representation used in Lateralus's optimizer. In SSA, every variable is assigned exactly once, and uses that occur after a control flow merge point are mediated by phi functions that select the correct value based on which predecessor block was executed. SSA makes data flow explicit: from any use of a variable, you can trace a single chain of definitions to find where the value came from. This property enables efficient implementations of dominance-based optimizations: constant propagation (replace uses of a constant-valued variable with the constant), dead code elimination (remove assignments whose values are never used), and value numbering (identify equivalent expressions).

Register allocation is the process of mapping IR variables to physical machine registers. Modern CPUs have 16-32 general-purpose registers; programs may use thousands of variables. When there are more live variables at a point than available registers, some variables must be 'spilled' to the stack. The goal of register allocation is to minimize spills. Lateralus uses the linear scan register allocator for its speed (O(n) in the number of instructions) and the graph-coloring allocator for its quality (minimizing spills for hot paths). The two allocators are applied in sequence: the graph-coloring allocator handles the function body, and linear scan handles critical loops.

Implementation Details

The Lateralus Bytecode (LBC) format is designed for the Lateralus virtual machine. Instructions are 32-bit words: the top

8 bits identify the opcode, and the remaining 24 bits carry operands (register numbers, constant pool indices, jump offsets). The fixed-width instruction format allows O(1) random access to any instruction, which is important for the JIT compiler's code patching mechanism. The constant pool stores all non-integer literals (floating-point numbers, strings, type descriptors) referenced by instructions. A separate debug section stores source file names, line numbers, and local variable names; this section can be stripped for production builds.

Bootstrapping a compiler means writing the compiler in the language it compiles. The process has three stages: Stage 1 is the initial implementation in a different language (Python for Lateralus); Stage 2 compiles the Lateralus compiler source with Stage 1 to produce a binary; Stage 3 re-compiles the Lateralus compiler source with Stage 2 and verifies that the Stage 2 and Stage 3 binaries are identical. This three-stage bootstrap, known as the Trusting Trust test, verifies that the compiler correctly compiles itself. Any discrepancy between Stage 2 and Stage 3 indicates a bug in the compiler's self-compilation path.

```
// SSA IR: a simple example
// Source: fn f(x: int) -> int { if x > 0 { x * 2 } else { -x } }
// SSA IR:
//   block entry(x_0: int):
//     t_0 = cmp_gt x_0, 0
//     branch t_0, then_block, else_block
//   block then_block():
//     t_1 = mul x_0, 2
//     jump merge_block(t_1)
//   block else_block():
//     t_2 = neg x_0
//     jump merge_block(t_2)
//   block merge_block(t_3: int):   // phi node
//     return t_3
```

Pass Input Output Time% Incremental

Lexer source text tokens 5% no

Parser tokens CST 8% file

Name res CST resolved AST 4% module

Type check resolved AST typed AST 28% interface

IR gen typed AST SSA IR 5% module

Optimize SSA IR opt IR 15% module

Codegen opt IR machine code 30% module

Link objects binary 5% changed

24. IR: SSA Form Construction

Single Static Assignment (SSA) form is the intermediate representation used in Lateralus's optimizer. In SSA, every variable is assigned exactly once, and uses that occur after a control flow merge point are mediated by phi functions that select the correct value based on which predecessor block was executed. SSA makes data flow explicit: from any use of a variable, you can trace a single chain of definitions to find where the value came from. This property enables efficient implementations of dominance-based optimizations: constant propagation (replace uses of a constant-valued variable with the constant), dead code elimination (remove assignments whose values are never used), and value numbering (identify equivalent expressions).

Register allocation is the process of mapping IR variables to physical machine registers. Modern CPUs have 16-32 general-purpose registers; programs may use thousands of variables. When there are more live variables at a point than available registers, some variables must be 'spilled' to the stack. The goal of register allocation is to minimize spills. Lateralus uses the linear scan register allocator for its speed (O(n) in the number of instructions) and the graph-coloring allocator for its quality (minimizing spills for hot paths). The two allocators are applied in sequence: the graph-coloring allocator handles the function body, and linear scan handles critical loops.

The Lateralus Bytecode (LBC) format is designed for the Lateralus virtual machine. Instructions are 32-bit words: the top 8 bits identify the opcode, and the remaining 24 bits carry operands (register numbers, constant pool indices, jump offsets). The fixed-width instruction format allows O(1) random access to any instruction, which is important for the JIT

compiler's code patching mechanism. The constant pool stores all non-integer literals (floating-point numbers, strings, type descriptors) referenced by instructions. A separate debug section stores source file names, line numbers, and local variable names; this section can be stripped for production builds.

Implementation Details

Bootstrapping a compiler means writing the compiler in the language it compiles. The process has three stages: Stage 1 is the initial implementation in a different language (Python for Lateralus); Stage 2 compiles the Lateralus compiler source with Stage 1 to produce a binary; Stage 3 re-compiles the Lateralus compiler source with Stage 2 and verifies that the Stage 2 and Stage 3 binaries are identical. This three-stage bootstrap, known as the Trusting Trust test, verifies that the compiler correctly compiles itself. Any discrepancy between Stage 2 and Stage 3 indicates a bug in the compiler's self-compilation path.

Incremental compilation avoids recompiling the entire program when a single file changes. Lateralus's module system makes incremental compilation tractable: each module has a well-defined interface (the set of exported types and functions), and changes to a module's implementation that do not change its interface do not require recompilation of dependent modules. The compiler maintains a dependency graph and a cache of compiled module interfaces. When a source file changes, the compiler recompiles that module, checks whether its interface changed, and if so, propagates recompilation to dependent modules. For large codebases, incremental compilation reduces rebuild time from minutes to seconds.

```
// LBC bytecode instruction encoding
// Bits: [31:24]=opcode [23:16]=dst [15:8]=src1 [7:0]=src2
const OP_ADD:  u8 = 0x01   // dst = src1 + src2
const OP_SUB:  u8 = 0x02   // dst = src1 - src2
const OP_MUL:  u8 = 0x03   // dst = src1 * src2
const OP_CALL: u8 = 0x10   // call func at const_pool[src1], argc=src2
const OP_RET:  u8 = 0x11   // return src1
const OP_JMP:  u8 = 0x20   // unconditional jump to offset
const OP_JZ:   u8 = 0x21   // jump if src1 == 0
```

```
fn encode(op: u8, dst: u8, s1: u8, s2: u8) -> u32 {
    return (op as u32 << 24) | (dst as u32 << 16) | (s1 as u32 << 8) | s2 as u32
}
```

Pass Input Output Time% Incremental

Lexer source text tokens 5% no

Parser tokens CST 8% file

Name res CST resolved AST 4% module

Type check resolved AST typed AST 28% interface

IR gen typed AST SSA IR 5% module

Optimize SSA IR opt IR 15% module

Codegen opt IR machine code 30% module

Link objects binary 5% changed

25. IR: Phi Node Placement

Register allocation is the process of mapping IR variables to physical machine registers. Modern CPUs have 16-32 general-purpose registers; programs may use thousands of variables. When there are more live variables at a point than available registers, some variables must be 'spilled' to the stack. The goal of register allocation is to minimize spills. Lateralus uses the linear scan register allocator for its speed (O(n) in the number of instructions) and the graph-coloring allocator for its quality (minimizing spills for hot paths). The two allocators are applied in sequence: the graph-coloring allocator handles the function body, and linear scan handles critical loops.

The Lateralus Bytecode (LBC) format is designed for the Lateralus virtual machine. Instructions are 32-bit words: the top 8 bits identify the opcode, and the remaining 24 bits carry operands (register numbers, constant pool indices, jump offsets). The fixed-width instruction format allows O(1) random access to any instruction, which is important for the JIT compiler's code patching mechanism. The constant pool stores all non-integer literals (floating-point numbers, strings,

type descriptors) referenced by instructions. A separate debug section stores source file names, line numbers, and local variable names; this section can be stripped for production builds.

Bootstrapping a compiler means writing the compiler in the language it compiles. The process has three stages: Stage 1 is the initial implementation in a different language (Python for Lateralus); Stage 2 compiles the Lateralus compiler source with Stage 1 to produce a binary; Stage 3 re-compiles the Lateralus compiler source with Stage 2 and verifies that the Stage 2 and Stage 3 binaries are identical. This three-stage bootstrap, known as the Trusting Trust test, verifies that the compiler correctly compiles itself. Any discrepancy between Stage 2 and Stage 3 indicates a bug in the compiler's self-compilation path.

Implementation Details

Incremental compilation avoids recompiling the entire program when a single file changes. Lateralus's module system makes incremental compilation tractable: each module has a well-defined interface (the set of exported types and functions), and changes to a module's implementation that do not change its interface do not require recompilation of dependent modules. The compiler maintains a dependency graph and a cache of compiled module interfaces. When a source file changes, the compiler recompiles that module, checks whether its interface changed, and if so, propagates recompilation to dependent

modules. For large codebases, incremental compilation reduces rebuild time from minutes to seconds.

Code generation translates the high-level SSA IR into machine code for a specific target architecture. Lateralus supports four backends: Python (transpilation to Python source, for portability), C (transpilation to C99, for maximum compatibility), RISC-V assembly (for the LateralusOS target and embedded systems), and LBC bytecode (for the Lateralus VM). Each backend performs instruction selection (choosing machine instructions for IR operations), register allocation (mapping IR values to machine registers or stack slots), and instruction scheduling (reordering instructions to avoid pipeline stalls). The C backend is the most commonly used for production deployments, as it leverages mature C compilers like GCC and Clang for the final machine code generation.

```
// Lexer token types
enum Token {
    Ident(str), IntLit(int), FloatLit(float), StrLit(str),
    Plus, Minus, Star, Slash, PipeFwd,  // |>
    BitorOp,                            // |
    Assign, Eq, Neq, Lt, Gt, Le, Ge,
    LParen, RParen, LBrace, RBrace,
    Fn, Let, If, Else, Return, Match, Struct, Enum,
    Eof,
}

fn next_token(src: &str, pos: &mut int) -> Token {
    skip_whitespace(src, pos)
    match src[*pos] {
        '|' => { *pos += 1
                 if src[*pos] == '>' { *pos += 1; return Token.PipeFwd }
                 return Token.BitorOp },
        // ...
    }
}
```

Pass Input Output Time% Incremental

Lexer source text tokens 5% no

Parser tokens CST 8% file

Name res CST resolved AST 4% module

Type check resolved AST typed AST 28% interface

IR gen typed AST SSA IR 5% module

Optimize SSA IR opt IR 15% module

Codegen opt IR machine code 30% module

Link objects binary 5% changed

26. IR: Memory SSA

The Lateralus Bytecode (LBC) format is designed for the Lateralus virtual machine. Instructions are 32-bit words: the top 8 bits identify the opcode, and the remaining 24 bits carry operands (register numbers, constant pool indices, jump offsets). The fixed-width instruction format allows O(1) random access to any instruction, which is important for the JIT compiler's code patching mechanism. The constant pool stores all non-integer literals (floating-point numbers, strings, type descriptors) referenced by instructions. A separate debug section stores source file names, line numbers, and local variable names; this section can be stripped for production builds.

Bootstrapping a compiler means writing the compiler in the language it compiles. The process has three stages: Stage 1 is the initial implementation in a different language (Python for Lateralus); Stage 2 compiles the Lateralus compiler source with Stage 1 to produce a binary; Stage 3 re-compiles the Lateralus compiler source with Stage 2 and verifies that the Stage 2 and Stage 3 binaries are identical. This three-stage bootstrap, known as the Trusting Trust test, verifies that the compiler correctly compiles itself. Any discrepancy between Stage 2 and Stage 3 indicates a bug in the compiler's self-compilation path.

Incremental compilation avoids recompiling the entire program when a single file changes. Lateralus's module system makes incremental compilation tractable: each module has a well-defined interface (the set of exported types and functions), and changes to a module's implementation that do not change its interface do not require recompilation of dependent modules. The compiler maintains a dependency graph and a cache of compiled module interfaces. When a source file changes, the compiler recompiles that module, checks whether its interface changed, and if so, propagates recompilation to dependent modules. For large codebases, incremental compilation reduces rebuild time from minutes to seconds.

Implementation Details

Code generation translates the high-level SSA IR into machine code for a specific target architecture. Lateralus supports four backends: Python (transpilation to Python source, for portability), C (transpilation to C99, for maximum compatibility), RISC-V assembly (for the LateralusOS target and embedded systems), and LBC bytecode (for the Lateralus VM). Each backend performs instruction selection (choosing machine instructions for IR operations), register allocation (mapping IR values to machine registers or stack slots), and instruction scheduling (reordering instructions to avoid pipeline stalls). The C backend is the most commonly used for production deployments, as it leverages mature C compilers like GCC and Clang for the final machine code generation.

Compiler construction sits at the intersection of formal language theory and practical systems engineering. The theoretical foundations -- regular languages for lexing, context-free grammars for parsing, type theory for semantic analysis -- give compiler writers the tools to reason about correctness. The engineering challenges -- performance, error recovery, incremental compilation, cross-platform code generation -- require the tools of systems programming. Lateralus's compiler is implemented in Lateralus itself (bootstrapped from an initial Python implementation), which means the compiler is both a producer and consumer of all the language features it compiles.

```
// Pratt parser: precedence climbing
const PREC: map = {
    "+": 10, "-": 10, "*": 20, "/": 20,
    "|>": 2, "==": 5, "!=": 5, "<": 5, ">": 5,
    ".": 30,  // field access
}

fn parse_expr(p: &Parser, min_prec: int) -> Expr {
    let mut left = parse_primary(p)
    while p.peek().prec() >= min_prec {
        let op = p.consume()
        let right = parse_expr(p, PREC[op.text] + 1)
        left = Expr.BinOp(op.kind, left, right)
    }
    return left
}
```

Pass Input Output Time% Incremental

Lexer source text tokens 5% no

Parser tokens CST 8% file

Name res CST resolved AST 4% module

Type check resolved AST typed AST 28% interface

IR gen typed AST SSA IR 5% module

Optimize SSA IR opt IR 15% module

Codegen opt IR machine code 30% module

Link objects binary 5% changed

27. Optimization: Constant Folding

Bootstrapping a compiler means writing the compiler in the language it compiles. The process has three stages: Stage 1 is the initial implementation in a different language (Python for Lateralus); Stage 2 compiles the Lateralus compiler source with Stage 1 to produce a binary; Stage 3 re-compiles the Lateralus compiler source with Stage 2 and verifies that the Stage 2 and Stage 3 binaries are identical. This three-stage bootstrap, known as the Trusting Trust test, verifies that the compiler correctly compiles itself. Any discrepancy between Stage 2 and Stage 3 indicates a bug in the compiler's self-compilation path.

Incremental compilation avoids recompiling the entire program when a single file changes. Lateralus's module system makes incremental compilation tractable: each module has a well-defined interface (the set of exported types and functions), and changes to a module's implementation that do not change its interface do not require recompilation of dependent modules. The compiler maintains a dependency graph and a cache of compiled module interfaces. When a source file changes, the compiler recompiles that module, checks whether its interface changed, and if so, propagates recompilation to dependent modules. For large codebases, incremental compilation reduces rebuild time from minutes to seconds.

Code generation translates the high-level SSA IR into machine code for a specific target architecture. Lateralus supports four backends: Python (transpilation to Python source, for portability), C (transpilation to C99, for maximum compatibility), RISC-V assembly (for the LateralusOS target and embedded systems), and LBC bytecode (for the Lateralus VM). Each backend performs instruction selection (choosing machine instructions for IR operations), register allocation (mapping IR values to machine registers or stack slots), and instruction scheduling (reordering instructions to avoid pipeline stalls). The C backend is the most commonly used for production deployments, as it leverages mature C compilers like GCC and Clang for the final machine code generation.

Implementation Details

Compiler construction sits at the intersection of formal language theory and practical systems engineering. The theoretical foundations -- regular languages for lexing, context-free grammars for parsing, type theory for semantic analysis -- give compiler writers the tools to reason about correctness. The engineering challenges -- performance, error recovery, incremental compilation, cross-platform code generation -- require the tools of systems programming. Lateralus's compiler is implemented in Lateralus itself (bootstrapped from an initial Python implementation), which means the compiler is both a producer and consumer of all the language features it compiles.

The lexer transforms a stream of characters into a stream of tokens. Each token has a type (IDENT, INT_LIT, STRING, PLUS, PIPEFWD, etc.) and a value (the matched text). Lateralus's lexer handles several non-trivial features: string interpolation (parsing 'Hello, ${name}!' into a sequence of string literal, expression, and string literal tokens), multi-line strings (tracking line numbers for accurate error reporting), and the pipeline operator |> (which must be distinguished from the bitwise-or operator | followed by >). The lexer is implemented as a handwritten DFA for maximum performance, achieving

about 50MB/s throughput on typical source files.

```
// Hindley-Milner unification
fn unify(t1: Type, t2: Type, subst: &mut Subst) -> Result[(), TypeError] {
    let t1 = apply(subst, t1)
    let t2 = apply(subst, t2)
    match (t1, t2) {
        (Type.Var(a), Type.Var(b)) if a == b => Ok(()),
        (Type.Var(a), t) | (t, Type.Var(a)) => {
            if occurs(a, t) { return Err(TypeError.InfiniteType(a, t)) }

            subst.insert(a, t); Ok(())
        },
        (Type.App(f1, a1), Type.App(f2, a2)) => {
            unify(f1, f2, subst)?;
            unify(a1, a2, subst)
        },
        _ => Err(TypeError.Mismatch(t1, t2)),
    }
}
```

Pass Input Output Time% Incremental

Lexer source text tokens 5% no

Parser tokens CST 8% file

Name res CST resolved AST 4% module

Type check resolved AST typed AST 28% interface

IR gen typed AST SSA IR 5% module

Optimize SSA IR opt IR 15% module

Codegen opt IR machine code 30% module

Link objects binary 5% changed

28. Optimization: Dead Code Elimination

Incremental compilation avoids recompiling the entire program when a single file changes. Lateralus's module system makes incremental compilation tractable: each module has a well-defined interface (the set of exported types and functions), and changes to a module's implementation that do not change its interface do not require recompilation of dependent modules. The compiler maintains a dependency graph and a cache of compiled module interfaces. When a source file changes, the compiler recompiles that module, checks whether its interface changed, and if so, propagates recompilation to dependent modules. For large codebases, incremental compilation reduces rebuild time from minutes to seconds.

Code generation translates the high-level SSA IR into machine code for a specific target architecture. Lateralus supports four backends: Python (transpilation to Python source, for portability), C (transpilation to C99, for maximum compatibility), RISC-V assembly (for the LateralusOS target and embedded systems), and LBC bytecode (for the Lateralus VM). Each backend performs instruction selection (choosing machine instructions for IR operations), register allocation (mapping IR values to machine registers or stack slots), and instruction scheduling (reordering instructions to avoid pipeline stalls). The C backend is the most commonly used for production deployments, as it leverages mature C compilers like GCC and Clang for the final machine code generation.

Compiler construction sits at the intersection of formal language theory and practical systems engineering. The theoretical foundations -- regular languages for lexing, context-free grammars for parsing, type theory for semantic analysis -- give compiler writers the tools to reason about correctness. The engineering challenges -- performance, error recovery, incremental compilation, cross-platform code generation -- require the tools of systems programming. Lateralus's compiler is implemented in Lateralus itself (bootstrapped from an initial Python implementation), which means the compiler is both a producer and consumer of all the language features it compiles.

Implementation Details

The lexer transforms a stream of characters into a stream of tokens. Each token has a type (IDENT, INT_LIT, STRING, PLUS, PIPEFWD, etc.) and a value (the matched text). Lateralus's lexer handles several non-trivial features: string interpolation (parsing 'Hello, ${name}!' into a sequence of string literal, expression, and string literal tokens), multi-line strings (tracking line numbers for accurate error reporting), and the pipeline operator |> (which must be distinguished from the bitwise-or operator | followed by >). The lexer is implemented as a handwritten DFA for maximum performance, achieving about 50MB/s throughput on typical source files.

Pratt parsing, also called top-down operator precedence parsing, is the technique used in Lateralus's expression parser. Each token type is assigned a binding power -- a number representing its precedence. The parser is a loop: parse a primary expression (literal, identifier, parenthesized expression), then repeatedly consume infix operators whose binding

power exceeds the current right-binding context. This naturally handles operator precedence and associativity without explicit grammar rules for each precedence level. The Pratt parser for Lateralus handles 25 infix operators, 8 prefix operators, and 5 postfix operators in about 300 lines of code.

```
// SSA IR: a simple example
// Source: fn f(x: int) -> int { if x > 0 { x * 2 } else { -x } }
// SSA IR:
//   block entry(x_0: int):
//     t_0 = cmp_gt x_0, 0
//     branch t_0, then_block, else_block
//   block then_block():
//     t_1 = mul x_0, 2
//     jump merge_block(t_1)
//   block else_block():
//     t_2 = neg x_0
//     jump merge_block(t_2)
//   block merge_block(t_3: int):   // phi node
//     return t_3
```

Pass Input Output Time% Incremental

Lexer source text tokens 5% no

Parser tokens CST 8% file

Name res CST resolved AST 4% module

Type check resolved AST typed AST 28% interface

IR gen typed AST SSA IR 5% module

Optimize SSA IR opt IR 15% module

Codegen opt IR machine code 30% module

Link objects binary 5% changed

29. Optimization: Inlining

Code generation translates the high-level SSA IR into machine code for a specific target architecture. Lateralus supports four backends: Python (transpilation to Python source, for portability), C (transpilation to C99, for maximum compatibility), RISC-V assembly (for the LateralusOS target and embedded systems), and LBC bytecode (for the Lateralus VM). Each backend performs instruction selection (choosing machine instructions for IR operations), register allocation (mapping IR values to machine registers or stack slots), and instruction scheduling (reordering instructions to avoid pipeline stalls). The C backend is the most commonly used for production deployments, as it leverages mature C compilers like GCC and Clang for the final machine code generation.

Compiler construction sits at the intersection of formal language theory and practical systems engineering. The theoretical foundations -- regular languages for lexing, context-free grammars for parsing, type theory for semantic analysis -- give compiler writers the tools to reason about correctness. The engineering challenges -- performance, error recovery, incremental compilation, cross-platform code generation -- require the tools of systems programming. Lateralus's compiler is implemented in Lateralus itself (bootstrapped from an initial Python implementation), which means the compiler is both a producer and consumer of all the language features it compiles.

The lexer transforms a stream of characters into a stream of tokens. Each token has a type (IDENT, INT_LIT, STRING, PLUS, PIPEFWD, etc.) and a value (the matched text). Lateralus's lexer handles several non-trivial features: string interpolation (parsing 'Hello, ${name}!' into a sequence of string literal, expression, and string literal tokens), multi-line strings (tracking line numbers for accurate error reporting), and the pipeline operator |> (which must be distinguished from the bitwise-or operator | followed by >). The lexer is implemented as a handwritten DFA for maximum performance, achieving about 50MB/s throughput on typical source files.

Implementation Details

Pratt parsing, also called top-down operator precedence parsing, is the technique used in Lateralus's expression parser. Each token type is assigned a binding power -- a number representing its precedence. The parser is a loop: parse a primary expression (literal, identifier, parenthesized expression), then repeatedly consume infix operators whose binding

power exceeds the current right-binding context. This naturally handles operator precedence and associativity without explicit grammar rules for each precedence level. The Pratt parser for Lateralus handles 25 infix operators, 8 prefix operators, and 5 postfix operators in about 300 lines of code.

Hindley-Milner type inference, formalized as Algorithm W, is the theoretical foundation for Lateralus's type checker. The algorithm processes expressions in a single pass, generating type constraints (equations between type variables) and then solving them using unification. Unification finds the most general substitution that satisfies all constraints simultaneously. The result is that every expression in a Lateralus program receives its most general type without any programmer-provided annotations. Annotations, when provided, are checked against the inferred type and produce an error if they are more restrictive or less restrictive than the inferred type.

```
// LBC bytecode instruction encoding
// Bits: [31:24]=opcode [23:16]=dst [15:8]=src1 [7:0]=src2
const OP_ADD:  u8 = 0x01   // dst = src1 + src2
const OP_SUB:  u8 = 0x02   // dst = src1 - src2
const OP_MUL:  u8 = 0x03   // dst = src1 * src2
const OP_CALL: u8 = 0x10   // call func at const_pool[src1], argc=src2
const OP_RET:  u8 = 0x11   // return src1
const OP_JMP:  u8 = 0x20   // unconditional jump to offset
const OP_JZ:   u8 = 0x21   // jump if src1 == 0
```

```
fn encode(op: u8, dst: u8, s1: u8, s2: u8) -> u32 {
    return (op as u32 << 24) | (dst as u32 << 16) | (s1 as u32 << 8) | s2 as u32
}
```

Pass Input Output Time% Incremental

Lexer source text tokens 5% no

Parser tokens CST 8% file

Name res CST resolved AST 4% module

Type check resolved AST typed AST 28% interface

IR gen typed AST SSA IR 5% module

Optimize SSA IR opt IR 15% module

Codegen opt IR machine code 30% module

Link objects binary 5% changed

30. Optimization: Loop Unrolling

Compiler construction sits at the intersection of formal language theory and practical systems engineering. The theoretical foundations -- regular languages for lexing, context-free grammars for parsing, type theory for semantic analysis -- give compiler writers the tools to reason about correctness. The engineering challenges -- performance, error recovery, incremental compilation, cross-platform code generation -- require the tools of systems programming. Lateralus's compiler is implemented in Lateralus itself (bootstrapped from an initial Python implementation), which means the compiler is both a producer and consumer of all the language features it compiles.

The lexer transforms a stream of characters into a stream of tokens. Each token has a type (IDENT, INT_LIT, STRING, PLUS, PIPEFWD, etc.) and a value (the matched text). Lateralus's lexer handles several non-trivial features: string interpolation (parsing 'Hello, ${name}!' into a sequence of string literal, expression, and string literal tokens), multi-line strings (tracking line numbers for accurate error reporting), and the pipeline operator |> (which must be distinguished from the bitwise-or operator | followed by >). The lexer is implemented as a handwritten DFA for maximum performance, achieving about 50MB/s throughput on typical source files.

Pratt parsing, also called top-down operator precedence parsing, is the technique used in Lateralus's expression parser. Each token type is assigned a binding power -- a number representing its precedence. The parser is a loop: parse a primary expression (literal, identifier, parenthesized expression), then repeatedly consume infix operators whose binding power exceeds the current right-binding context. This naturally handles operator precedence and associativity without explicit grammar rules for each precedence level. The Pratt parser for Lateralus handles 25 infix operators, 8 prefix operators, and 5 postfix operators in about 300 lines of code.

Implementation Details

Hindley-Milner type inference, formalized as Algorithm W, is the theoretical foundation for Lateralus's type checker. The algorithm processes expressions in a single pass, generating type constraints (equations between type variables) and then solving them using unification. Unification finds the most general substitution that satisfies all constraints simultaneously. The result is that every expression in a Lateralus program receives its most general type without any programmer-provided annotations. Annotations, when provided, are checked against the inferred type and produce an error if they are more restrictive or less restrictive than the inferred type.

Single Static Assignment (SSA) form is the intermediate representation used in Lateralus's optimizer. In SSA, every variable is assigned exactly once, and uses that occur after a control flow merge point are mediated by phi functions that select the correct value based on which predecessor block was executed. SSA makes data flow explicit: from any use of a variable, you can trace a single chain of definitions to find where the value came from. This property enables efficient implementations of dominance-based optimizations: constant propagation (replace uses of a constant-valued variable with the constant), dead code elimination (remove assignments whose values are never used), and value numbering (identify equivalent expressions).

```
// Lexer token types
enum Token {
    Ident(str), IntLit(int), FloatLit(float), StrLit(str),
    Plus, Minus, Star, Slash, PipeFwd,  // |>
    BitorOp,                            // |
    Assign, Eq, Neq, Lt, Gt, Le, Ge,
    LParen, RParen, LBrace, RBrace,
    Fn, Let, If, Else, Return, Match, Struct, Enum,
    Eof,
}

fn next_token(src: &str, pos: &mut int) -> Token {
    skip_whitespace(src, pos)
    match src[*pos] {
        '|' => { *pos += 1
                 if src[*pos] == '>' { *pos += 1; return Token.PipeFwd }
                 return Token.BitorOp },
        // ...
    }
}
```

Pass Input Output Time% Incremental

Lexer source text tokens 5% no

Parser tokens CST 8% file

Name res CST resolved AST 4% module

Type check resolved AST typed AST 28% interface

IR gen typed AST SSA IR 5% module

Optimize SSA IR opt IR 15% module

Codegen opt IR machine code 30% module

Link objects binary 5% changed

31. Optimization: Pipeline Fusion

The lexer transforms a stream of characters into a stream of tokens. Each token has a type (IDENT, INT_LIT, STRING, PLUS, PIPEFWD, etc.) and a value (the matched text). Lateralus's lexer handles several non-trivial features: string interpolation (parsing 'Hello, ${name}!' into a sequence of string literal, expression, and string literal tokens), multi-line strings (tracking line numbers for accurate error reporting), and the pipeline operator |> (which must be distinguished from the bitwise-or operator | followed by >). The lexer is implemented as a handwritten DFA for maximum performance, achieving about 50MB/s throughput on typical source files.

Pratt parsing, also called top-down operator precedence parsing, is the technique used in Lateralus's expression parser. Each token type is assigned a binding power -- a number representing its precedence. The parser is a loop: parse a

primary expression (literal, identifier, parenthesized expression), then repeatedly consume infix operators whose binding power exceeds the current right-binding context. This naturally handles operator precedence and associativity without explicit grammar rules for each precedence level. The Pratt parser for Lateralus handles 25 infix operators, 8 prefix operators, and 5 postfix operators in about 300 lines of code.

Hindley-Milner type inference, formalized as Algorithm W, is the theoretical foundation for Lateralus's type checker. The algorithm processes expressions in a single pass, generating type constraints (equations between type variables) and then solving them using unification. Unification finds the most general substitution that satisfies all constraints simultaneously. The result is that every expression in a Lateralus program receives its most general type without any programmer-provided annotations. Annotations, when provided, are checked against the inferred type and produce an error if they are more restrictive or less restrictive than the inferred type.

Implementation Details

Single Static Assignment (SSA) form is the intermediate representation used in Lateralus's optimizer. In SSA, every variable is assigned exactly once, and uses that occur after a control flow merge point are mediated by phi functions that select the correct value based on which predecessor block was executed. SSA makes data flow explicit: from any use of a variable, you can trace a single chain of definitions to find where the value came from. This property enables efficient implementations of dominance-based optimizations: constant propagation (replace uses of a constant-valued variable with the constant), dead code elimination (remove assignments whose values are never used), and value numbering (identify equivalent expressions).

Register allocation is the process of mapping IR variables to physical machine registers. Modern CPUs have 16-32 general-purpose registers; programs may use thousands of variables. When there are more live variables at a point than available registers, some variables must be 'spilled' to the stack. The goal of register allocation is to minimize spills. Lateralus uses the linear scan register allocator for its speed (O(n) in the number of instructions) and the graph-coloring allocator for its quality (minimizing spills for hot paths). The two allocators are applied in sequence: the graph-coloring allocator handles the function body, and linear scan handles critical loops.

```
// Pratt parser: precedence climbing
const PREC: map = {
    "+": 10, "-": 10, "*": 20, "/": 20,
    "|>": 2, "==": 5, "!=": 5, "<": 5, ">": 5,
    ".": 30,  // field access
}

fn parse_expr(p: &Parser, min_prec: int) -> Expr {
    let mut left = parse_primary(p)
    while p.peek().prec() >= min_prec {
        let op = p.consume()
        let right = parse_expr(p, PREC[op.text] + 1)
        left = Expr.BinOp(op.kind, left, right)
    }
    return left
}
```

Pass Input Output Time% Incremental

Lexer source text tokens 5% no

Parser tokens CST 8% file

Name res CST resolved AST 4% module

Type check resolved AST typed AST 28% interface

IR gen typed AST SSA IR 5% module

Optimize SSA IR opt IR 15% module

Codegen opt IR machine code 30% module

Link objects binary 5% changed

32. Optimization: Escape Analysis

Pratt parsing, also called top-down operator precedence parsing, is the technique used in Lateralus's expression parser.

Each token type is assigned a binding power -- a number representing its precedence. The parser is a loop: parse a primary expression (literal, identifier, parenthesized expression), then repeatedly consume infix operators whose binding power exceeds the current right-binding context. This naturally handles operator precedence and associativity without explicit grammar rules for each precedence level. The Pratt parser for Lateralus handles 25 infix operators, 8 prefix operators, and 5 postfix operators in about 300 lines of code.

Hindley-Milner type inference, formalized as Algorithm W, is the theoretical foundation for Lateralus's type checker. The algorithm processes expressions in a single pass, generating type constraints (equations between type variables) and then solving them using unification. Unification finds the most general substitution that satisfies all constraints simultaneously. The result is that every expression in a Lateralus program receives its most general type without any programmer-provided annotations. Annotations, when provided, are checked against the inferred type and produce an error if they are more restrictive or less restrictive than the inferred type.

Single Static Assignment (SSA) form is the intermediate representation used in Lateralus's optimizer. In SSA, every variable is assigned exactly once, and uses that occur after a control flow merge point are mediated by phi functions that select the correct value based on which predecessor block was executed. SSA makes data flow explicit: from any use of a variable, you can trace a single chain of definitions to find where the value came from. This property enables efficient implementations of dominance-based optimizations: constant propagation (replace uses of a constant-valued variable with the constant), dead code elimination (remove assignments whose values are never used), and value numbering (identify equivalent expressions).

Implementation Details

Register allocation is the process of mapping IR variables to physical machine registers. Modern CPUs have 16-32 general-purpose registers; programs may use thousands of variables. When there are more live variables at a point than available registers, some variables must be 'spilled' to the stack. The goal of register allocation is to minimize spills. Lateralus uses the linear scan register allocator for its speed (O(n) in the number of instructions) and the graph-coloring allocator for its quality (minimizing spills for hot paths). The two allocators are applied in sequence: the graph-coloring allocator handles the function body, and linear scan handles critical loops.

The Lateralus Bytecode (LBC) format is designed for the Lateralus virtual machine. Instructions are 32-bit words: the top 8 bits identify the opcode, and the remaining 24 bits carry operands (register numbers, constant pool indices, jump offsets). The fixed-width instruction format allows O(1) random

access to any instruction, which is important for the JIT compiler's code patching mechanism. The constant pool stores all non-integer literals (floating-point numbers, strings, type descriptors) referenced by instructions. A separate debug section stores source file names, line numbers, and local variable names; this section can be stripped for production builds.

```
// Hindley-Milner unification
fn unify(t1: Type, t2: Type, subst: &mut Subst) -> Result[(), TypeError] {
    let t1 = apply(subst, t1)
    let t2 = apply(subst, t2)
    match (t1, t2) {
        (Type.Var(a), Type.Var(b)) if a == b => Ok(()),
        (Type.Var(a), t) | (t, Type.Var(a)) => {
            if occurs(a, t) { return Err(TypeError.InfiniteType(a, t)) }
            subst.insert(a, t); Ok(())
        },
        (Type.App(f1, a1), Type.App(f2, a2)) => {
            unify(f1, f2, subst)?;
            unify(a1, a2, subst)
        },
        _ => Err(TypeError.Mismatch(t1, t2)),
    }
}
```

Pass Input Output Time% Incremental

Lexer source text tokens 5% no

Parser tokens CST 8% file

Name res CST resolved AST 4% module

Type check resolved AST typed AST 28% interface

IR gen typed AST SSA IR 5% module

Optimize SSA IR opt IR 15% module

Codegen opt IR machine code 30% module

Link objects binary 5% changed

33. Optimization: Devirtualization

Hindley-Milner type inference, formalized as Algorithm W, is the theoretical foundation for Lateralus's type checker. The algorithm processes expressions in a single pass, generating type constraints (equations between type variables) and then solving them using unification. Unification finds the most general substitution that satisfies all constraints simultaneously. The result is that every expression in a Lateralus program receives its most general type without any programmer-provided annotations. Annotations, when provided, are checked against the inferred type and produce an error if they are more restrictive or less restrictive than the inferred type.

Single Static Assignment (SSA) form is the intermediate representation used in Lateralus's optimizer. In SSA, every variable is assigned exactly once, and uses that occur after a control flow merge point are mediated by phi functions that select the correct value based on which predecessor block was executed. SSA makes data flow explicit: from any use of a variable, you can trace a single chain of definitions to find where the value came from. This property enables efficient implementations of dominance-based optimizations: constant propagation (replace uses of a constant-valued variable with the constant), dead code elimination (remove assignments whose values are never used), and value

numbering (identify equivalent expressions).

Register allocation is the process of mapping IR variables to physical machine registers. Modern CPUs have 16-32 general-purpose registers; programs may use thousands of variables. When there are more live variables at a point than available registers, some variables must be 'spilled' to the stack. The goal of register allocation is to minimize spills. Lateralus uses the linear scan register allocator for its speed (O(n) in the number of instructions) and the graph-coloring allocator for its quality (minimizing spills for hot paths). The two allocators are applied in sequence: the graph-coloring allocator handles the function body, and linear scan handles critical loops.

Implementation Details

The Lateralus Bytecode (LBC) format is designed for the Lateralus virtual machine. Instructions are 32-bit words: the top 8 bits identify the opcode, and the remaining 24 bits carry operands (register numbers, constant pool indices, jump offsets). The fixed-width instruction format allows O(1) random access to any instruction, which is important for the JIT compiler's code patching mechanism. The constant pool stores all non-integer literals (floating-point numbers, strings, type descriptors) referenced by instructions. A separate debug section stores source file names, line numbers, and local variable names; this section can be stripped for production builds.

Bootstrapping a compiler means writing the compiler in the language it compiles. The process has three stages: Stage 1 is the initial implementation in a different language (Python for Lateralus); Stage 2 compiles the Lateralus compiler source with Stage 1 to produce a binary; Stage 3 re-compiles the Lateralus compiler source with Stage 2 and verifies that the Stage 2 and Stage 3 binaries are identical. This three-stage bootstrap, known as the Trusting Trust test, verifies that the compiler correctly compiles itself. Any discrepancy between Stage 2 and Stage 3 indicates a bug in the compiler's self-compilation path.

```
// SSA IR: a simple example
// Source: fn f(x: int) -> int { if x > 0 { x * 2 } else { -x } }
// SSA IR:
//   block entry(x_0: int):
//     t_0 = cmp_gt x_0, 0
//     branch t_0, then_block, else_block
//   block then_block():
//     t_1 = mul x_0, 2
//     jump merge_block(t_1)
//   block else_block():
//     t_2 = neg x_0
//     jump merge_block(t_2)
//   block merge_block(t_3: int):   // phi node

//     return t_3
```

Pass Input Output Time% Incremental

Lexer source text tokens 5% no

Parser tokens CST 8% file

Name res CST resolved AST 4% module

Type check resolved AST typed AST 28% interface

IR gen typed AST SSA IR 5% module

Optimize SSA IR opt IR 15% module

Codegen opt IR machine code 30% module

Link objects binary 5% changed

34. Code Generation: Instruction Selection

Single Static Assignment (SSA) form is the intermediate representation used in Lateralus's optimizer. In SSA, every variable is assigned exactly once, and uses that occur after a control flow merge point are mediated by phi functions that select the correct value based on which predecessor block was executed. SSA makes data flow explicit: from any use of a variable, you can trace a single chain of definitions to find where the value came from. This property enables efficient implementations of dominance-based optimizations: constant propagation (replace uses of a constant-valued variable with the constant), dead code elimination (remove assignments whose values are never used), and value numbering (identify equivalent expressions).

Register allocation is the process of mapping IR variables to physical machine registers. Modern CPUs have 16-32 general-purpose registers; programs may use thousands of variables. When there are more live variables at a point than available registers, some variables must be 'spilled' to the stack. The goal of register allocation is to minimize spills. Lateralus uses the linear scan register allocator for its speed (O(n) in the number of instructions) and the graph-coloring allocator for its quality (minimizing spills for hot paths). The two allocators are applied in sequence: the graph-coloring allocator handles the function body, and linear scan handles critical loops.

The Lateralus Bytecode (LBC) format is designed for the Lateralus virtual machine. Instructions are 32-bit words: the top 8 bits identify the opcode, and the remaining 24 bits carry operands (register numbers, constant pool indices, jump offsets). The fixed-width instruction format allows O(1) random access to any instruction, which is important for the JIT compiler's code patching mechanism. The constant pool stores all non-integer literals (floating-point numbers, strings, type descriptors) referenced by instructions. A separate debug section stores source file names, line numbers, and local variable names; this section can be stripped for production builds.

Implementation Details

Bootstrapping a compiler means writing the compiler in the language it compiles. The process has three stages: Stage 1 is the initial implementation in a different language (Python for Lateralus); Stage 2 compiles the Lateralus compiler source with Stage 1 to produce a binary; Stage 3 re-compiles the Lateralus compiler source with Stage 2 and verifies that the Stage 2 and Stage 3 binaries are identical. This three-stage bootstrap, known as the Trusting Trust test, verifies that the compiler correctly compiles itself. Any discrepancy between Stage 2 and Stage 3 indicates a bug in the compiler's self-compilation path.

Incremental compilation avoids recompiling the entire program when a single file changes. Lateralus's module system makes incremental compilation tractable: each module has a well-defined interface (the set of exported types and functions), and changes to a module's implementation that do not change its interface do not require recompilation of dependent modules. The compiler maintains a dependency graph and a cache of compiled module interfaces. When a source file changes, the compiler recompiles that module, checks whether its interface changed, and if so, propagates recompilation to dependent modules. For large codebases, incremental compilation reduces rebuild time from minutes to seconds.

```
// LBC bytecode instruction encoding
// Bits: [31:24]=opcode [23:16]=dst [15:8]=src1 [7:0]=src2
const OP_ADD:  u8 = 0x01   // dst = src1 + src2
const OP_SUB:  u8 = 0x02   // dst = src1 - src2
const OP_MUL:  u8 = 0x03   // dst = src1 * src2

const OP_CALL: u8 = 0x10   // call func at const_pool[src1], argc=src2
const OP_RET:  u8 = 0x11   // return src1
```

```
const OP_JMP:  u8 = 0x20   // unconditional jump to offset
const OP_JZ:   u8 = 0x21   // jump if src1 == 0

fn encode(op: u8, dst: u8, s1: u8, s2: u8) -> u32 {
    return (op as u32 << 24) | (dst as u32 << 16) | (s1 as u32 << 8) | s2 as u32
}
```

Pass Input Output Time% Incremental

Lexer source text tokens 5% no

Parser tokens CST 8% file

Name res CST resolved AST 4% module

Type check resolved AST typed AST 28% interface

IR gen typed AST SSA IR 5% module

Optimize SSA IR opt IR 15% module

Codegen opt IR machine code 30% module

Link objects binary 5% changed

35. Code Generation: Register Allocation (Linear Scan)

Register allocation is the process of mapping IR variables to physical machine registers. Modern CPUs have 16-32 general-purpose registers; programs may use thousands of variables. When there are more live variables at a point than available registers, some variables must be 'spilled' to the stack. The goal of register allocation is to minimize spills. Lateralus uses the linear scan register allocator for its speed (O(n) in the number of instructions) and the graph-coloring allocator for its quality (minimizing spills for hot paths). The two allocators are applied in sequence: the graph-coloring allocator handles the function body, and linear scan handles critical loops.

The Lateralus Bytecode (LBC) format is designed for the Lateralus virtual machine. Instructions are 32-bit words: the top 8 bits identify the opcode, and the remaining 24 bits carry operands (register numbers, constant pool indices, jump offsets). The fixed-width instruction format allows O(1) random access to any instruction, which is important for the JIT compiler's code patching mechanism. The constant pool stores all non-integer literals (floating-point numbers, strings, type descriptors) referenced by instructions. A separate debug section stores source file names, line numbers, and local variable names; this section can be stripped for production builds.

Bootstrapping a compiler means writing the compiler in the language it compiles. The process has three stages: Stage 1 is the initial implementation in a different language (Python for Lateralus); Stage 2 compiles the Lateralus compiler source with Stage 1 to produce a binary; Stage 3 re-compiles the Lateralus compiler source with Stage 2 and verifies that the Stage 2 and Stage 3 binaries are identical. This three-stage bootstrap, known as the Trusting Trust test, verifies that the compiler correctly compiles itself. Any discrepancy between Stage 2 and Stage 3 indicates a bug in the compiler's self-compilation path.

Implementation Details

Incremental compilation avoids recompiling the entire program when a single file changes. Lateralus's module system makes incremental compilation tractable: each module has a well-defined interface (the set of exported types and functions), and changes to a module's implementation that do not change its interface do not require recompilation of dependent modules. The compiler maintains a dependency

graph and a cache of compiled module interfaces. When a source file changes, the compiler recompiles that module, checks whether its interface changed, and if so, propagates recompilation to dependent modules. For large codebases, incremental compilation reduces rebuild time from minutes to seconds.

Code generation translates the high-level SSA IR into machine code for a specific target architecture. Lateralus supports four backends: Python (transpilation to Python source, for portability), C (transpilation to C99, for maximum compatibility), RISC-V assembly (for the LateralusOS target and embedded systems), and LBC bytecode (for the Lateralus VM). Each backend performs instruction selection (choosing machine instructions for IR operations), register allocation (mapping IR values to machine registers or stack slots), and instruction scheduling (reordering instructions to

avoid pipeline stalls). The C backend is the most commonly used for production deployments, as it leverages mature C compilers like GCC and Clang for the final machine code generation.

```
// Lexer token types
enum Token {
    Ident(str), IntLit(int), FloatLit(float), StrLit(str),
    Plus, Minus, Star, Slash, PipeFwd,  // |>
    BitorOp,                            // |
    Assign, Eq, Neq, Lt, Gt, Le, Ge,
    LParen, RParen, LBrace, RBrace,
    Fn, Let, If, Else, Return, Match, Struct, Enum,
    Eof,
}

fn next_token(src: &str, pos: &mut int) -> Token {
    skip_whitespace(src, pos)
    match src[*pos] {
        '|' => { *pos += 1
                 if src[*pos] == '>' { *pos += 1; return Token.PipeFwd }
                 return Token.BitorOp },
        // ...
    }
}
```

Pass Input Output Time% Incremental

Lexer source text tokens 5% no

Parser tokens CST 8% file

Name res CST resolved AST 4% module

Type check resolved AST typed AST 28% interface

IR gen typed AST SSA IR 5% module

Optimize SSA IR opt IR 15% module

Codegen opt IR machine code 30% module

Link objects binary 5% changed

36. Code Generation: Register Allocation (Graph Coloring)

The Lateralus Bytecode (LBC) format is designed for the Lateralus virtual machine. Instructions are 32-bit words: the top 8 bits identify the opcode, and the remaining 24 bits carry operands (register numbers, constant pool indices, jump offsets). The fixed-width instruction format allows O(1) random access to any instruction, which is important for the JIT compiler's code patching mechanism. The constant pool stores all non-integer literals (floating-point numbers, strings, type descriptors) referenced by instructions. A separate debug section stores source file names, line numbers, and local variable names; this section can be stripped for production builds.

Bootstrapping a compiler means writing the compiler in the language it compiles. The process has three stages: Stage 1 is the initial implementation in a different language (Python for Lateralus); Stage 2 compiles the Lateralus compiler source with Stage 1 to produce a binary; Stage 3 re-compiles the Lateralus compiler source with Stage 2 and verifies that the Stage 2 and Stage 3 binaries are identical. This three-stage bootstrap, known as the Trusting Trust test, verifies that the compiler correctly compiles itself. Any discrepancy between Stage 2 and Stage 3 indicates a bug in the compiler's self-compilation path.

Incremental compilation avoids recompiling the entire program when a single file changes. Lateralus's module system makes incremental compilation tractable: each module has a well-defined interface (the set of exported types and functions), and changes to a module's implementation that do not change its interface do not require recompilation of dependent modules. The compiler maintains a dependency graph and a cache of compiled module interfaces. When a source file changes, the compiler recompiles that module, checks whether its interface changed, and if so, propagates recompilation to dependent modules. For large codebases, incremental compilation reduces rebuild time from minutes to seconds.

Implementation Details

Code generation translates the high-level SSA IR into machine code for a specific target architecture. Lateralus supports four backends: Python (transpilation to Python source, for portability), C (transpilation to C99, for maximum compatibility), RISC-V assembly (for the LateralusOS target and embedded systems), and LBC bytecode (for the Lateralus VM). Each backend performs instruction selection (choosing machine instructions for IR operations), register allocation (mapping IR values to machine registers or stack slots), and instruction scheduling (reordering instructions to avoid pipeline stalls). The C backend is the most commonly used for production deployments, as it leverages mature C compilers like GCC and Clang for the final machine code generation.

Compiler construction sits at the intersection of formal language theory and practical systems engineering. The theoretical foundations -- regular languages for lexing, context-free grammars for parsing, type theory for semantic analysis -- give compiler writers the tools to reason about correctness. The engineering challenges -- performance, error recovery, incremental compilation, cross-platform code generation -- require the tools of systems programming. Lateralus's compiler is implemented in Lateralus itself (bootstrapped from an initial Python implementation), which means the compiler is both a producer and consumer of all the language features it compiles.

```
// Pratt parser: precedence climbing
const PREC: map = {
    "+": 10, "-": 10, "*": 20, "/": 20,
    "|>": 2, "==": 5, "!=": 5, "<": 5, ">": 5,
    ".": 30,  // field access
}

fn parse_expr(p: &Parser, min_prec: int) -> Expr {
    let mut left = parse_primary(p)
    while p.peek().prec() >= min_prec {
```

```
        let op = p.consume()
        let right = parse_expr(p, PREC[op.text] + 1)
        left = Expr.BinOp(op.kind, left, right)
    }
    return left
}
```

Pass Input Output Time% Incremental

Lexer source text tokens 5% no

Parser tokens CST 8% file

Name res CST resolved AST 4% module

Type check resolved AST typed AST 28% interface

IR gen typed AST SSA IR 5% module

Optimize SSA IR opt IR 15% module

Codegen opt IR machine code 30% module

Link objects binary 5% changed

37. Code Generation: Instruction Scheduling

Bootstrapping a compiler means writing the compiler in the language it compiles. The process has three stages: Stage 1 is the initial implementation in a different language (Python for Lateralus); Stage 2 compiles the Lateralus compiler source with Stage 1 to produce a binary; Stage 3 re-compiles the Lateralus compiler source with Stage 2 and verifies that the Stage 2 and Stage 3 binaries are identical. This three-stage bootstrap, known as the Trusting Trust test, verifies that the compiler correctly compiles itself. Any discrepancy between Stage 2 and Stage 3 indicates a bug in the compiler's self-compilation path.

Incremental compilation avoids recompiling the entire program when a single file changes. Lateralus's module system makes incremental compilation tractable: each module has a well-defined interface (the set of exported types and functions), and changes to a module's implementation that do not change its interface do not require recompilation of dependent modules. The compiler maintains a dependency graph and a cache of compiled module interfaces. When a source file changes, the compiler recompiles that module, checks whether its interface changed, and if so, propagates recompilation to dependent modules. For large codebases, incremental compilation reduces rebuild time from minutes to seconds.

Code generation translates the high-level SSA IR into machine code for a specific target architecture. Lateralus supports four backends: Python (transpilation to Python source, for portability), C (transpilation to C99, for maximum compatibility), RISC-V assembly (for the LateralusOS target and embedded systems), and LBC bytecode (for the Lateralus VM). Each backend performs instruction selection (choosing machine instructions for IR operations), register allocation (mapping IR values to machine registers or stack slots), and instruction scheduling (reordering instructions to avoid pipeline stalls). The C backend is the most commonly used for production deployments, as it leverages mature C compilers like GCC and Clang for the final machine code generation.

Implementation Details

Compiler construction sits at the intersection of formal language theory and practical systems engineering. The theoretical foundations -- regular languages for lexing, context-free grammars for parsing, type theory for semantic analysis -- give compiler writers the tools to reason about correctness.

The engineering challenges -- performance, error recovery, incremental compilation, cross-platform code generation -- require the tools of systems programming. Lateralus's compiler is implemented in Lateralus itself (bootstrapped from an initial Python implementation), which means the compiler is both a producer and consumer of all the language features it compiles.

The lexer transforms a stream of characters into a stream of tokens. Each token has a type (IDENT, INT_LIT, STRING, PLUS, PIPEFWD, etc.) and a value (the matched text). Lateralus's lexer handles several non-trivial features: string interpolation (parsing 'Hello, ${name}!' into a sequence of string literal, expression, and string literal tokens), multi-line strings (tracking line numbers for accurate error reporting), and the pipeline operator |> (which must be distinguished from the bitwise-or operator | followed by >). The lexer is implemented as a handwritten DFA for maximum performance, achieving about 50MB/s throughput on typical source files.

```
// Hindley-Milner unification
fn unify(t1: Type, t2: Type, subst: &mut Subst) -> Result[(), TypeError] {
    let t1 = apply(subst, t1)
    let t2 = apply(subst, t2)
    match (t1, t2) {
        (Type.Var(a), Type.Var(b)) if a == b => Ok(()),
        (Type.Var(a), t) | (t, Type.Var(a)) => {
            if occurs(a, t) { return Err(TypeError.InfiniteType(a, t)) }
            subst.insert(a, t); Ok(())
        },
        (Type.App(f1, a1), Type.App(f2, a2)) => {
            unify(f1, f2, subst)?;
            unify(a1, a2, subst)
        },
        _ => Err(TypeError.Mismatch(t1, t2)),
    }
}
```

Pass Input Output Time% Incremental

Lexer source text tokens 5% no

Parser tokens CST 8% file

Name res CST resolved AST 4% module

Type check resolved AST typed AST 28% interface

IR gen typed AST SSA IR 5% module

Optimize SSA IR opt IR 15% module

Codegen opt IR machine code 30% module

Link objects binary 5% changed

38. Backend: Python Transpilation

Incremental compilation avoids recompiling the entire program when a single file changes. Lateralus's module system makes incremental compilation tractable: each module has a well-defined interface (the set of exported types and functions), and changes to a module's implementation that do not change its interface do not require recompilation of dependent modules. The compiler maintains a dependency graph and a cache of compiled module interfaces. When a

source file changes, the compiler recompiles that module, checks whether its interface changed, and if so, propagates recompilation to dependent modules. For large codebases, incremental compilation reduces rebuild time from minutes to seconds.

Code generation translates the high-level SSA IR into machine code for a specific target architecture. Lateralus supports four backends: Python (transpilation to Python source, for portability), C (transpilation to C99, for maximum compatibility), RISC-V assembly (for the LateralusOS target and embedded systems), and LBC bytecode (for the Lateralus VM). Each backend performs instruction selection (choosing machine instructions for IR operations), register allocation (mapping IR values to machine registers or stack slots), and instruction scheduling (reordering instructions to avoid pipeline stalls). The C backend is the most commonly used for production deployments, as it leverages mature C compilers like GCC and Clang for the final machine code generation.

Compiler construction sits at the intersection of formal language theory and practical systems engineering. The theoretical foundations -- regular languages for lexing, context-free grammars for parsing, type theory for semantic analysis -- give compiler writers the tools to reason about correctness. The engineering challenges -- performance, error recovery, incremental compilation, cross-platform code generation -- require the tools of systems programming. Lateralus's compiler is implemented in Lateralus itself (bootstrapped from an initial Python implementation), which means the compiler is both a producer and consumer of all the language features it compiles.

Implementation Details

The lexer transforms a stream of characters into a stream of tokens. Each token has a type (IDENT, INT_LIT, STRING, PLUS, PIPEFWD, etc.) and a value (the matched text). Lateralus's lexer handles several non-trivial features: string interpolation (parsing 'Hello, ${name}!' into a sequence of string literal, expression, and string literal tokens), multi-line strings (tracking line numbers for accurate error reporting), and the pipeline operator |> (which must be distinguished from the bitwise-or operator | followed by >). The lexer is implemented as a handwritten DFA for maximum performance, achieving about 50MB/s throughput on typical source files.

Pratt parsing, also called top-down operator precedence parsing, is the technique used in Lateralus's expression parser. Each token type is assigned a binding power -- a number representing its precedence. The parser is a loop: parse a primary expression (literal, identifier, parenthesized expression), then repeatedly consume infix operators whose binding power exceeds the current right-binding context. This naturally handles operator precedence and associativity without explicit grammar rules for each precedence level. The Pratt parser for Lateralus handles 25 infix operators, 8 prefix operators, and 5 postfix operators in about 300 lines of code.

```
// SSA IR: a simple example
// Source: fn f(x: int) -> int { if x > 0 { x * 2 } else { -x } }
// SSA IR:
//   block entry(x_0: int):
//     t_0 = cmp_gt x_0, 0
//     branch t_0, then_block, else_block
//   block then_block():
//     t_1 = mul x_0, 2
//     jump merge_block(t_1)
//   block else_block():
//     t_2 = neg x_0
//     jump merge_block(t_2)
//   block merge_block(t_3: int):   // phi node
//     return t_3
```

Pass Input Output Time% Incremental

Lexer source text tokens 5% no

Parser tokens CST 8% file

Name res CST resolved AST 4% module

Type check resolved AST typed AST 28% interface

IR gen typed AST SSA IR 5% module

Optimize SSA IR opt IR 15% module

Codegen opt IR machine code 30% module

Link objects binary 5% changed

39. Backend: C99 Transpilation

Code generation translates the high-level SSA IR into machine code for a specific target architecture. Lateralus supports four backends: Python (transpilation to Python source, for portability), C (transpilation to C99, for maximum compatibility), RISC-V assembly (for the LateralusOS target and embedded systems), and LBC bytecode (for the Lateralus VM). Each backend performs instruction selection (choosing machine instructions for IR operations), register allocation (mapping IR values to machine registers or stack slots), and instruction scheduling (reordering instructions to avoid pipeline stalls). The C backend is the most commonly used for production deployments, as it leverages mature C compilers like GCC and Clang for the final machine code generation.

Compiler construction sits at the intersection of formal language theory and practical systems engineering. The theoretical foundations -- regular languages for lexing, context-free grammars for parsing, type theory for semantic analysis -- give compiler writers the tools to reason about correctness. The engineering challenges -- performance, error recovery, incremental compilation, cross-platform code generation -- require the tools of systems programming. Lateralus's compiler is implemented in Lateralus itself (bootstrapped from an initial Python implementation), which means the compiler is both a producer and consumer of all the language features it compiles.

The lexer transforms a stream of characters into a stream of tokens. Each token has a type (IDENT, INT_LIT, STRING, PLUS, PIPEFWD, etc.) and a value (the matched text). Lateralus's lexer handles several non-trivial features: string interpolation (parsing 'Hello, ${name}!' into a sequence of string literal, expression, and string literal tokens), multi-line strings (tracking line numbers for accurate error reporting), and the pipeline operator |> (which must be distinguished from the bitwise-or operator | followed by >). The lexer is implemented as a handwritten DFA for maximum performance, achieving about 50MB/s throughput on typical source files.

Implementation Details

Pratt parsing, also called top-down operator precedence parsing, is the technique used in Lateralus's expression parser. Each token type is assigned a binding power -- a number representing its precedence. The parser is a loop: parse a primary expression (literal, identifier, parenthesized expression), then repeatedly consume infix operators whose binding power exceeds the current right-binding context. This naturally handles operator precedence and associativity without explicit grammar rules for each precedence level. The Pratt parser for Lateralus handles 25 infix operators, 8 prefix operators, and 5 postfix operators in about 300 lines of code.

Hindley-Milner type inference, formalized as Algorithm W, is the theoretical foundation for Lateralus's type checker. The algorithm processes expressions in a single pass, generating type constraints (equations between type variables) and then solving them using unification. Unification finds the most

general substitution that satisfies all constraints simultaneously. The result is that every expression in a Lateralus program receives its most general type without any programmer-provided annotations. Annotations, when provided, are checked against the inferred type and produce an error if they are more restrictive or less restrictive than the inferred type.

```
// LBC bytecode instruction encoding
// Bits: [31:24]=opcode [23:16]=dst [15:8]=src1 [7:0]=src2
const OP_ADD:  u8 = 0x01   // dst = src1 + src2
const OP_SUB:  u8 = 0x02   // dst = src1 - src2
const OP_MUL:  u8 = 0x03   // dst = src1 * src2
const OP_CALL: u8 = 0x10   // call func at const_pool[src1], argc=src2
const OP_RET:  u8 = 0x11   // return src1
const OP_JMP:  u8 = 0x20   // unconditional jump to offset
const OP_JZ:   u8 = 0x21   // jump if src1 == 0

fn encode(op: u8, dst: u8, s1: u8, s2: u8) -> u32 {
    return (op as u32 << 24) | (dst as u32 << 16) | (s1 as u32 << 8) | s2 as u32
}
```

Pass Input Output Time% Incremental

Lexer source text tokens 5% no

Parser tokens CST 8% file

Name res CST resolved AST 4% module

Type check resolved AST typed AST 28% interface

IR gen typed AST SSA IR 5% module

Optimize SSA IR opt IR 15% module

Codegen opt IR machine code 30% module

Link objects binary 5% changed

40. Backend: RISC-V Assembly

Compiler construction sits at the intersection of formal language theory and practical systems engineering. The theoretical foundations -- regular languages for lexing, context-free grammars for parsing, type theory for semantic analysis -- give compiler writers the tools to reason about correctness. The engineering challenges -- performance, error recovery, incremental compilation, cross-platform code generation -- require the tools of systems programming. Lateralus's compiler is implemented in Lateralus itself (bootstrapped from an initial Python implementation), which means the compiler is both a producer and consumer of all the language features it compiles.

The lexer transforms a stream of characters into a stream of tokens. Each token has a type (IDENT, INT_LIT, STRING, PLUS, PIPEFWD, etc.) and a value (the matched text). Lateralus's lexer handles several non-trivial features: string interpolation (parsing 'Hello, ${name}!' into a sequence of string literal, expression, and string literal tokens), multi-line strings (tracking line numbers for accurate error reporting), and the pipeline operator |> (which must be distinguished from the bitwise-or operator | followed by >). The lexer is implemented as a handwritten DFA for maximum performance, achieving about 50MB/s throughput on typical source files.

Pratt parsing, also called top-down operator precedence parsing, is the technique used in Lateralus's expression parser. Each token type is assigned a binding power -- a number representing its precedence. The parser is a loop: parse a primary expression (literal, identifier, parenthesized

expression), then repeatedly consume infix operators whose binding power exceeds the current right-binding context. This naturally handles operator precedence and associativity without explicit grammar rules for each precedence level. The Pratt parser for Lateralus handles 25 infix operators, 8 prefix operators, and 5 postfix operators in about 300 lines of code.

Implementation Details

Hindley-Milner type inference, formalized as Algorithm W, is the theoretical foundation for Lateralus's type checker. The algorithm processes expressions in a single pass, generating type constraints (equations between type variables) and then solving them using unification. Unification finds the most general substitution that satisfies all constraints simultaneously. The result is that every expression in a Lateralus program receives its most general type without any programmer-provided annotations. Annotations, when provided, are checked against the inferred type and produce an error if they are more restrictive or less restrictive than the inferred type.

Single Static Assignment (SSA) form is the intermediate representation used in Lateralus's optimizer. In SSA, every variable is assigned exactly once, and uses that occur after a control flow merge point are mediated by phi functions that select the correct value based on which predecessor block was executed. SSA makes data flow explicit: from any use of a variable, you can trace a single chain of definitions to find where the value came from. This property enables efficient implementations of dominance-based optimizations: constant propagation (replace uses of a constant-valued variable with the constant), dead code elimination (remove assignments whose values are never used), and value numbering (identify equivalent expressions).

```
// Lexer token types
enum Token {
    Ident(str), IntLit(int), FloatLit(float), StrLit(str),
    Plus, Minus, Star, Slash, PipeFwd,  // |>
    BitorOp,                            // |
    Assign, Eq, Neq, Lt, Gt, Le, Ge,
    LParen, RParen, LBrace, RBrace,
    Fn, Let, If, Else, Return, Match, Struct, Enum,
    Eof,
}

fn next_token(src: &str, pos: &mut int) -> Token {
    skip_whitespace(src, pos)
    match src[*pos] {
        '|' => { *pos += 1

                 if src[*pos] == '>' { *pos += 1; return Token.PipeFwd }
                 return Token.BitorOp },
        // ...
    }
}
```

Pass Input Output Time% Incremental

Lexer source text tokens 5% no

Parser tokens CST 8% file

Name res CST resolved AST 4% module

Type check resolved AST typed AST 28% interface

IR gen typed AST SSA IR 5% module

Optimize SSA IR opt IR 15% module

Codegen opt IR machine code 30% module

Link objects binary 5% changed

41. Backend: x86-64 Assembly

The lexer transforms a stream of characters into a stream of tokens. Each token has a type (IDENT, INT_LIT, STRING, PLUS, PIPEFWD, etc.) and a value (the matched text). Lateralus's lexer handles several non-trivial features: string interpolation (parsing 'Hello, ${name}!' into a sequence of string literal, expression, and string literal tokens), multi-line strings (tracking line numbers for accurate error reporting), and the pipeline operator |> (which must be distinguished from the bitwise-or operator | followed by >). The lexer is implemented as a handwritten DFA for maximum performance, achieving about 50MB/s throughput on typical source files.

Pratt parsing, also called top-down operator precedence parsing, is the technique used in Lateralus's expression parser. Each token type is assigned a binding power -- a number representing its precedence. The parser is a loop: parse a primary expression (literal, identifier, parenthesized expression), then repeatedly consume infix operators whose binding power exceeds the current right-binding context. This naturally handles operator precedence and associativity without explicit grammar rules for each precedence level. The Pratt parser for Lateralus handles 25 infix operators, 8 prefix operators, and 5 postfix operators in about 300 lines of code.

Hindley-Milner type inference, formalized as Algorithm W, is the theoretical foundation for Lateralus's type checker. The algorithm processes expressions in a single pass, generating type constraints (equations between type variables) and then solving them using unification. Unification finds the most general substitution that satisfies all constraints simultaneously. The result is that every expression in a Lateralus program receives its most general type without any programmer-provided annotations. Annotations, when provided, are checked against the inferred type and produce an error if they are more restrictive or less restrictive than the inferred type.

Implementation Details

Single Static Assignment (SSA) form is the intermediate representation used in Lateralus's optimizer. In SSA, every variable is assigned exactly once, and uses that occur after a control flow merge point are mediated by phi functions that select the correct value based on which predecessor block was executed. SSA makes data flow explicit: from any use of a variable, you can trace a single chain of definitions to find where the value came from. This property enables efficient implementations of dominance-based optimizations: constant propagation (replace uses of a constant-valued variable with the constant), dead code elimination (remove assignments whose values are never used), and value numbering (identify equivalent expressions).

Register allocation is the process of mapping IR variables to physical machine registers. Modern CPUs have 16-32 general-purpose registers; programs may use thousands of variables. When there are more live variables at a point than available registers, some variables must be 'spilled' to the stack. The goal of register allocation is to minimize spills. Lateralus uses the linear scan register allocator for its speed (O(n) in the number of instructions) and the graph-coloring allocator for its quality (minimizing spills for hot paths). The two allocators are applied in sequence: the graph-coloring allocator handles the function body, and linear scan handles critical loops.

```
// Pratt parser: precedence climbing
const PREC: map = {

    "+": 10, "-": 10, "*": 20, "/": 20,
    "|>": 2, "==": 5, "!=": 5, "<": 5, ">": 5,
    ".": 30,  // field access
```

```
}

fn parse_expr(p: &Parser, min_prec: int) -> Expr {
    let mut left = parse_primary(p)
    while p.peek().prec() >= min_prec {
        let op = p.consume()
        let right = parse_expr(p, PREC[op.text] + 1)
        left = Expr.BinOp(op.kind, left, right)
    }
    return left
}
```

Pass Input Output Time% Incremental

Lexer source text tokens 5% no

Parser tokens CST 8% file

Name res CST resolved AST 4% module

Type check resolved AST typed AST 28% interface

IR gen typed AST SSA IR 5% module

Optimize SSA IR opt IR 15% module

Codegen opt IR machine code 30% module

Link objects binary 5% changed

42. Backend: WebAssembly

Pratt parsing, also called top-down operator precedence parsing, is the technique used in Lateralus's expression parser. Each token type is assigned a binding power -- a number representing its precedence. The parser is a loop: parse a primary expression (literal, identifier, parenthesized expression), then repeatedly consume infix operators whose binding power exceeds the current right-binding context. This naturally handles operator precedence and associativity without explicit grammar rules for each precedence level. The Pratt parser for Lateralus handles 25 infix operators, 8 prefix operators, and 5 postfix operators in about 300 lines of code.

Hindley-Milner type inference, formalized as Algorithm W, is the theoretical foundation for Lateralus's type checker. The algorithm processes expressions in a single pass, generating type constraints (equations between type variables) and then solving them using unification. Unification finds the most general substitution that satisfies all constraints simultaneously. The result is that every expression in a Lateralus program receives its most general type without any programmer-provided annotations. Annotations, when provided, are checked against the inferred type and produce an error if they are more restrictive or less restrictive than the inferred type.

Single Static Assignment (SSA) form is the intermediate representation used in Lateralus's optimizer. In SSA, every variable is assigned exactly once, and uses that occur after a control flow merge point are mediated by phi functions that select the correct value based on which predecessor block was executed. SSA makes data flow explicit: from any use of a variable, you can trace a single chain of definitions to find where the value came from. This property enables efficient implementations of dominance-based optimizations: constant propagation (replace uses of a constant-valued variable with the constant), dead code elimination (remove assignments whose values are never used), and value numbering (identify equivalent expressions).

Implementation Details

Register allocation is the process of mapping IR variables to physical machine registers. Modern CPUs have 16-32 general-purpose registers; programs may use thousands of variables. When there are more live variables at a point than available registers, some variables must be 'spilled' to the stack. The goal of register allocation is to minimize spills. Lateralus uses the linear scan register allocator for its speed (O(n) in the number of instructions) and the graph-coloring allocator for its quality (minimizing spills for hot paths). The two allocators are applied in sequence: the graph-coloring allocator handles the function body, and linear scan handles critical loops.

The Lateralus Bytecode (LBC) format is designed for the Lateralus virtual machine. Instructions are 32-bit words: the top 8 bits identify the opcode, and the remaining 24 bits carry operands (register numbers, constant pool indices, jump offsets). The fixed-width instruction format allows O(1) random access to any instruction, which is important for the JIT compiler's code patching mechanism. The constant pool stores all non-integer literals (floating-point numbers, strings, type descriptors) referenced by instructions. A separate debug section stores source file names, line numbers, and local variable names; this section can be stripped for production builds.

```
// Hindley-Milner unification
fn unify(t1: Type, t2: Type, subst: &mut Subst) -> Result[(), TypeError] {
    let t1 = apply(subst, t1)
    let t2 = apply(subst, t2)
    match (t1, t2) {
        (Type.Var(a), Type.Var(b)) if a == b => Ok(()),
        (Type.Var(a), t) | (t, Type.Var(a)) => {
            if occurs(a, t) { return Err(TypeError.InfiniteType(a, t)) }
            subst.insert(a, t); Ok(())
        },
        (Type.App(f1, a1), Type.App(f2, a2)) => {
            unify(f1, f2, subst)?;
            unify(a1, a2, subst)
        },
        _ => Err(TypeError.Mismatch(t1, t2)),
    }
}
```

Pass Input Output Time% Incremental

Lexer source text tokens 5% no

Parser tokens CST 8% file

Name res CST resolved AST 4% module

Type check resolved AST typed AST 28% interface

IR gen typed AST SSA IR 5% module

Optimize SSA IR opt IR 15% module

Codegen opt IR machine code 30% module

Link objects binary 5% changed

43. Backend: LBC Bytecode

Hindley-Milner type inference, formalized as Algorithm W, is the theoretical foundation for Lateralus's type checker. The algorithm processes expressions in a single pass, generating type constraints (equations between type variables) and then solving them using unification. Unification finds the most general substitution that satisfies all constraints simultaneously. The result is that every expression in a Lateralus program receives its most general type without any programmer-provided annotations. Annotations, when provided, are checked against the inferred type and produce an error if they are more restrictive or less restrictive than the inferred type.

Single Static Assignment (SSA) form is the intermediate representation used in Lateralus's optimizer. In SSA, every variable is assigned exactly once, and uses that occur after a control flow merge point are mediated by phi functions that select the correct value based on which predecessor block was executed. SSA makes data flow explicit: from any use of a variable, you can trace a single chain of definitions to find where the value came from. This property enables efficient implementations of dominance-based optimizations: constant propagation (replace uses of a constant-valued variable with the constant), dead code elimination (remove assignments whose values are never used), and value numbering (identify equivalent expressions).

Register allocation is the process of mapping IR variables to physical machine registers. Modern CPUs have 16-32 general-purpose registers; programs may use thousands of variables. When there are more live variables at a point than available registers, some variables must be 'spilled' to the stack. The goal of register allocation is to minimize spills. Lateralus uses the linear scan register allocator for its speed (O(n) in the number of instructions) and the graph-coloring allocator for its quality (minimizing spills for hot paths). The two allocators are applied in sequence: the graph-coloring allocator handles the function body, and linear scan handles critical loops.

Implementation Details

The Lateralus Bytecode (LBC) format is designed for the Lateralus virtual machine. Instructions are 32-bit words: the top 8 bits identify the opcode, and the remaining 24 bits carry operands (register numbers, constant pool indices, jump offsets). The fixed-width instruction format allows O(1) random access to any instruction, which is important for the JIT compiler's code patching mechanism. The constant pool stores all non-integer literals (floating-point numbers, strings, type descriptors) referenced by instructions. A separate debug section stores source file names, line numbers, and local variable names; this section can be stripped for production builds.

Bootstrapping a compiler means writing the compiler in the language it compiles. The process has three stages: Stage 1 is the initial implementation in a different language (Python for Lateralus); Stage 2 compiles the Lateralus compiler source with Stage 1 to produce a binary; Stage 3 re-compiles the Lateralus compiler source with Stage 2 and verifies that the Stage 2 and Stage 3 binaries are identical. This three-stage bootstrap, known as the Trusting Trust test, verifies that the compiler correctly compiles itself. Any discrepancy between Stage 2 and Stage 3 indicates a bug in the compiler's self-compilation path.

```
// SSA IR: a simple example
// Source: fn f(x: int) -> int { if x > 0 { x * 2 } else { -x } }
// SSA IR:
//   block entry(x_0: int):
//     t_0 = cmp_gt x_0, 0
//     branch t_0, then_block, else_block
//   block then_block():
//     t_1 = mul x_0, 2
//     jump merge_block(t_1)
//   block else_block():
```

```
//     t_2 = neg x_0
//     jump merge_block(t_2)
//   block merge_block(t_3: int):   // phi node
//     return t_3
```

Pass Input Output Time% Incremental

Lexer source text tokens 5% no

Parser tokens CST 8% file

Name res CST resolved AST 4% module

Type check resolved AST typed AST 28% interface

IR gen typed AST SSA IR 5% module

Optimize SSA IR opt IR 15% module

Codegen opt IR machine code 30% module

Link objects binary 5% changed

44. LBC: Instruction Encoding

Single Static Assignment (SSA) form is the intermediate representation used in Lateralus's optimizer. In SSA, every variable is assigned exactly once, and uses that occur after a control flow merge point are mediated by phi functions that select the correct value based on which predecessor block was executed. SSA makes data flow explicit: from any use of a variable, you can trace a single chain of definitions to find where the value came from. This property enables efficient implementations of dominance-based optimizations: constant propagation (replace uses of a constant-valued variable with the constant), dead code elimination (remove assignments whose values are never used), and value numbering (identify equivalent expressions).

Register allocation is the process of mapping IR variables to physical machine registers. Modern CPUs have 16-32 general-purpose registers; programs may use thousands of variables. When there are more live variables at a point than available registers, some variables must be 'spilled' to the stack. The goal of register allocation is to minimize spills. Lateralus uses the linear scan register allocator for its speed (O(n) in the number of instructions) and the graph-coloring allocator for its quality (minimizing spills for hot paths). The two allocators are applied in sequence: the graph-coloring allocator handles the function body, and linear scan handles critical loops.

The Lateralus Bytecode (LBC) format is designed for the Lateralus virtual machine. Instructions are 32-bit words: the top

8 bits identify the opcode, and the remaining 24 bits carry operands (register numbers, constant pool indices, jump offsets). The fixed-width instruction format allows O(1) random access to any instruction, which is important for the JIT compiler's code patching mechanism. The constant pool stores all non-integer literals (floating-point numbers, strings, type descriptors) referenced by instructions. A separate debug section stores source file names, line numbers, and local variable names; this section can be stripped for production builds.

Implementation Details

Bootstrapping a compiler means writing the compiler in the language it compiles. The process has three stages: Stage 1 is the initial implementation in a different language (Python for Lateralus); Stage 2 compiles the Lateralus compiler source with Stage 1 to produce a binary; Stage 3 re-compiles the

Lateralus compiler source with Stage 2 and verifies that the Stage 2 and Stage 3 binaries are identical. This three-stage bootstrap, known as the Trusting Trust test, verifies that the compiler correctly compiles itself. Any discrepancy between Stage 2 and Stage 3 indicates a bug in the compiler's self-compilation path.

Incremental compilation avoids recompiling the entire program when a single file changes. Lateralus's module system makes incremental compilation tractable: each module has a well-defined interface (the set of exported types and functions), and changes to a module's implementation that do not change its interface do not require recompilation of dependent modules. The compiler maintains a dependency graph and a cache of compiled module interfaces. When a source file changes, the compiler recompiles that module, checks whether its interface changed, and if so, propagates recompilation to dependent modules. For large codebases, incremental compilation reduces rebuild time from minutes to seconds.

```
// LBC bytecode instruction encoding
// Bits: [31:24]=opcode [23:16]=dst [15:8]=src1 [7:0]=src2
const OP_ADD:  u8 = 0x01   // dst = src1 + src2
const OP_SUB:  u8 = 0x02   // dst = src1 - src2
const OP_MUL:  u8 = 0x03   // dst = src1 * src2
const OP_CALL: u8 = 0x10   // call func at const_pool[src1], argc=src2
const OP_RET:  u8 = 0x11   // return src1
const OP_JMP:  u8 = 0x20   // unconditional jump to offset
const OP_JZ:   u8 = 0x21   // jump if src1 == 0

fn encode(op: u8, dst: u8, s1: u8, s2: u8) -> u32 {
    return (op as u32 << 24) | (dst as u32 << 16) | (s1 as u32 << 8) | s2 as u32
}
```

Pass Input Output Time% Incremental

Lexer source text tokens 5% no

Parser tokens CST 8% file

Name res CST resolved AST 4% module

Type check resolved AST typed AST 28% interface

IR gen typed AST SSA IR 5% module

Optimize SSA IR opt IR 15% module

Codegen opt IR machine code 30% module

Link objects binary 5% changed

45. LBC: Constant Pool

Register allocation is the process of mapping IR variables to physical machine registers. Modern CPUs have 16-32 general-purpose registers; programs may use thousands of variables. When there are more live variables at a point than available registers, some variables must be 'spilled' to the stack. The goal of register allocation is to minimize spills. Lateralus uses the linear scan register allocator for its speed (O(n) in the number of instructions) and the graph-coloring allocator for its quality (minimizing spills for hot paths). The two allocators are applied in sequence: the graph-coloring allocator handles the function body, and linear scan handles critical loops.

The Lateralus Bytecode (LBC) format is designed for the Lateralus virtual machine. Instructions are 32-bit words: the top 8 bits identify the opcode, and the remaining 24 bits carry operands (register numbers, constant pool indices, jump

offsets). The fixed-width instruction format allows O(1) random access to any instruction, which is important for the JIT compiler's code patching mechanism. The constant pool stores all non-integer literals (floating-point numbers, strings, type descriptors) referenced by instructions. A separate debug section stores source file names, line numbers, and local variable names; this section can be stripped for production builds.

Bootstrapping a compiler means writing the compiler in the language it compiles. The process has three stages: Stage 1 is the initial implementation in a different language (Python for Lateralus); Stage 2 compiles the Lateralus compiler source with Stage 1 to produce a binary; Stage 3 re-compiles the Lateralus compiler source with Stage 2 and verifies that the Stage 2 and Stage 3 binaries are identical. This three-stage bootstrap, known as the Trusting Trust test, verifies that the compiler correctly compiles itself. Any discrepancy between Stage 2 and Stage 3 indicates a bug in the compiler's self-compilation path.

Implementation Details

Incremental compilation avoids recompiling the entire program when a single file changes. Lateralus's module system makes incremental compilation tractable: each module has a well-defined interface (the set of exported types and functions), and changes to a module's implementation that do not change its interface do not require recompilation of dependent modules. The compiler maintains a dependency graph and a cache of compiled module interfaces. When a source file changes, the compiler recompiles that module, checks whether its interface changed, and if so, propagates recompilation to dependent modules. For large codebases, incremental compilation reduces rebuild time from minutes to seconds.

Code generation translates the high-level SSA IR into machine code for a specific target architecture. Lateralus supports four backends: Python (transpilation to Python source, for portability), C (transpilation to C99, for maximum compatibility), RISC-V assembly (for the LateralusOS target and embedded systems), and LBC bytecode (for the Lateralus VM). Each backend performs instruction selection (choosing machine instructions for IR operations), register allocation (mapping IR values to machine registers or stack slots), and instruction scheduling (reordering instructions to avoid pipeline stalls). The C backend is the most commonly used for production deployments, as it leverages mature C compilers like GCC and Clang for the final machine code generation.

```
// Lexer token types
enum Token {
    Ident(str), IntLit(int), FloatLit(float), StrLit(str),
    Plus, Minus, Star, Slash, PipeFwd,  // |>
    BitorOp,                            // |
    Assign, Eq, Neq, Lt, Gt, Le, Ge,
    LParen, RParen, LBrace, RBrace,
    Fn, Let, If, Else, Return, Match, Struct, Enum,
    Eof,
}

fn next_token(src: &str, pos: &mut int) -> Token {
    skip_whitespace(src, pos)
    match src[*pos] {
        '|' => { *pos += 1
                 if src[*pos] == '>' { *pos += 1; return Token.PipeFwd }
                 return Token.BitorOp },
        // ...
    }
}
```

Pass Input Output Time% Incremental

Lexer source text tokens 5% no

Parser tokens CST 8% file

Name res CST resolved AST 4% module

Type check resolved AST typed AST 28% interface

IR gen typed AST SSA IR 5% module

Optimize SSA IR opt IR 15% module

Codegen opt IR machine code 30% module

Link objects binary 5% changed

46. LBC: Debug Information

The Lateralus Bytecode (LBC) format is designed for the Lateralus virtual machine. Instructions are 32-bit words: the top 8 bits identify the opcode, and the remaining 24 bits carry operands (register numbers, constant pool indices, jump offsets). The fixed-width instruction format allows O(1) random access to any instruction, which is important for the JIT compiler's code patching mechanism. The constant pool stores all non-integer literals (floating-point numbers, strings, type descriptors) referenced by instructions. A separate debug section stores source file names, line numbers, and local variable names; this section can be stripped for production builds.

Bootstrapping a compiler means writing the compiler in the language it compiles. The process has three stages: Stage 1 is the initial implementation in a different language (Python for Lateralus); Stage 2 compiles the Lateralus compiler source with Stage 1 to produce a binary; Stage 3 re-compiles the Lateralus compiler source with Stage 2 and verifies that the Stage 2 and Stage 3 binaries are identical. This three-stage bootstrap, known as the Trusting Trust test, verifies that the compiler correctly compiles itself. Any discrepancy between Stage 2 and Stage 3 indicates a bug in the compiler's self-compilation path.

Incremental compilation avoids recompiling the entire program when a single file changes. Lateralus's module system makes incremental compilation tractable: each module has a well-defined interface (the set of exported types and functions), and changes to a module's implementation that do not change its interface do not require recompilation of dependent modules. The compiler maintains a dependency graph and a cache of compiled module interfaces. When a source file changes, the compiler recompiles that module, checks whether its interface changed, and if so, propagates recompilation to dependent modules. For large codebases, incremental compilation reduces rebuild time from minutes to seconds.

Implementation Details

Code generation translates the high-level SSA IR into machine code for a specific target architecture. Lateralus supports four backends: Python (transpilation to Python source, for portability), C (transpilation to C99, for maximum compatibility), RISC-V assembly (for the LateralusOS target and embedded systems), and LBC bytecode (for the Lateralus VM). Each backend performs instruction selection (choosing machine instructions for IR operations), register allocation (mapping IR values to machine registers or stack slots), and instruction scheduling (reordering instructions to avoid pipeline stalls). The C backend is the most commonly used for production deployments, as it leverages mature C compilers like GCC and Clang for the final machine code generation.

Compiler construction sits at the intersection of formal language theory and practical systems engineering. The theoretical foundations -- regular languages for lexing, context-free grammars for parsing, type theory for semantic analysis -- give compiler writers the tools to reason about correctness. The engineering challenges -- performance, error recovery, incremental compilation, cross-platform

code generation -- require the tools of systems programming. Lateralus's compiler is implemented in Lateralus itself (bootstrapped from an initial Python implementation), which means the compiler is both a producer and consumer of all the language features it compiles.

```
// Pratt parser: precedence climbing
const PREC: map = {
    "+": 10, "-": 10, "*": 20, "/": 20,
    "|>": 2, "==": 5, "!=": 5, "<": 5, ">": 5,
    ".": 30,  // field access
}

fn parse_expr(p: &Parser, min_prec: int) -> Expr {
    let mut left = parse_primary(p)
    while p.peek().prec() >= min_prec {
        let op = p.consume()
        let right = parse_expr(p, PREC[op.text] + 1)
        left = Expr.BinOp(op.kind, left, right)
    }
    return left
}
```

Pass Input Output Time% Incremental

Lexer source text tokens 5% no

Parser tokens CST 8% file

Name res CST resolved AST 4% module

Type check resolved AST typed AST 28% interface

IR gen typed AST SSA IR 5% module

Optimize SSA IR opt IR 15% module

Codegen opt IR machine code 30% module

Link objects binary 5% changed

47. VM: Interpreter Dispatch Loop

Bootstrapping a compiler means writing the compiler in the language it compiles. The process has three stages: Stage 1 is the initial implementation in a different language (Python for Lateralus); Stage 2 compiles the Lateralus compiler source with Stage 1 to produce a binary; Stage 3 re-compiles the Lateralus compiler source with Stage 2 and verifies that the Stage 2 and Stage 3 binaries are identical. This three-stage bootstrap, known as the Trusting Trust test, verifies that the compiler correctly compiles itself. Any discrepancy between Stage 2 and Stage 3 indicates a bug in the compiler's self-compilation path.

Incremental compilation avoids recompiling the entire program when a single file changes. Lateralus's module system makes incremental compilation tractable: each module has a well-defined interface (the set of exported types and functions), and changes to a module's implementation that do not change its interface do not require recompilation of dependent modules. The compiler maintains a dependency graph and a cache of compiled module interfaces. When a source file changes, the compiler recompiles that module, checks whether its interface changed, and if so, propagates recompilation to dependent modules. For large codebases, incremental compilation reduces rebuild time from minutes to seconds.

Code generation translates the high-level SSA IR into machine code for a specific target architecture. Lateralus supports four backends: Python (transpilation to Python source, for portability), C (transpilation to C99, for maximum compatibility), RISC-V assembly (for the LateralusOS target and embedded systems), and LBC bytecode (for the Lateralus VM). Each backend performs instruction selection (choosing machine instructions for IR operations), register allocation (mapping IR values to machine registers or stack slots), and instruction scheduling (reordering instructions to avoid pipeline stalls). The C backend is the most commonly used for production deployments, as it leverages mature C compilers like GCC and Clang for the final machine code generation.

Implementation Details

Compiler construction sits at the intersection of formal language theory and practical systems engineering. The theoretical foundations -- regular languages for lexing, context-free grammars for parsing, type theory for semantic analysis -- give compiler writers the tools to reason about correctness. The engineering challenges -- performance, error recovery, incremental compilation, cross-platform code generation -- require the tools of systems programming. Lateralus's compiler is implemented in Lateralus itself (bootstrapped from an initial Python implementation), which means the compiler is both a producer and consumer of all the language features it compiles.

The lexer transforms a stream of characters into a stream of tokens. Each token has a type (IDENT, INT_LIT, STRING, PLUS, PIPEFWD, etc.) and a value (the matched text). Lateralus's lexer handles several non-trivial features: string interpolation (parsing 'Hello, ${name}!' into a sequence of string literal, expression, and string literal tokens), multi-line strings (tracking line numbers for accurate error reporting), and the pipeline operator |> (which must be distinguished from the bitwise-or operator | followed by >). The lexer is implemented as a handwritten DFA for maximum performance, achieving about 50MB/s throughput on typical source files.

```
// Hindley-Milner unification
fn unify(t1: Type, t2: Type, subst: &mut Subst) -> Result[(), TypeError] {
    let t1 = apply(subst, t1)
    let t2 = apply(subst, t2)
    match (t1, t2) {
        (Type.Var(a), Type.Var(b)) if a == b => Ok(()),
        (Type.Var(a), t) | (t, Type.Var(a)) => {

            if occurs(a, t) { return Err(TypeError.InfiniteType(a, t)) }
            subst.insert(a, t); Ok(())
        },
        (Type.App(f1, a1), Type.App(f2, a2)) => {
            unify(f1, f2, subst)?;
            unify(a1, a2, subst)
        },
        _ => Err(TypeError.Mismatch(t1, t2)),
    }
}
```

Pass Input Output Time% Incremental

Lexer source text tokens 5% no

Parser tokens CST 8% file

Name res CST resolved AST 4% module

Type check resolved AST typed AST 28% interface

IR gen typed AST SSA IR 5% module

Optimize SSA IR opt IR 15% module

Codegen opt IR machine code 30% module

Link objects binary 5% changed

48. VM: Stack Layout

Incremental compilation avoids recompiling the entire program when a single file changes. Lateralus's module system makes incremental compilation tractable: each module has a well-defined interface (the set of exported types and functions), and changes to a module's implementation that do not change its interface do not require recompilation of dependent modules. The compiler maintains a dependency graph and a cache of compiled module interfaces. When a source file changes, the compiler recompiles that module, checks whether its interface changed, and if so, propagates recompilation to dependent modules. For large codebases, incremental compilation reduces rebuild time from minutes to seconds.

Code generation translates the high-level SSA IR into machine code for a specific target architecture. Lateralus supports four backends: Python (transpilation to Python source, for portability), C (transpilation to C99, for maximum compatibility), RISC-V assembly (for the LateralusOS target and embedded systems), and LBC bytecode (for the Lateralus VM). Each backend performs instruction selection (choosing machine instructions for IR operations), register allocation (mapping IR values to machine registers or stack slots), and instruction scheduling (reordering instructions to avoid pipeline stalls). The C backend is the most commonly used for production deployments, as it leverages mature C compilers like GCC and Clang for the final machine code generation.

Compiler construction sits at the intersection of formal language theory and practical systems engineering. The theoretical foundations -- regular languages for lexing, context-free grammars for parsing, type theory for semantic analysis -- give compiler writers the tools to reason about correctness. The engineering challenges -- performance, error recovery, incremental compilation, cross-platform code generation -- require the tools of systems programming. Lateralus's compiler is implemented in Lateralus itself (bootstrapped from an initial Python implementation), which means the compiler is both a producer and consumer of all the language features it compiles.

Implementation Details

The lexer transforms a stream of characters into a stream of tokens. Each token has a type (IDENT, INT_LIT, STRING, PLUS, PIPEFWD, etc.) and a value (the matched text). Lateralus's lexer handles several non-trivial features: string interpolation (parsing 'Hello, ${name}!' into a sequence of string literal, expression, and string literal tokens), multi-line strings (tracking line numbers for accurate error reporting), and the pipeline operator |> (which must be distinguished from the bitwise-or operator | followed by >). The lexer is implemented as a handwritten DFA for maximum performance, achieving about 50MB/s throughput on typical source files.

Pratt parsing, also called top-down operator precedence parsing, is the technique used in Lateralus's expression parser. Each token type is assigned a binding power -- a number representing its precedence. The parser is a loop: parse a primary expression (literal, identifier, parenthesized expression), then repeatedly consume infix operators whose binding

power exceeds the current right-binding context. This naturally handles operator precedence and associativity without explicit grammar rules for each precedence level. The Pratt parser for Lateralus handles 25 infix operators, 8 prefix operators, and 5 postfix operators in about 300 lines of code.

```
// SSA IR: a simple example
// Source: fn f(x: int) -> int { if x > 0 { x * 2 } else { -x } }
// SSA IR:
```

```
//   block entry(x_0: int):
//     t_0 = cmp_gt x_0, 0
//     branch t_0, then_block, else_block
//   block then_block():
//     t_1 = mul x_0, 2
//     jump merge_block(t_1)
//   block else_block():
//     t_2 = neg x_0
//     jump merge_block(t_2)
//   block merge_block(t_3: int):   // phi node
//     return t_3
```

Pass Input Output Time% Incremental

Lexer source text tokens 5% no

Parser tokens CST 8% file

Name res CST resolved AST 4% module

Type check resolved AST typed AST 28% interface

IR gen typed AST SSA IR 5% module

Optimize SSA IR opt IR 15% module

Codegen opt IR machine code 30% module

Link objects binary 5% changed

49. VM: Garbage Collector

Code generation translates the high-level SSA IR into machine code for a specific target architecture. Lateralus supports four backends: Python (transpilation to Python source, for portability), C (transpilation to C99, for maximum compatibility), RISC-V assembly (for the LateralusOS target and embedded systems), and LBC bytecode (for the Lateralus VM). Each backend performs instruction selection (choosing machine instructions for IR operations), register allocation (mapping IR values to machine registers or stack slots), and instruction scheduling (reordering instructions to avoid pipeline stalls). The C backend is the most commonly used for production deployments, as it leverages mature C compilers like GCC and Clang for the final machine code generation.

Compiler construction sits at the intersection of formal language theory and practical systems engineering. The theoretical foundations -- regular languages for lexing, context-free grammars for parsing, type theory for semantic analysis -- give compiler writers the tools to reason about correctness. The engineering challenges -- performance, error recovery, incremental compilation, cross-platform code generation -- require the tools of systems programming. Lateralus's compiler is implemented in Lateralus itself (bootstrapped from an initial Python implementation), which means the compiler is both a producer and consumer of all the language features it compiles.

The lexer transforms a stream of characters into a stream of tokens. Each token has a type (IDENT, INT_LIT, STRING, PLUS, PIPEFWD, etc.) and a value (the matched text). Lateralus's lexer handles several non-trivial features: string interpolation (parsing 'Hello, ${name}!' into a sequence of string literal, expression, and string literal tokens), multi-line strings (tracking line numbers for accurate error reporting), and the pipeline operator |> (which must be distinguished from the bitwise-or operator | followed by >). The lexer is implemented as a handwritten DFA for maximum performance, achieving about 50MB/s throughput on typical source files.

Implementation Details

Pratt parsing, also called top-down operator precedence parsing, is the technique used in Lateralus's expression parser. Each token type is assigned a binding power -- a number representing its precedence. The parser is a loop: parse a primary expression (literal, identifier, parenthesized expression), then repeatedly consume infix operators whose binding

power exceeds the current right-binding context. This naturally handles operator precedence and associativity without explicit grammar rules for each precedence level. The Pratt parser for Lateralus handles 25 infix operators, 8 prefix operators, and 5 postfix operators in about 300 lines of code.

Hindley-Milner type inference, formalized as Algorithm W, is the theoretical foundation for Lateralus's type checker. The algorithm processes expressions in a single pass, generating type constraints (equations between type variables) and then solving them using unification. Unification finds the most general substitution that satisfies all constraints simultaneously. The result is that every expression in a Lateralus program receives its most general type without any programmer-provided annotations. Annotations, when provided, are checked against the inferred type and produce an error if they are more restrictive or less restrictive than the inferred type.

```
// LBC bytecode instruction encoding
// Bits: [31:24]=opcode [23:16]=dst [15:8]=src1 [7:0]=src2
const OP_ADD:  u8 = 0x01   // dst = src1 + src2
const OP_SUB:  u8 = 0x02   // dst = src1 - src2
const OP_MUL:  u8 = 0x03   // dst = src1 * src2
const OP_CALL: u8 = 0x10   // call func at const_pool[src1], argc=src2
const OP_RET:  u8 = 0x11   // return src1
const OP_JMP:  u8 = 0x20   // unconditional jump to offset
const OP_JZ:   u8 = 0x21   // jump if src1 == 0

fn encode(op: u8, dst: u8, s1: u8, s2: u8) -> u32 {
    return (op as u32 << 24) | (dst as u32 << 16) | (s1 as u32 << 8) | s2 as u32
}
```

Pass Input Output Time% Incremental

Lexer source text tokens 5% no

Parser tokens CST 8% file

Name res CST resolved AST 4% module

Type check resolved AST typed AST 28% interface

IR gen typed AST SSA IR 5% module

Optimize SSA IR opt IR 15% module

Codegen opt IR machine code 30% module

Link objects binary 5% changed

50. VM: JIT Compilation Basics

Compiler construction sits at the intersection of formal language theory and practical systems engineering. The theoretical foundations -- regular languages for lexing, context-free grammars for parsing, type theory for semantic analysis -- give compiler writers the tools to reason about correctness. The engineering challenges -- performance, error recovery, incremental compilation, cross-platform code generation -- require the tools of systems programming. Lateralus's compiler is implemented in Lateralus itself (bootstrapped from an initial Python implementation), which means the compiler is both a producer and consumer of all the language features it compiles.

The lexer transforms a stream of characters into a stream of tokens. Each token has a type (IDENT, INT_LIT, STRING, PLUS, PIPEFWD, etc.) and a value (the matched text). Lateralus's lexer handles several non-trivial features: string interpolation (parsing 'Hello, ${name}!' into a sequence of string literal, expression, and string literal tokens), multi-line strings (tracking line numbers for accurate error reporting), and the pipeline operator |> (which must be distinguished from the bitwise-or operator | followed by >). The lexer is implemented as a handwritten DFA for maximum performance, achieving about 50MB/s throughput on typical source files.

Pratt parsing, also called top-down operator precedence parsing, is the technique used in Lateralus's expression parser. Each token type is assigned a binding power -- a number representing its precedence. The parser is a loop: parse a primary expression (literal, identifier, parenthesized expression), then repeatedly consume infix operators whose binding power exceeds the current right-binding context. This naturally handles operator precedence and associativity without explicit grammar rules for each precedence level. The Pratt parser for Lateralus handles 25 infix operators, 8 prefix operators, and 5 postfix operators in about 300 lines of code.

Implementation Details

Hindley-Milner type inference, formalized as Algorithm W, is the theoretical foundation for Lateralus's type checker. The algorithm processes expressions in a single pass, generating type constraints (equations between type variables) and then solving them using unification. Unification finds the most general substitution that satisfies all constraints simultaneously. The result is that every expression in a Lateralus program receives its most general type without any programmer-provided annotations. Annotations, when provided, are checked against the inferred type and produce an error if they are more restrictive or less restrictive than the inferred type.

Single Static Assignment (SSA) form is the intermediate representation used in Lateralus's optimizer. In SSA, every variable is assigned exactly once, and uses that occur after a control flow merge point are mediated by phi functions that select the correct value based on which predecessor block was executed. SSA makes data flow explicit: from any use of a variable, you can trace a single chain of definitions to find where the value came from. This property enables efficient implementations of dominance-based optimizations: constant propagation (replace uses of a constant-valued variable with the constant), dead code elimination (remove assignments whose values are never used), and value numbering (identify equivalent expressions).

```
// Lexer token types
enum Token {
    Ident(str), IntLit(int), FloatLit(float), StrLit(str),
    Plus, Minus, Star, Slash, PipeFwd,  // |>
    BitorOp,                            // |
    Assign, Eq, Neq, Lt, Gt, Le, Ge,
    LParen, RParen, LBrace, RBrace,
    Fn, Let, If, Else, Return, Match, Struct, Enum,
    Eof,
}
```

```
fn next_token(src: &str, pos: &mut int) -> Token {
    skip_whitespace(src, pos)
    match src[*pos] {
        '|' => { *pos += 1
                 if src[*pos] == '>' { *pos += 1; return Token.PipeFwd }
                 return Token.BitorOp },
        // ...
    }
}
```

Pass Input Output Time% Incremental

Lexer source text tokens 5% no

Parser tokens CST 8% file

Name res CST resolved AST 4% module

Type check resolved AST typed AST 28% interface

IR gen typed AST SSA IR 5% module

Optimize SSA IR opt IR 15% module

Codegen opt IR machine code 30% module

Link objects binary 5% changed

51. Module System: Resolution Algorithm

The lexer transforms a stream of characters into a stream of tokens. Each token has a type (IDENT, INT_LIT, STRING, PLUS, PIPEFWD, etc.) and a value (the matched text). Lateralus's lexer handles several non-trivial features: string interpolation (parsing 'Hello, ${name}!' into a sequence of string literal, expression, and string literal tokens), multi-line strings (tracking line numbers for accurate error reporting), and the pipeline operator |> (which must be distinguished from the bitwise-or operator | followed by >). The lexer is implemented as a handwritten DFA for maximum performance, achieving about 50MB/s throughput on typical source files.

Pratt parsing, also called top-down operator precedence parsing, is the technique used in Lateralus's expression parser. Each token type is assigned a binding power -- a number representing its precedence. The parser is a loop: parse a

primary expression (literal, identifier, parenthesized expression), then repeatedly consume infix operators whose binding power exceeds the current right-binding context. This naturally handles operator precedence and associativity without explicit grammar rules for each precedence level. The Pratt parser for Lateralus handles 25 infix operators, 8 prefix operators, and 5 postfix operators in about 300 lines of code.

Hindley-Milner type inference, formalized as Algorithm W, is the theoretical foundation for Lateralus's type checker. The algorithm processes expressions in a single pass, generating type constraints (equations between type variables) and then solving them using unification. Unification finds the most general substitution that satisfies all constraints simultaneously. The result is that every expression in a Lateralus program receives its most general type without any programmer-provided annotations. Annotations, when provided, are checked against the inferred type and produce an error if they are more restrictive or less restrictive than the inferred type.

Implementation Details

Single Static Assignment (SSA) form is the intermediate representation used in Lateralus's optimizer. In SSA, every variable is assigned exactly once, and uses that occur after a control flow merge point are mediated by phi functions that select the correct value based on which predecessor block was executed. SSA makes data flow explicit: from any use of a variable, you can trace a single chain of definitions to find where the value came from. This property enables efficient implementations of dominance-based optimizations: constant propagation (replace uses of a constant-valued variable with the constant), dead code elimination (remove assignments whose values are never used), and value numbering (identify equivalent expressions).

Register allocation is the process of mapping IR variables to physical machine registers. Modern CPUs have 16-32 general-purpose registers; programs may use thousands of variables. When there are more live variables at a point than available registers, some variables must be 'spilled' to the stack. The goal of register allocation is to minimize spills. Lateralus uses the linear scan register allocator for its speed (O(n) in the number of instructions) and the graph-coloring allocator for its quality (minimizing spills for hot paths). The two allocators are applied in sequence: the graph-coloring allocator handles the function body, and linear scan handles critical loops.

```
// Pratt parser: precedence climbing
const PREC: map = {
    "+": 10, "-": 10, "*": 20, "/": 20,
    "|>": 2, "==": 5, "!=": 5, "<": 5, ">": 5,
    ".": 30,  // field access
}

fn parse_expr(p: &Parser, min_prec: int) -> Expr {
    let mut left = parse_primary(p)
    while p.peek().prec() >= min_prec {
        let op = p.consume()
        let right = parse_expr(p, PREC[op.text] + 1)
        left = Expr.BinOp(op.kind, left, right)
    }
    return left
}
```

Pass Input Output Time% Incremental

Lexer source text tokens 5% no

Parser tokens CST 8% file

Name res CST resolved AST 4% module

Type check resolved AST typed AST 28% interface

IR gen typed AST SSA IR 5% module

Optimize SSA IR opt IR 15% module

Codegen opt IR machine code 30% module

Link objects binary 5% changed

52. Module System: Interface Files (.lti)

Pratt parsing, also called top-down operator precedence parsing, is the technique used in Lateralus's expression parser.

Each token type is assigned a binding power -- a number representing its precedence. The parser is a loop: parse a primary expression (literal, identifier, parenthesized expression), then repeatedly consume infix operators whose binding power exceeds the current right-binding context. This naturally handles operator precedence and associativity without explicit grammar rules for each precedence level. The Pratt parser for Lateralus handles 25 infix operators, 8 prefix operators, and 5 postfix operators in about 300 lines of code.

Hindley-Milner type inference, formalized as Algorithm W, is the theoretical foundation for Lateralus's type checker. The algorithm processes expressions in a single pass, generating type constraints (equations between type variables) and then solving them using unification. Unification finds the most general substitution that satisfies all constraints simultaneously. The result is that every expression in a Lateralus program receives its most general type without any programmer-provided annotations. Annotations, when provided, are checked against the inferred type and produce an error if they are more restrictive or less restrictive than the inferred type.

Single Static Assignment (SSA) form is the intermediate representation used in Lateralus's optimizer. In SSA, every variable is assigned exactly once, and uses that occur after a control flow merge point are mediated by phi functions that select the correct value based on which predecessor block was executed. SSA makes data flow explicit: from any use of a variable, you can trace a single chain of definitions to find where the value came from. This property enables efficient implementations of dominance-based optimizations: constant propagation (replace uses of a constant-valued variable with the constant), dead code elimination (remove assignments whose values are never used), and value numbering (identify equivalent expressions).

Implementation Details

Register allocation is the process of mapping IR variables to physical machine registers. Modern CPUs have 16-32 general-purpose registers; programs may use thousands of variables. When there are more live variables at a point than available registers, some variables must be 'spilled' to the stack. The goal of register allocation is to minimize spills. Lateralus uses the linear scan register allocator for its speed (O(n) in the number of instructions) and the graph-coloring allocator for its quality (minimizing spills for hot paths). The two allocators are applied in sequence: the graph-coloring allocator handles the function body, and linear scan handles critical loops.

The Lateralus Bytecode (LBC) format is designed for the Lateralus virtual machine. Instructions are 32-bit words: the top 8 bits identify the opcode, and the remaining 24 bits carry operands (register numbers, constant pool indices, jump offsets). The fixed-width instruction format allows O(1) random access to any instruction, which is important for the JIT compiler's code patching mechanism. The constant pool stores all non-integer literals (floating-point numbers, strings, type descriptors) referenced by instructions. A separate debug section stores source file names, line numbers, and local variable names; this section can be stripped for production builds.

```
// Hindley-Milner unification
fn unify(t1: Type, t2: Type, subst: &mut Subst) -> Result[(), TypeError] {
    let t1 = apply(subst, t1)
    let t2 = apply(subst, t2)
    match (t1, t2) {
        (Type.Var(a), Type.Var(b)) if a == b => Ok(()),
        (Type.Var(a), t) | (t, Type.Var(a)) => {
            if occurs(a, t) { return Err(TypeError.InfiniteType(a, t)) }
            subst.insert(a, t); Ok(())
        },
        (Type.App(f1, a1), Type.App(f2, a2)) => {
            unify(f1, f2, subst)?;
            unify(a1, a2, subst)
```

```
        },
        _ => Err(TypeError.Mismatch(t1, t2)),
    }
}
```

Pass Input Output Time% Incremental

Lexer source text tokens 5% no

Parser tokens CST 8% file

Name res CST resolved AST 4% module

Type check resolved AST typed AST 28% interface

IR gen typed AST SSA IR 5% module

Optimize SSA IR opt IR 15% module

Codegen opt IR machine code 30% module

Link objects binary 5% changed

53. Module System: Circular Dependencies

Hindley-Milner type inference, formalized as Algorithm W, is the theoretical foundation for Lateralus's type checker. The algorithm processes expressions in a single pass, generating type constraints (equations between type variables) and then solving them using unification. Unification finds the most general substitution that satisfies all constraints simultaneously. The result is that every expression in a Lateralus program receives its most general type without any programmer-provided annotations. Annotations, when provided, are checked against the inferred type and produce an error if they are more restrictive or less restrictive than the inferred type.

Single Static Assignment (SSA) form is the intermediate representation used in Lateralus's optimizer. In SSA, every variable is assigned exactly once, and uses that occur after a control flow merge point are mediated by phi functions that select the correct value based on which predecessor block was executed. SSA makes data flow explicit: from any use of a variable, you can trace a single chain of definitions to find where the value came from. This property enables efficient implementations of dominance-based optimizations: constant propagation (replace uses of a constant-valued variable with the constant), dead code elimination (remove assignments whose values are never used), and value numbering (identify equivalent expressions).

Register allocation is the process of mapping IR variables to physical machine registers. Modern CPUs have 16-32 general-purpose registers; programs may use thousands of variables. When there are more live variables at a point than available registers, some variables must be 'spilled' to the stack. The goal of register allocation is to minimize spills. Lateralus uses the linear scan register allocator for its speed (O(n) in the number of instructions) and the graph-coloring allocator for its quality (minimizing spills for hot paths). The two allocators are applied in sequence: the graph-coloring allocator handles the function body, and linear scan handles critical loops.

Implementation Details

The Lateralus Bytecode (LBC) format is designed for the Lateralus virtual machine. Instructions are 32-bit words: the top 8 bits identify the opcode, and the remaining 24 bits carry operands (register numbers, constant pool indices, jump offsets). The fixed-width instruction format allows O(1) random access to any instruction, which is important for the JIT compiler's code patching mechanism. The constant pool stores all non-integer literals (floating-point numbers, strings, type descriptors) referenced

by instructions. A separate debug section stores source file names, line numbers, and local variable names; this section can be stripped for production builds.

Bootstrapping a compiler means writing the compiler in the language it compiles. The process has three stages: Stage 1 is the initial implementation in a different language (Python for Lateralus); Stage 2 compiles the Lateralus compiler source with Stage 1 to produce a binary; Stage 3 re-compiles the Lateralus compiler source with Stage 2 and verifies that the Stage 2 and Stage 3 binaries are identical. This three-stage bootstrap, known as the Trusting Trust test, verifies that the compiler correctly compiles itself. Any discrepancy between Stage 2 and Stage 3 indicates a bug in the compiler's self-compilation path.

```
// SSA IR: a simple example
// Source: fn f(x: int) -> int { if x > 0 { x * 2 } else { -x } }
// SSA IR:
//   block entry(x_0: int):
//     t_0 = cmp_gt x_0, 0
//     branch t_0, then_block, else_block
//   block then_block():
//     t_1 = mul x_0, 2
//     jump merge_block(t_1)
//   block else_block():
//     t_2 = neg x_0
//     jump merge_block(t_2)
//   block merge_block(t_3: int):   // phi node

//     return t_3
```

Pass Input Output Time% Incremental

Lexer source text tokens 5% no

Parser tokens CST 8% file

Name res CST resolved AST 4% module

Type check resolved AST typed AST 28% interface

IR gen typed AST SSA IR 5% module

Optimize SSA IR opt IR 15% module

Codegen opt IR machine code 30% module

Link objects binary 5% changed

54. Bootstrapping: Stage 1 (Python)

Single Static Assignment (SSA) form is the intermediate representation used in Lateralus's optimizer. In SSA, every variable is assigned exactly once, and uses that occur after a control flow merge point are mediated by phi functions that select the correct value based on which predecessor block was executed. SSA makes data flow explicit: from any use of a variable, you can trace a single chain of definitions to find where the value came from. This property enables efficient implementations of dominance-based optimizations: constant propagation (replace uses of a constant-valued variable with the constant), dead code elimination (remove assignments whose values are never used), and value numbering (identify equivalent expressions).

Register allocation is the process of mapping IR variables to physical machine registers. Modern CPUs have 16-32 general-purpose registers; programs may use thousands of variables. When there are more

live variables at a point than available registers, some variables must be 'spilled' to the stack. The goal of register allocation is to minimize spills. Lateralus uses the linear scan register allocator for its speed (O(n) in the number of instructions) and the graph-coloring allocator for its quality (minimizing spills for hot paths). The two allocators are applied in sequence: the graph-coloring allocator handles the function body, and linear scan handles critical loops.

The Lateralus Bytecode (LBC) format is designed for the Lateralus virtual machine. Instructions are 32-bit words: the top 8 bits identify the opcode, and the remaining 24 bits carry operands (register numbers, constant pool indices, jump offsets). The fixed-width instruction format allows O(1) random access to any instruction, which is important for the JIT compiler's code patching mechanism. The constant pool stores all non-integer literals (floating-point numbers, strings, type descriptors) referenced by instructions. A separate debug section stores source file names, line numbers, and local variable names; this section can be stripped for production builds.

Implementation Details

Bootstrapping a compiler means writing the compiler in the language it compiles. The process has three stages: Stage 1 is the initial implementation in a different language (Python for Lateralus); Stage 2 compiles the Lateralus compiler source with Stage 1 to produce a binary; Stage 3 re-compiles the Lateralus compiler source with Stage 2 and verifies that the Stage 2 and Stage 3 binaries are identical. This three-stage bootstrap, known as the Trusting Trust test, verifies that the compiler correctly compiles itself. Any discrepancy between Stage 2 and Stage 3 indicates a bug in the compiler's self-compilation path.

Incremental compilation avoids recompiling the entire program when a single file changes. Lateralus's module system makes incremental compilation tractable: each module has a well-defined interface (the set of exported types and functions), and changes to a module's implementation that do not change its interface do not require recompilation of dependent modules. The compiler maintains a dependency graph and a cache of compiled module interfaces. When a source file changes, the compiler recompiles that module, checks whether its interface changed, and if so, propagates recompilation to dependent modules. For large codebases, incremental compilation reduces rebuild time from minutes to seconds.

```
// LBC bytecode instruction encoding
// Bits: [31:24]=opcode [23:16]=dst [15:8]=src1 [7:0]=src2
const OP_ADD:  u8 = 0x01   // dst = src1 + src2
const OP_SUB:  u8 = 0x02   // dst = src1 - src2
const OP_MUL:  u8 = 0x03   // dst = src1 * src2

const OP_CALL: u8 = 0x10   // call func at const_pool[src1], argc=src2
const OP_RET:  u8 = 0x11   // return src1
const OP_JMP:  u8 = 0x20   // unconditional jump to offset
const OP_JZ:   u8 = 0x21   // jump if src1 == 0

fn encode(op: u8, dst: u8, s1: u8, s2: u8) -> u32 {
    return (op as u32 << 24) | (dst as u32 << 16) | (s1 as u32 << 8) | s2 as u32
}
```

Pass Input Output Time% Incremental

Lexer source text tokens 5% no

Parser tokens CST 8% file

Name res CST resolved AST 4% module

Type check resolved AST typed AST 28% interface

IR gen typed AST SSA IR 5% module

Optimize SSA IR opt IR 15% module

Codegen opt IR machine code 30% module

Link objects binary 5% changed

55. Bootstrapping: Stage 2 (self-hosted)

Register allocation is the process of mapping IR variables to physical machine registers. Modern CPUs have 16-32 general-purpose registers; programs may use thousands of variables. When there are more live variables at a point than available registers, some variables must be 'spilled' to the stack. The goal of register allocation is to minimize spills. Lateralus uses the linear scan register allocator for its speed (O(n) in the number of instructions) and the graph-coloring allocator for its quality (minimizing spills for hot paths). The two allocators are applied in sequence: the graph-coloring allocator handles the function body, and linear scan handles critical loops.

The Lateralus Bytecode (LBC) format is designed for the Lateralus virtual machine. Instructions are 32-bit words: the top 8 bits identify the opcode, and the remaining 24 bits carry operands (register numbers, constant pool indices, jump offsets). The fixed-width instruction format allows O(1) random access to any instruction, which is important for the JIT compiler's code patching mechanism. The constant pool stores all non-integer literals (floating-point numbers, strings, type descriptors) referenced by instructions. A separate debug section stores source file names, line numbers, and local variable names; this section can be stripped for production builds.

Bootstrapping a compiler means writing the compiler in the language it compiles. The process has three stages: Stage 1 is the initial implementation in a different language (Python for Lateralus); Stage 2 compiles the Lateralus compiler source with Stage 1 to produce a binary; Stage 3 re-compiles the Lateralus compiler source with Stage 2 and verifies that the Stage 2 and Stage 3 binaries are identical. This three-stage bootstrap, known as the Trusting Trust test, verifies that the compiler correctly compiles itself. Any discrepancy between Stage 2 and Stage 3 indicates a bug in the compiler's self-compilation path.

Implementation Details

Incremental compilation avoids recompiling the entire program when a single file changes. Lateralus's module system makes incremental compilation tractable: each module has a well-defined interface (the set of exported types and functions), and changes to a module's implementation that do not change its interface do not require recompilation of dependent modules. The compiler maintains a dependency graph and a cache of compiled module interfaces. When a source file changes, the compiler recompiles that module, checks whether its interface changed, and if so, propagates recompilation to dependent modules. For large codebases, incremental compilation reduces rebuild time from minutes to seconds.

Code generation translates the high-level SSA IR into machine code for a specific target architecture. Lateralus supports four backends: Python (transpilation to Python source, for portability), C (transpilation to C99, for maximum compatibility), RISC-V assembly (for the LateralusOS target and embedded systems), and LBC bytecode (for the Lateralus VM). Each backend performs instruction selection (choosing machine instructions for IR operations), register allocation (mapping IR values to machine registers or stack slots), and instruction scheduling (reordering instructions to

avoid pipeline stalls). The C backend is the most commonly used for production deployments, as it leverages mature C compilers like GCC and Clang for the final machine code generation.

```
// Lexer token types
enum Token {
    Ident(str), IntLit(int), FloatLit(float), StrLit(str),
```

```
    Plus, Minus, Star, Slash, PipeFwd,  // |>
    BitorOp,                            // |
    Assign, Eq, Neq, Lt, Gt, Le, Ge,
    LParen, RParen, LBrace, RBrace,
    Fn, Let, If, Else, Return, Match, Struct, Enum,
    Eof,
}

fn next_token(src: &str, pos: &mut int) -> Token {
    skip_whitespace(src, pos)
    match src[*pos] {
        '|' => { *pos += 1
                 if src[*pos] == '>' { *pos += 1; return Token.PipeFwd }
                 return Token.BitorOp },
        // ...
    }
}
```

Pass Input Output Time% Incremental

Lexer source text tokens 5% no

Parser tokens CST 8% file

Name res CST resolved AST 4% module

Type check resolved AST typed AST 28% interface

IR gen typed AST SSA IR 5% module

Optimize SSA IR opt IR 15% module

Codegen opt IR machine code 30% module

Link objects binary 5% changed

56. Bootstrapping: Trusting Trust Test

The Lateralus Bytecode (LBC) format is designed for the Lateralus virtual machine. Instructions are 32-bit words: the top 8 bits identify the opcode, and the remaining 24 bits carry operands (register numbers, constant pool indices, jump offsets). The fixed-width instruction format allows O(1) random access to any instruction, which is important for the JIT compiler's code patching mechanism. The constant pool stores all non-integer literals (floating-point numbers, strings, type descriptors) referenced by instructions. A separate debug section stores source file names, line numbers, and local variable names; this section can be stripped for production builds.

Bootstrapping a compiler means writing the compiler in the language it compiles. The process has three stages: Stage 1 is the initial implementation in a different language (Python for Lateralus); Stage 2 compiles the Lateralus compiler source with Stage 1 to produce a binary; Stage 3 re-compiles the Lateralus compiler source with Stage 2 and verifies that the Stage 2 and Stage 3 binaries are identical. This three-stage bootstrap, known as the Trusting Trust test, verifies that the compiler correctly compiles itself. Any discrepancy between Stage 2 and Stage 3 indicates a bug in the compiler's self-compilation path.

Incremental compilation avoids recompiling the entire program when a single file changes. Lateralus's module system makes incremental compilation tractable: each module has a well-defined interface (the set of exported types and functions), and changes to a module's implementation that do not change its interface do not require recompilation of dependent modules. The compiler maintains a dependency

graph and a cache of compiled module interfaces. When a source file changes, the compiler recompiles that module, checks whether its interface changed, and if so, propagates recompilation to dependent modules. For large codebases, incremental compilation reduces rebuild time from minutes to seconds.

Implementation Details

Code generation translates the high-level SSA IR into machine code for a specific target architecture. Lateralus supports four backends: Python (transpilation to Python source, for portability), C (transpilation to C99, for maximum compatibility), RISC-V assembly (for the LateralusOS target and embedded systems), and LBC bytecode (for the Lateralus VM). Each backend performs instruction selection (choosing machine instructions for IR operations), register allocation (mapping IR values to machine registers or stack slots), and instruction scheduling (reordering instructions to avoid pipeline stalls). The C backend is the most commonly used for production deployments, as it leverages mature C compilers like GCC and Clang for the final machine code generation.

Compiler construction sits at the intersection of formal language theory and practical systems engineering. The theoretical foundations -- regular languages for lexing, context-free grammars for parsing, type theory for semantic analysis -- give compiler writers the tools to reason about correctness. The engineering challenges -- performance, error recovery, incremental compilation, cross-platform code generation -- require the tools of systems programming. Lateralus's compiler is implemented in Lateralus itself (bootstrapped from an initial Python implementation), which means the compiler is both a producer and consumer of all the language features it compiles.

```
// Pratt parser: precedence climbing
const PREC: map = {
    "+": 10, "-": 10, "*": 20, "/": 20,
    "|>": 2, "==": 5, "!=": 5, "<": 5, ">": 5,
    ".": 30,  // field access
}

fn parse_expr(p: &Parser, min_prec: int) -> Expr {
    let mut left = parse_primary(p)
    while p.peek().prec() >= min_prec {
        let op = p.consume()
        let right = parse_expr(p, PREC[op.text] + 1)
        left = Expr.BinOp(op.kind, left, right)
    }
    return left
}
```

Pass Input Output Time% Incremental

Lexer source text tokens 5% no

Parser tokens CST 8% file

Name res CST resolved AST 4% module

Type check resolved AST typed AST 28% interface

IR gen typed AST SSA IR 5% module

Optimize SSA IR opt IR 15% module

Codegen opt IR machine code 30% module

Link objects binary 5% changed

57. Incremental Compilation

Bootstrapping a compiler means writing the compiler in the language it compiles. The process has three stages: Stage 1 is the initial implementation in a different language (Python for Lateralus); Stage 2 compiles the Lateralus compiler source with Stage 1 to produce a binary; Stage 3 re-compiles the Lateralus compiler source with Stage 2 and verifies that the Stage 2 and Stage 3 binaries are identical. This three-stage bootstrap, known as the Trusting Trust test, verifies that the compiler correctly compiles itself. Any discrepancy between Stage 2 and Stage 3 indicates a bug in the compiler's self-compilation path.

Incremental compilation avoids recompiling the entire program when a single file changes. Lateralus's module system makes incremental compilation tractable: each module has a well-defined interface (the set of exported types and functions), and changes to a module's implementation that do not change its interface do not require recompilation of dependent modules. The compiler maintains a dependency graph and a cache of compiled module interfaces. When a source file changes, the compiler recompiles that module, checks whether its interface changed, and if so, propagates recompilation to dependent modules. For large codebases, incremental compilation reduces rebuild time from minutes to seconds.

Code generation translates the high-level SSA IR into machine code for a specific target architecture. Lateralus supports four backends: Python (transpilation to Python source, for portability), C (transpilation to C99, for maximum compatibility), RISC-V assembly (for the LateralusOS target and embedded systems), and LBC bytecode (for the Lateralus VM). Each backend performs instruction selection (choosing machine instructions for IR operations), register allocation (mapping IR values to machine registers or stack slots), and instruction scheduling (reordering instructions to avoid pipeline stalls). The C backend is the most commonly used for production deployments, as it leverages mature C compilers like GCC and Clang for the final machine code generation.

Implementation Details

Compiler construction sits at the intersection of formal language theory and practical systems engineering. The theoretical foundations -- regular languages for lexing, context-free grammars for parsing, type theory for semantic analysis -- give compiler writers the tools to reason about correctness. The engineering challenges -- performance, error recovery, incremental compilation, cross-platform code generation -- require the tools of systems programming. Lateralus's compiler is implemented in Lateralus itself (bootstrapped from an initial Python implementation), which means the compiler is both a producer and consumer of all the language features it compiles.

The lexer transforms a stream of characters into a stream of tokens. Each token has a type (IDENT, INT_LIT, STRING, PLUS, PIPEFWD, etc.) and a value (the matched text). Lateralus's lexer handles several non-trivial features: string interpolation (parsing 'Hello, ${name}!' into a sequence of string literal, expression, and string literal tokens), multi-line strings (tracking line numbers for accurate error reporting), and the pipeline operator |> (which must be distinguished from the bitwise-or operator | followed by >). The lexer is implemented as a handwritten DFA for maximum performance, achieving about 50MB/s throughput on typical source files.

```
// Hindley-Milner unification
fn unify(t1: Type, t2: Type, subst: &mut Subst) -> Result[(), TypeError] {
    let t1 = apply(subst, t1)
    let t2 = apply(subst, t2)
    match (t1, t2) {
        (Type.Var(a), Type.Var(b)) if a == b => Ok(()),
        (Type.Var(a), t) | (t, Type.Var(a)) => {
            if occurs(a, t) { return Err(TypeError.InfiniteType(a, t)) }
            subst.insert(a, t); Ok(())
        },
```

```
        (Type.App(f1, a1), Type.App(f2, a2)) => {
            unify(f1, f2, subst)?;
            unify(a1, a2, subst)
        },
        _ => Err(TypeError.Mismatch(t1, t2)),
    }
}
```

Pass Input Output Time% Incremental

Lexer source text tokens 5% no

Parser tokens CST 8% file

Name res CST resolved AST 4% module

Type check resolved AST typed AST 28% interface

IR gen typed AST SSA IR 5% module

Optimize SSA IR opt IR 15% module

Codegen opt IR machine code 30% module

Link objects binary 5% changed

58. Parallel Compilation

Incremental compilation avoids recompiling the entire program when a single file changes. Lateralus's module system makes incremental compilation tractable: each module has a well-defined interface (the set of exported types and functions), and changes to a module's implementation that do not change its interface do not require recompilation of dependent modules. The compiler maintains a dependency graph and a cache of compiled module interfaces. When a

source file changes, the compiler recompiles that module, checks whether its interface changed, and if so, propagates recompilation to dependent modules. For large codebases, incremental compilation reduces rebuild time from minutes to seconds.

Code generation translates the high-level SSA IR into machine code for a specific target architecture. Lateralus supports four backends: Python (transpilation to Python source, for portability), C (transpilation to C99, for maximum compatibility), RISC-V assembly (for the LateralusOS target and embedded systems), and LBC bytecode (for the Lateralus VM). Each backend performs instruction selection (choosing machine instructions for IR operations), register allocation (mapping IR values to machine registers or stack slots), and instruction scheduling (reordering instructions to avoid pipeline stalls). The C backend is the most commonly used for production deployments, as it leverages mature C compilers like GCC and Clang for the final machine code generation.

Compiler construction sits at the intersection of formal language theory and practical systems engineering. The theoretical foundations -- regular languages for lexing, context-free grammars for parsing, type theory for semantic analysis -- give compiler writers the tools to reason about correctness. The engineering challenges -- performance, error recovery, incremental compilation, cross-platform code generation -- require the tools of systems programming. Lateralus's compiler is implemented in Lateralus itself (bootstrapped from an initial Python implementation), which means the compiler is both a producer and consumer of all the language features it compiles.

Implementation Details

The lexer transforms a stream of characters into a stream of tokens. Each token has a type (IDENT, INT_LIT, STRING, PLUS, PIPEFWD, etc.) and a value (the matched text). Lateralus's lexer handles several non-trivial features: string interpolation (parsing 'Hello, ${name}!' into a sequence of string literal, expression, and string literal tokens), multi-line strings (tracking line numbers for accurate error reporting), and the pipeline operator |> (which must be distinguished from the bitwise-or operator | followed by >). The lexer is implemented as a handwritten DFA for maximum performance, achieving about 50MB/s throughput on typical source files.

Pratt parsing, also called top-down operator precedence parsing, is the technique used in Lateralus's expression parser. Each token type is assigned a binding power -- a number representing its precedence. The parser is a loop: parse a primary expression (literal, identifier, parenthesized expression), then repeatedly consume infix operators whose binding power exceeds the current right-binding context. This naturally handles operator precedence and associativity without explicit grammar rules for each precedence level. The Pratt parser for Lateralus handles 25 infix operators, 8 prefix operators, and 5 postfix operators in about 300 lines of code.

```
// SSA IR: a simple example
// Source: fn f(x: int) -> int { if x > 0 { x * 2 } else { -x } }
// SSA IR:
//   block entry(x_0: int):
//     t_0 = cmp_gt x_0, 0
//     branch t_0, then_block, else_block
//   block then_block():
//     t_1 = mul x_0, 2
//     jump merge_block(t_1)
//   block else_block():
//     t_2 = neg x_0
//     jump merge_block(t_2)
//   block merge_block(t_3: int):   // phi node
//     return t_3
```

Pass Input Output Time% Incremental

Lexer source text tokens 5% no

Parser tokens CST 8% file

Name res CST resolved AST 4% module

Type check resolved AST typed AST 28% interface

IR gen typed AST SSA IR 5% module

Optimize SSA IR opt IR 15% module

Codegen opt IR machine code 30% module

Link objects binary 5% changed

59. Cross-Compilation

Code generation translates the high-level SSA IR into machine code for a specific target architecture. Lateralus supports four backends: Python (transpilation to Python source, for portability), C (transpilation to C99, for maximum compatibility), RISC-V assembly (for the LateralusOS target and embedded systems), and LBC bytecode (for the Lateralus VM). Each backend performs instruction selection (choosing machine instructions for IR operations), register allocation (mapping IR values to machine registers or stack slots), and instruction scheduling (reordering instructions to avoid pipeline stalls). The C backend is the most commonly used for production deployments, as it leverages mature C compilers like GCC and Clang for the final machine code generation.

Compiler construction sits at the intersection of formal language theory and practical systems engineering. The theoretical foundations -- regular languages for lexing, context-free grammars for parsing, type theory for semantic analysis -- give compiler writers the tools to reason about correctness. The engineering challenges -- performance, error recovery, incremental compilation, cross-platform code generation -- require the tools of systems programming. Lateralus's compiler is implemented in Lateralus itself (bootstrapped from an initial Python implementation), which means the compiler is both a producer and consumer of all the language features it compiles.

The lexer transforms a stream of characters into a stream of tokens. Each token has a type (IDENT, INT_LIT, STRING, PLUS, PIPEFWD, etc.) and a value (the matched text). Lateralus's lexer handles several non-trivial features: string interpolation (parsing 'Hello, ${name}!' into a sequence of string literal, expression, and string literal tokens), multi-line strings (tracking line numbers for accurate error reporting), and the pipeline operator |> (which must be distinguished from the bitwise-or operator | followed by >). The lexer is implemented as a handwritten DFA for maximum performance, achieving about 50MB/s throughput on typical source files.

Implementation Details

Pratt parsing, also called top-down operator precedence parsing, is the technique used in Lateralus's expression parser. Each token type is assigned a binding power -- a number representing its precedence. The parser is a loop: parse a primary expression (literal, identifier, parenthesized expression), then repeatedly consume infix operators whose binding power exceeds the current right-binding context. This naturally handles operator precedence and associativity without explicit grammar rules for each precedence level. The Pratt parser for Lateralus handles 25 infix operators, 8 prefix operators, and 5 postfix operators in about 300 lines of code.

Hindley-Milner type inference, formalized as Algorithm W, is the theoretical foundation for Lateralus's type checker. The algorithm processes expressions in a single pass, generating type constraints (equations between type variables) and then solving them using unification. Unification finds the most general substitution that satisfies all constraints simultaneously. The result is that every expression in a Lateralus program receives its most general type without any programmer-provided annotations. Annotations, when provided, are checked against the inferred type and produce an error if they are more restrictive or less restrictive than the inferred type.

```
// LBC bytecode instruction encoding
// Bits: [31:24]=opcode [23:16]=dst [15:8]=src1 [7:0]=src2
const OP_ADD:  u8 = 0x01   // dst = src1 + src2
const OP_SUB:  u8 = 0x02   // dst = src1 - src2
const OP_MUL:  u8 = 0x03   // dst = src1 * src2
const OP_CALL: u8 = 0x10   // call func at const_pool[src1], argc=src2
const OP_RET:  u8 = 0x11   // return src1
const OP_JMP:  u8 = 0x20   // unconditional jump to offset
const OP_JZ:   u8 = 0x21   // jump if src1 == 0
```

```
fn encode(op: u8, dst: u8, s1: u8, s2: u8) -> u32 {
    return (op as u32 << 24) | (dst as u32 << 16) | (s1 as u32 << 8) | s2 as u32
}
```

Pass Input Output Time% Incremental

Lexer source text tokens 5% no

Parser tokens CST 8% file

Name res CST resolved AST 4% module

Type check resolved AST typed AST 28% interface

IR gen typed AST SSA IR 5% module

Optimize SSA IR opt IR 15% module

Codegen opt IR machine code 30% module

Link objects binary 5% changed

60. Compiler Testing: Unit Tests

Compiler construction sits at the intersection of formal language theory and practical systems engineering. The theoretical foundations -- regular languages for lexing, context-free grammars for parsing, type theory for semantic analysis -- give compiler writers the tools to reason about correctness. The engineering challenges -- performance, error recovery, incremental compilation, cross-platform code generation -- require the tools of systems programming. Lateralus's compiler is implemented in Lateralus itself (bootstrapped from an initial Python implementation), which means the compiler is both a producer and consumer of all the language features it compiles.

The lexer transforms a stream of characters into a stream of tokens. Each token has a type (IDENT, INT_LIT, STRING, PLUS, PIPEFWD, etc.) and a value (the matched text). Lateralus's lexer handles several non-trivial features: string interpolation (parsing 'Hello, ${name}!' into a sequence of string literal, expression, and string literal tokens), multi-line strings (tracking line numbers for accurate error reporting), and the pipeline operator |> (which must be distinguished from the bitwise-or operator | followed by >). The lexer is implemented as a handwritten DFA for maximum performance, achieving about 50MB/s throughput on typical source files.

Pratt parsing, also called top-down operator precedence parsing, is the technique used in Lateralus's expression parser. Each token type is assigned a binding power -- a number representing its precedence. The parser is a loop: parse a primary expression (literal, identifier, parenthesized expression), then repeatedly consume infix operators whose binding power exceeds the current right-binding context. This naturally handles operator precedence and associativity without explicit grammar rules for each precedence level. The Pratt parser for Lateralus handles 25 infix operators, 8 prefix operators, and 5 postfix operators in about 300 lines of code.

Implementation Details

Hindley-Milner type inference, formalized as Algorithm W, is the theoretical foundation for Lateralus's type checker. The algorithm processes expressions in a single pass, generating type constraints (equations between type variables) and then solving them using unification. Unification finds the most general substitution that satisfies all constraints simultaneously. The result is that every expression in a Lateralus program receives its most general type without any programmer-provided annotations. Annotations, when provided, are checked against the inferred type and produce an error if they are more restrictive or less restrictive than the inferred type.

Single Static Assignment (SSA) form is the intermediate representation used in Lateralus's optimizer. In SSA, every variable is assigned exactly once, and uses that occur after a control flow merge point are mediated by phi functions that select the correct value based on which predecessor block was executed. SSA makes data flow explicit: from any use of a variable, you can trace a single chain of definitions to find where the value came from. This property enables efficient implementations of dominance-based optimizations: constant propagation (replace uses of a constant-valued variable with the constant), dead code elimination (remove assignments whose values are never used), and value numbering (identify equivalent expressions).

```
// Lexer token types
enum Token {
    Ident(str), IntLit(int), FloatLit(float), StrLit(str),
    Plus, Minus, Star, Slash, PipeFwd,  // |>
    BitorOp,                            // |
    Assign, Eq, Neq, Lt, Gt, Le, Ge,
    LParen, RParen, LBrace, RBrace,
    Fn, Let, If, Else, Return, Match, Struct, Enum,
    Eof,
}

fn next_token(src: &str, pos: &mut int) -> Token {
    skip_whitespace(src, pos)
    match src[*pos] {
        '|' => { *pos += 1

                if src[*pos] == '>' { *pos += 1; return Token.PipeFwd }
                return Token.BitorOp },
        // ...
    }
}
```

Pass Input Output Time% Incremental

Lexer source text tokens 5% no

Parser tokens CST 8% file

Name res CST resolved AST 4% module

Type check resolved AST typed AST 28% interface

IR gen typed AST SSA IR 5% module

Optimize SSA IR opt IR 15% module

Codegen opt IR machine code 30% module

Link objects binary 5% changed

61. Compiler Testing: Differential Testing

The lexer transforms a stream of characters into a stream of tokens. Each token has a type (IDENT, INT_LIT, STRING, PLUS, PIPEFWD, etc.) and a value (the matched text). Lateralus's lexer handles several non-trivial features: string interpolation (parsing 'Hello, ${name}!' into a sequence of string literal, expression, and string literal tokens), multi-line strings (tracking line numbers for accurate error reporting), and the pipeline operator |> (which must be distinguished from the bitwise-or operator | followed by >). The lexer is implemented as a handwritten DFA for maximum performance, achieving about 50MB/s throughput on typical source files.

Pratt parsing, also called top-down operator precedence parsing, is the technique used in Lateralus's expression parser. Each token type is assigned a binding power -- a number representing its precedence. The parser is a loop: parse a primary expression (literal, identifier, parenthesized expression), then repeatedly consume infix operators whose binding power exceeds the current right-binding context. This naturally handles operator precedence and associativity without explicit grammar rules for each precedence level. The Pratt parser for Lateralus handles 25 infix operators, 8 prefix operators, and 5 postfix operators in about 300 lines of code.

Hindley-Milner type inference, formalized as Algorithm W, is the theoretical foundation for Lateralus's type checker. The algorithm processes expressions in a single pass, generating type constraints (equations between type variables) and then solving them using unification. Unification finds the most general substitution that satisfies all constraints simultaneously. The result is that every expression in a Lateralus program receives its most general type without any programmer-provided annotations. Annotations, when provided, are checked against the inferred type and produce an error if they are more restrictive or less restrictive than the inferred type.

Implementation Details

Single Static Assignment (SSA) form is the intermediate representation used in Lateralus's optimizer. In SSA, every variable is assigned exactly once, and uses that occur after a control flow merge point are mediated by phi functions that select the correct value based on which predecessor block was executed. SSA makes data flow explicit: from any use of a variable, you can trace a single chain of definitions to find where the value came from. This property enables efficient implementations of dominance-based optimizations: constant propagation (replace uses of a constant-valued variable with the constant), dead code elimination (remove assignments whose values are never used), and value numbering (identify equivalent expressions).

Register allocation is the process of mapping IR variables to physical machine registers. Modern CPUs have 16-32 general-purpose registers; programs may use thousands of variables. When there are more live variables at a point than available registers, some variables must be 'spilled' to the stack. The goal of register allocation is to minimize spills. Lateralus uses the linear scan register allocator for its speed (O(n) in the number of instructions) and the graph-coloring allocator for its quality (minimizing spills for hot paths). The two allocators are applied in sequence: the graph-coloring allocator handles the function body, and linear scan handles critical loops.

```
// Pratt parser: precedence climbing
const PREC: map = {

    "+": 10, "-": 10, "*": 20, "/": 20,
    "|>": 2, "==": 5, "!=": 5, "<": 5, ">": 5,
    ".": 30,  // field access
}

fn parse_expr(p: &Parser, min_prec: int) -> Expr {
    let mut left = parse_primary(p)
    while p.peek().prec() >= min_prec {
        let op = p.consume()
        let right = parse_expr(p, PREC[op.text] + 1)
        left = Expr.BinOp(op.kind, left, right)
    }
    return left
}
```

Pass Input Output Time% Incremental

Lexer source text tokens 5% no

Parser tokens CST 8% file

Name res CST resolved AST 4% module

Type check resolved AST typed AST 28% interface

IR gen typed AST SSA IR 5% module

Optimize SSA IR opt IR 15% module

Codegen opt IR machine code 30% module

Link objects binary 5% changed

62. Compiler Testing: Fuzzing

Pratt parsing, also called top-down operator precedence parsing, is the technique used in Lateralus's expression parser. Each token type is assigned a binding power -- a number representing its precedence. The parser is a loop: parse a primary expression (literal, identifier, parenthesized expression), then repeatedly consume infix operators whose binding power exceeds the current right-binding context. This naturally handles operator precedence and associativity without explicit grammar rules for each precedence level. The Pratt parser for Lateralus handles 25 infix operators, 8 prefix operators, and 5 postfix operators in about 300 lines of code.

Hindley-Milner type inference, formalized as Algorithm W, is the theoretical foundation for Lateralus's type checker. The algorithm processes expressions in a single pass, generating type constraints (equations between type variables) and then solving them using unification. Unification finds the most general substitution that satisfies all constraints simultaneously. The result is that every expression in a Lateralus program receives its most general type without any programmer-provided annotations. Annotations, when provided, are checked against the inferred type and produce an error if they are more restrictive or less restrictive than the inferred type.

Single Static Assignment (SSA) form is the intermediate representation used in Lateralus's optimizer. In SSA, every variable is assigned exactly once, and uses that occur after a control flow merge point are mediated by phi functions that select the correct value based on which predecessor block was executed. SSA makes data flow explicit: from any use of a variable, you can trace a single chain of definitions to find where the value came from. This property enables efficient implementations of dominance-based optimizations: constant propagation (replace uses of a constant-valued variable with the constant), dead code elimination (remove assignments whose values are never used), and value numbering (identify equivalent expressions).

Implementation Details

Register allocation is the process of mapping IR variables to physical machine registers. Modern CPUs have 16-32 general-purpose registers; programs may use thousands of variables. When there are more live variables at a point than available registers, some variables must be 'spilled' to the stack. The goal of register allocation is to minimize spills. Lateralus uses the linear scan register allocator for its speed (O(n) in the number of instructions) and the graph-coloring allocator for its quality (minimizing spills for hot paths). The two allocators are applied in sequence: the graph-coloring allocator handles the function body, and linear scan handles critical loops.

The Lateralus Bytecode (LBC) format is designed for the Lateralus virtual machine. Instructions are 32-bit words: the top 8 bits identify the opcode, and the remaining 24 bits carry operands (register numbers, constant pool indices, jump offsets). The fixed-width instruction format allows O(1) random access to any instruction, which is important for the JIT compiler's code patching mechanism. The

constant pool stores all non-integer literals (floating-point numbers, strings, type descriptors) referenced by instructions. A separate debug section stores source file names, line numbers, and local variable names; this section can be stripped for production builds.

```
// Hindley-Milner unification
fn unify(t1: Type, t2: Type, subst: &mut Subst) -> Result[(), TypeError] {
    let t1 = apply(subst, t1)
    let t2 = apply(subst, t2)
    match (t1, t2) {
        (Type.Var(a), Type.Var(b)) if a == b => Ok(()),
        (Type.Var(a), t) | (t, Type.Var(a)) => {
            if occurs(a, t) { return Err(TypeError.InfiniteType(a, t)) }
            subst.insert(a, t); Ok(())
        },
        (Type.App(f1, a1), Type.App(f2, a2)) => {
            unify(f1, f2, subst)?;
            unify(a1, a2, subst)
        },
        _ => Err(TypeError.Mismatch(t1, t2)),
    }
}
```

Pass Input Output Time% Incremental

Lexer source text tokens 5% no

Parser tokens CST 8% file

Name res CST resolved AST 4% module

Type check resolved AST typed AST 28% interface

IR gen typed AST SSA IR 5% module

Optimize SSA IR opt IR 15% module

Codegen opt IR machine code 30% module

Link objects binary 5% changed

63. Compiler Testing: Property-Based Tests

Hindley-Milner type inference, formalized as Algorithm W, is the theoretical foundation for Lateralus's type checker. The algorithm processes expressions in a single pass, generating type constraints (equations between type variables) and then solving them using unification. Unification finds the most general substitution that satisfies all constraints simultaneously. The result is that every expression in a Lateralus program receives its most general type without any programmer-provided annotations. Annotations, when provided, are checked against the inferred type and produce an error if they are more restrictive or less restrictive than the inferred type.

Single Static Assignment (SSA) form is the intermediate representation used in Lateralus's optimizer. In SSA, every variable is assigned exactly once, and uses that occur after a control flow merge point are mediated by phi functions that select the correct value based on which predecessor block was executed. SSA makes data flow explicit: from any use of a variable, you can trace a single chain of definitions to find where the value came from. This property enables efficient implementations of dominance-based optimizations: constant propagation (replace uses of a constant-valued variable with the constant), dead code elimination (remove assignments whose values are never used), and value numbering (identify equivalent expressions).

Register allocation is the process of mapping IR variables to physical machine registers. Modern CPUs have 16-32 general-purpose registers; programs may use thousands of variables. When there are more live variables at a point than available registers, some variables must be 'spilled' to the stack. The goal of register allocation is to minimize spills. Lateralus uses the linear scan register allocator for its speed (O(n) in the number of instructions) and the graph-coloring allocator for its quality (minimizing spills for hot paths). The two allocators are applied in sequence: the graph-coloring allocator handles the function body, and linear scan handles critical loops.

Implementation Details

The Lateralus Bytecode (LBC) format is designed for the Lateralus virtual machine. Instructions are 32-bit words: the top 8 bits identify the opcode, and the remaining 24 bits carry operands (register numbers, constant pool indices, jump offsets). The fixed-width instruction format allows O(1) random access to any instruction, which is important for the JIT compiler's code patching mechanism. The constant pool stores all non-integer literals (floating-point numbers, strings, type descriptors) referenced by instructions. A separate debug section stores source file names, line numbers, and local variable names; this section can be stripped for production builds.

Bootstrapping a compiler means writing the compiler in the language it compiles. The process has three stages: Stage 1 is the initial implementation in a different language (Python for Lateralus); Stage 2 compiles the Lateralus compiler source with Stage 1 to produce a binary; Stage 3 re-compiles the Lateralus compiler source with Stage 2 and verifies that the Stage 2 and Stage 3 binaries are identical. This three-stage bootstrap, known as the Trusting Trust test, verifies that the compiler correctly compiles itself. Any discrepancy between Stage 2 and Stage 3 indicates a bug in the compiler's self-compilation path.

```
// SSA IR: a simple example
// Source: fn f(x: int) -> int { if x > 0 { x * 2 } else { -x } }
// SSA IR:
//   block entry(x_0: int):
//     t_0 = cmp_gt x_0, 0
//     branch t_0, then_block, else_block
//   block then_block():
//     t_1 = mul x_0, 2
//     jump merge_block(t_1)
//   block else_block():
//     t_2 = neg x_0
//     jump merge_block(t_2)
//   block merge_block(t_3: int):   // phi node
//     return t_3
```

Pass Input Output Time% Incremental

Lexer source text tokens 5% no

Parser tokens CST 8% file

Name res CST resolved AST 4% module

Type check resolved AST typed AST 28% interface

IR gen typed AST SSA IR 5% module

Optimize SSA IR opt IR 15% module

Codegen opt IR machine code 30% module

Link objects binary 5% changed

64. Compiler Performance: Profiling Results

Single Static Assignment (SSA) form is the intermediate representation used in Lateralus's optimizer. In SSA, every variable is assigned exactly once, and uses that occur after a control flow merge point are mediated by phi functions that select the correct value based on which predecessor block was executed. SSA makes data flow explicit: from any use of a variable, you can trace a single chain of definitions to find where the value came from. This property enables efficient implementations of dominance-based optimizations: constant propagation (replace uses of a constant-valued variable with the constant), dead code elimination (remove assignments whose values are never used), and value numbering (identify equivalent expressions).

Register allocation is the process of mapping IR variables to physical machine registers. Modern CPUs have 16-32 general-purpose registers; programs may use thousands of variables. When there are more live variables at a point than available registers, some variables must be 'spilled' to the stack. The goal of register allocation is to minimize spills. Lateralus uses the linear scan register allocator for its speed (O(n) in the number of instructions) and the graph-coloring allocator for its quality (minimizing spills for hot paths). The two allocators are applied in sequence: the graph-coloring allocator handles the function body, and linear scan handles critical loops.

The Lateralus Bytecode (LBC) format is designed for the Lateralus virtual machine. Instructions are 32-bit words: the top

8 bits identify the opcode, and the remaining 24 bits carry operands (register numbers, constant pool indices, jump offsets). The fixed-width instruction format allows O(1) random access to any instruction, which is important for the JIT compiler's code patching mechanism. The constant pool stores all non-integer literals (floating-point numbers, strings, type descriptors) referenced by instructions. A separate debug section stores source file names, line numbers, and local variable names; this section can be stripped for production builds.

Implementation Details

Bootstrapping a compiler means writing the compiler in the language it compiles. The process has three stages: Stage 1 is the initial implementation in a different language (Python for Lateralus); Stage 2 compiles the Lateralus compiler source with Stage 1 to produce a binary; Stage 3 re-compiles the Lateralus compiler source with Stage 2 and verifies that the Stage 2 and Stage 3 binaries are identical. This three-stage bootstrap, known as the Trusting Trust test, verifies that the compiler correctly compiles itself. Any discrepancy between Stage 2 and Stage 3 indicates a bug in the compiler's self-compilation path.

Incremental compilation avoids recompiling the entire program when a single file changes. Lateralus's module system makes incremental compilation tractable: each module has a well-defined interface (the set of exported types and functions), and changes to a module's implementation that do not change its interface do not require recompilation of dependent modules. The compiler maintains a dependency graph and a cache of compiled module interfaces. When a source file changes, the compiler recompiles that module, checks whether its interface changed, and if so, propagates recompilation to dependent modules. For large codebases, incremental compilation reduces rebuild time from minutes to seconds.

```
// LBC bytecode instruction encoding
// Bits: [31:24]=opcode [23:16]=dst [15:8]=src1 [7:0]=src2
const OP_ADD:  u8 = 0x01   // dst = src1 + src2
const OP_SUB:  u8 = 0x02   // dst = src1 - src2
const OP_MUL:  u8 = 0x03   // dst = src1 * src2
const OP_CALL: u8 = 0x10   // call func at const_pool[src1], argc=src2
const OP_RET:  u8 = 0x11   // return src1
const OP_JMP:  u8 = 0x20   // unconditional jump to offset
```

```
const OP_JZ:   u8 = 0x21   // jump if src1 == 0

fn encode(op: u8, dst: u8, s1: u8, s2: u8) -> u32 {
    return (op as u32 << 24) | (dst as u32 << 16) | (s1 as u32 << 8) | s2 as u32
}
```

Pass Input Output Time% Incremental

Lexer source text tokens 5% no

Parser tokens CST 8% file

Name res CST resolved AST 4% module

Type check resolved AST typed AST 28% interface

IR gen typed AST SSA IR 5% module

Optimize SSA IR opt IR 15% module

Codegen opt IR machine code 30% module

Link objects binary 5% changed

65. Compiler: Macro System Design

Register allocation is the process of mapping IR variables to physical machine registers. Modern CPUs have 16-32 general-purpose registers; programs may use thousands of variables. When there are more live variables at a point than available registers, some variables must be 'spilled' to the stack. The goal of register allocation is to minimize spills. Lateralus uses the linear scan register allocator for its speed (O(n) in the number of instructions) and the graph-coloring allocator for its quality (minimizing spills for hot paths). The two allocators are applied in sequence: the graph-coloring allocator handles the function body, and linear scan handles critical loops.

The Lateralus Bytecode (LBC) format is designed for the Lateralus virtual machine. Instructions are 32-bit words: the top 8 bits identify the opcode, and the remaining 24 bits carry operands (register numbers, constant pool indices, jump

offsets). The fixed-width instruction format allows O(1) random access to any instruction, which is important for the JIT compiler's code patching mechanism. The constant pool stores all non-integer literals (floating-point numbers, strings, type descriptors) referenced by instructions. A separate debug section stores source file names, line numbers, and local variable names; this section can be stripped for production builds.

Bootstrapping a compiler means writing the compiler in the language it compiles. The process has three stages: Stage 1 is the initial implementation in a different language (Python for Lateralus); Stage 2 compiles the Lateralus compiler source with Stage 1 to produce a binary; Stage 3 re-compiles the Lateralus compiler source with Stage 2 and verifies that the Stage 2 and Stage 3 binaries are identical. This three-stage bootstrap, known as the Trusting Trust test, verifies that the compiler correctly compiles itself. Any discrepancy between Stage 2 and Stage 3 indicates a bug in the compiler's self-compilation path.

Implementation Details

Incremental compilation avoids recompiling the entire program when a single file changes. Lateralus's module system makes incremental compilation tractable: each module has a well-defined interface (the set of exported types and functions), and changes to a module's implementation that do not change its interface do not require recompilation of dependent modules. The compiler maintains a dependency

graph and a cache of compiled module interfaces. When a source file changes, the compiler recompiles that module, checks whether its interface changed, and if so, propagates recompilation to dependent modules. For large codebases, incremental compilation reduces rebuild time from minutes to seconds.

Code generation translates the high-level SSA IR into machine code for a specific target architecture. Lateralus supports four backends: Python (transpilation to Python source, for portability), C (transpilation to C99, for maximum compatibility), RISC-V assembly (for the LateralusOS target and embedded systems), and LBC bytecode (for the Lateralus VM). Each backend performs instruction selection (choosing machine instructions for IR operations), register allocation (mapping IR values to machine registers or stack slots), and instruction scheduling (reordering instructions to avoid pipeline stalls). The C backend is the most commonly used for production deployments, as it leverages mature C compilers like GCC and Clang for the final machine code generation.

```
// Lexer token types
enum Token {
    Ident(str), IntLit(int), FloatLit(float), StrLit(str),
    Plus, Minus, Star, Slash, PipeFwd,  // |>
    BitorOp,                            // |
    Assign, Eq, Neq, Lt, Gt, Le, Ge,
    LParen, RParen, LBrace, RBrace,
    Fn, Let, If, Else, Return, Match, Struct, Enum,
    Eof,
}

fn next_token(src: &str, pos: &mut int) -> Token {
    skip_whitespace(src, pos)
    match src[*pos] {
        '|' => { *pos += 1
                 if src[*pos] == '>' { *pos += 1; return Token.PipeFwd }
                 return Token.BitorOp },
        // ...
    }
}
```

Pass Input Output Time% Incremental

Lexer source text tokens 5% no

Parser tokens CST 8% file

Name res CST resolved AST 4% module

Type check resolved AST typed AST 28% interface

IR gen typed AST SSA IR 5% module

Optimize SSA IR opt IR 15% module

Codegen opt IR machine code 30% module

Link objects binary 5% changed

66. Conclusion

The Lateralus Bytecode (LBC) format is designed for the Lateralus virtual machine. Instructions are 32-bit words: the top 8 bits identify the opcode, and the remaining 24 bits carry operands (register numbers, constant pool indices, jump offsets). The fixed-width instruction format allows O(1) random access to any instruction, which is important for the JIT compiler's code patching mechanism. The constant pool stores all non-integer literals (floating-point numbers, strings, type descriptors) referenced by instructions. A separate debug section stores source file names, line numbers, and local variable names; this section can be stripped for production builds.

Bootstrapping a compiler means writing the compiler in the language it compiles. The process has three stages: Stage 1 is the initial implementation in a different language (Python for Lateralus); Stage 2 compiles the Lateralus compiler source with Stage 1 to produce a binary; Stage 3 re-compiles the Lateralus compiler source with Stage 2 and verifies that the Stage 2 and Stage 3 binaries are identical. This three-stage bootstrap, known as the Trusting Trust test, verifies that the compiler correctly compiles itself. Any discrepancy between Stage 2 and Stage 3 indicates a bug in the compiler's self-compilation path.

Incremental compilation avoids recompiling the entire program when a single file changes. Lateralus's module system makes incremental compilation tractable: each module has a well-defined interface (the set of exported types and functions), and changes to a module's implementation that do not change its interface do not require recompilation of dependent modules. The compiler maintains a dependency graph and a cache of compiled module interfaces. When a source file changes, the compiler recompiles that module, checks whether its interface changed, and if so, propagates recompilation to dependent modules. For large codebases, incremental compilation reduces rebuild time from minutes to seconds.

Implementation Details

Code generation translates the high-level SSA IR into machine code for a specific target architecture. Lateralus supports four backends: Python (transpilation to Python source, for portability), C (transpilation to C99, for maximum compatibility), RISC-V assembly (for the LateralusOS target and embedded systems), and LBC bytecode (for the Lateralus VM). Each backend performs instruction selection (choosing machine instructions for IR operations), register allocation (mapping IR values to machine registers or stack slots), and instruction scheduling (reordering instructions to avoid pipeline stalls). The C backend is the most commonly used for production deployments, as it leverages mature C compilers like GCC and Clang for the final machine code generation.

Compiler construction sits at the intersection of formal language theory and practical systems engineering. The theoretical foundations -- regular languages for lexing, context-free grammars for parsing, type theory for semantic analysis -- give compiler writers the tools to reason about correctness. The engineering challenges -- performance, error recovery, incremental compilation, cross-platform code generation -- require the tools of systems programming. Lateralus's compiler is implemented in Lateralus itself (bootstrapped from an initial Python implementation), which means the compiler is both a producer and consumer of all the language features it compiles.

```
// Pratt parser: precedence climbing
const PREC: map = {
    "+": 10, "-": 10, "*": 20, "/": 20,
    "|>": 2, "==": 5, "!=": 5, "<": 5, ">": 5,
    ".": 30,  // field access
}

fn parse_expr(p: &Parser, min_prec: int) -> Expr {
    let mut left = parse_primary(p)
    while p.peek().prec() >= min_prec {
```

```
        let op = p.consume()
        let right = parse_expr(p, PREC[op.text] + 1)
        left = Expr.BinOp(op.kind, left, right)
    }
    return left
}
```

Pass Input Output Time% Incremental

Lexer source text tokens 5% no

Parser tokens CST 8% file

Name res CST resolved AST 4% module

Type check resolved AST typed AST 28% interface

IR gen typed AST SSA IR 5% module

Optimize SSA IR opt IR 15% module

Codegen opt IR machine code 30% module

Link objects binary 5% changed

67. Appendix: Grammar (EBNF)

Bootstrapping a compiler means writing the compiler in the language it compiles. The process has three stages: Stage 1 is the initial implementation in a different language (Python for Lateralus); Stage 2 compiles the Lateralus compiler source with Stage 1 to produce a binary; Stage 3 re-compiles the Lateralus compiler source with Stage 2 and verifies that the Stage 2 and Stage 3 binaries are identical. This three-stage bootstrap, known as the Trusting Trust test, verifies that the compiler correctly compiles itself. Any discrepancy between Stage 2 and Stage 3 indicates a bug in the compiler's self-compilation path.

Incremental compilation avoids recompiling the entire program when a single file changes. Lateralus's module system makes incremental compilation tractable: each module has a well-defined interface (the set of exported types and functions), and changes to a module's implementation that do not change its interface do not require recompilation of dependent modules. The compiler maintains a dependency graph and a cache of compiled module interfaces. When a source file changes, the compiler recompiles that module, checks whether its interface changed, and if so, propagates recompilation to dependent modules. For large codebases, incremental compilation reduces rebuild time from minutes to seconds.

Code generation translates the high-level SSA IR into machine code for a specific target architecture. Lateralus supports four backends: Python (transpilation to Python source, for portability), C (transpilation to C99, for maximum compatibility), RISC-V assembly (for the LateralusOS target and embedded systems), and LBC bytecode (for the Lateralus VM). Each backend performs instruction selection (choosing machine instructions for IR operations), register allocation (mapping IR values to machine registers or stack slots), and instruction scheduling (reordering instructions to avoid pipeline stalls). The C backend is the most commonly used for production deployments, as it leverages mature C compilers like GCC and Clang for the final machine code generation.

Implementation Details

Compiler construction sits at the intersection of formal language theory and practical systems engineering. The theoretical foundations -- regular languages for lexing, context-free grammars for parsing, type theory for semantic analysis -- give compiler writers the tools to reason about correctness.

The engineering challenges -- performance, error recovery, incremental compilation, cross-platform code generation -- require the tools of systems programming. Lateralus's compiler is implemented in Lateralus itself (bootstrapped from an initial Python implementation), which means the compiler is both a producer and consumer of all the language features it compiles.

The lexer transforms a stream of characters into a stream of tokens. Each token has a type (IDENT, INT_LIT, STRING, PLUS, PIPEFWD, etc.) and a value (the matched text). Lateralus's lexer handles several non-trivial features: string interpolation (parsing 'Hello, ${name}!' into a sequence of string literal, expression, and string literal tokens), multi-line strings (tracking line numbers for accurate error reporting), and the pipeline operator |> (which must be distinguished from the bitwise-or operator | followed by >). The lexer is implemented as a handwritten DFA for maximum performance, achieving about 50MB/s throughput on typical source files.

```
// Hindley-Milner unification
fn unify(t1: Type, t2: Type, subst: &mut Subst) -> Result[(), TypeError] {
    let t1 = apply(subst, t1)
    let t2 = apply(subst, t2)
    match (t1, t2) {
        (Type.Var(a), Type.Var(b)) if a == b => Ok(()),
        (Type.Var(a), t) | (t, Type.Var(a)) => {

            if occurs(a, t) { return Err(TypeError.InfiniteType(a, t)) }
            subst.insert(a, t); Ok(())
        },
        (Type.App(f1, a1), Type.App(f2, a2)) => {
            unify(f1, f2, subst)?;
            unify(a1, a2, subst)
        },
        _ => Err(TypeError.Mismatch(t1, t2)),
    }
}
```

Pass Input Output Time% Incremental

Lexer source text tokens 5% no

Parser tokens CST 8% file

Name res CST resolved AST 4% module

Type check resolved AST typed AST 28% interface

IR gen typed AST SSA IR 5% module

Optimize SSA IR opt IR 15% module

Codegen opt IR machine code 30% module

Link objects binary 5% changed

68. Appendix: IR Instruction Reference

Incremental compilation avoids recompiling the entire program when a single file changes. Lateralus's module system makes incremental compilation tractable: each module has a well-defined interface (the set of exported types and functions), and changes to a module's implementation that do not change its interface do not require recompilation of dependent modules. The compiler maintains a dependency graph and a cache of compiled module interfaces. When a source file changes, the compiler recompiles that module, checks whether its interface changed, and if so, propagates recompilation to dependent modules. For large codebases, incremental compilation reduces rebuild time from minutes to seconds.

Code generation translates the high-level SSA IR into machine code for a specific target architecture. Lateralus supports four backends: Python (transpilation to Python source, for portability), C (transpilation to C99, for maximum compatibility), RISC-V assembly (for the LateralusOS target and embedded systems), and LBC bytecode (for the Lateralus VM). Each backend performs instruction selection (choosing machine instructions for IR operations), register allocation (mapping IR values to machine registers or stack slots), and instruction scheduling (reordering instructions to avoid pipeline stalls). The C backend is the most commonly used for production deployments, as it leverages mature C compilers like GCC and Clang for the final machine code generation.

Compiler construction sits at the intersection of formal language theory and practical systems engineering. The theoretical foundations -- regular languages for lexing, context-free grammars for parsing, type theory for semantic analysis -- give compiler writers the tools to reason about correctness. The engineering challenges -- performance, error recovery, incremental compilation, cross-platform code generation -- require the tools of systems programming. Lateralus's compiler is implemented in Lateralus itself (bootstrapped from an initial Python implementation), which means the compiler is both a producer and consumer of all the language features it compiles.

Implementation Details

The lexer transforms a stream of characters into a stream of tokens. Each token has a type (IDENT, INT_LIT, STRING, PLUS, PIPEFWD, etc.) and a value (the matched text). Lateralus's lexer handles several non-trivial features: string interpolation (parsing 'Hello, ${name}!' into a sequence of string literal, expression, and string literal tokens), multi-line strings (tracking line numbers for accurate error reporting), and the pipeline operator |> (which must be distinguished from the bitwise-or operator | followed by >). The lexer is implemented as a handwritten DFA for maximum performance, achieving about 50MB/s throughput on typical source files.

Pratt parsing, also called top-down operator precedence parsing, is the technique used in Lateralus's expression parser. Each token type is assigned a binding power -- a number representing its precedence. The parser is a loop: parse a primary expression (literal, identifier, parenthesized expression), then repeatedly consume infix operators whose binding

power exceeds the current right-binding context. This naturally handles operator precedence and associativity without explicit grammar rules for each precedence level. The Pratt parser for Lateralus handles 25 infix operators, 8 prefix operators, and 5 postfix operators in about 300 lines of code.

```
// SSA IR: a simple example
// Source: fn f(x: int) -> int { if x > 0 { x * 2 } else { -x } }
// SSA IR:
//   block entry(x_0: int):
//     t_0 = cmp_gt x_0, 0
//     branch t_0, then_block, else_block
//   block then_block():
//     t_1 = mul x_0, 2
//     jump merge_block(t_1)
```

```
//   block else_block():
//     t_2 = neg x_0
//     jump merge_block(t_2)
//   block merge_block(t_3: int):   // phi node
//     return t_3
```

Pass Input Output Time% Incremental

Lexer source text tokens 5% no

Parser tokens CST 8% file

Name res CST resolved AST 4% module

Type check resolved AST typed AST 28% interface

IR gen typed AST SSA IR 5% module

Optimize SSA IR opt IR 15% module

Codegen opt IR machine code 30% module

Link objects binary 5% changed

Lateralus is an open-source, zero-dependency programming language. Project home: https://lateralus.dev. Source: github.com/bad-antics/lateralus-lang. Released under CC BY 4.0.